broker_server

Project Documentation

Generated on: 2026-08-21 22:47
Total Files: 52

Table of Contents

26	main_server\core\position_keeper.py	
27	main_server\core\tick_center.py	
28	main_server\core\tick_publisher.py	
29	main_server\core\trade_center.py	
30	main_server\core\interfaces\i_event_bus.py	
31	main_server\core\interfaces\i_matching.py	
32	main_server\core\interfaces\i_oms.py	
33	main_server\core\interfaces\i_position_keeper.py	
34	main_server\core\interfaces\i_rms.py	
35	main_server\feeds\mock_feed.py	
36	main_server\matching\engine.py	
37	main_server\rms\accounting.py	
38	main_server\rms\engine.py	
39	main_server\routing\engine.py	
40	main_server\shared\event_bus.py	
41	main_server\shared\schemas\v1.py	
42	main_server\tests\test_accounts_crud.py	
43	main_server\tests\test_bugs.py	
44	main_server\tests\test_group_name_validation.py	
45	main_server\tests\test_margin.py	
46	main_server\tests\test_phase2_ticks.py	
47	main_server\tests\test_phase3.py	
48	main_server\tests\test_phase4.py	
49	main_server\tests\test_phase6.py	
50	main_server\tests\test_phase7.py	
51	main_server\tests\test_trade.py	
52	main_server\webapi\manager_api.py	

Project Structure

```
broker_server/
■■■ access_server/
■   ■■■ access_main.py
■   ■■■ config/
■   ■   ■■■ access.ini
■   ■■■ logs/
■■■ history_server/
■   ■■■ bar_aggregator.py
■   ■■■ bases/
■   ■   ■■■ connection.py
■   ■   ■■■ history.db
■   ■   ■■■ schema.sql
■   ■   ■■■ schema_sqlite.sql
■   ■■■ config/
■   ■■■ Datafeeds/
■   ■   ■■■ mt5_feed.py
■   ■■■ Gateways/
■   ■   ■■■ mt5_gateway.py
■   ■■■ history_main.py
■   ■■■ logs/
■   ■■■ scratch/
■       ■■■ test_feeder_scenarios.py
■       ■■■ test_frontend_logic.py
■■■ history_server.log
■■■ main_server/
■   ■■■ archive/
■   ■■■ bases/
■   ■   ■■■ broker.db
■   ■   ■■■ connection.py
■   ■   ■■■ schema.sql
■   ■   ■■■ schema_sqlite.sql
■   ■■■ config/
■   ■■■ core/
■   ■   ■■■ deal_writer.py
■   ■   ■■■ interfaces/
■   ■   ■   ■■■ i_event_bus.py
■   ■   ■   ■■■ i_matching.py
■   ■   ■   ■■■ i_oms.py
■   ■   ■   ■■■ i_position_keeper.py
■   ■   ■   ■■■ i_rms.py
■   ■   ■■■ position_keeper.py
■   ■   ■■■ tick_center.py
■   ■   ■■■ tick_publisher.py
■   ■   ■■■ trade_center.py
■   ■■■ feeds/
■   ■   ■■■ mock_feed.py
■   ■■■ logs/
■   ■■■ main.py
■   ■■■ main_server.log
■   ■■■ matching/
■   ■   ■■■ engine.py
■   ■■■ requirements.txt
■   ■■■ rms/
```

```
■   ■   ■■■ accounting.py
■   ■   ■■■ engine.py
■   ■■■ routing/
■   ■   ■■■ engine.py
■   ■■■ shared/
■   ■   ■■■ event_bus.py
■   ■   ■■■ schemas/
■   ■       ■■■ v1.py
■   ■■■ test_accounts.db
■   ■■■ test_phase7_server.db
■   ■■■ tests/
■   ■   ■■■ test_accounts_crud.py
■   ■   ■■■ test_bugs.py
■   ■   ■■■ test_group_name_validation.py
■   ■   ■■■ test_margin.py
■   ■   ■■■ test_phase2_ticks.py
■   ■   ■■■ test_phase3.py
■   ■   ■■■ test_phase4.py
■   ■   ■■■ test_phase6.py
■   ■   ■■■ test_phase7.py
■   ■   ■■■ test_trade.py
■   ■■■ trade_server.db
■   ■■■ webapi/
■       ■■■ manager_api.py
■■■ requirements.txt
```

■ history_server.log

```
u■v■i■c■o■r■n■ ■:■ ■I■N■F■O■:■ ■ ■ ■ ■ ■ ■S■t■a■r■t■e■d■ ■s■e■r■v■e■r■ ■p■r■o■c■e■s■s■ ■[■2■0■9■
■
■A■t■ ■l■i■n■e■:■1■ ■c■h■a■r■:■1■
■
■+■ ■u■v■i■c■o■r■n■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■h■i■s■t■o■r■y■_■m■a■i■n■:■a■p■p■ ■-■-■h■o■
■
■+■ ■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■~■
■
■ ■ ■ ■ ■+■ ■C■a■t■e■g■o■r■y■I■n■f■o■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■:■ ■N■o■t■S■p■e■c■i■f■i■e■d■:■ ■(■I■
■
■ ■ ■ ■ ■+■ ■F■u■l■l■y■Q■u■a■l■i■f■i■e■d■E■r■r■o■r■I■d■ ■:■ ■N■a■t■i■v■e■C■o■m■m■a■n■d■E■r■r■
■
■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■ ■W■a■i■t■i■n■g■ ■f■o■r■ ■a■p■p■l■i■c■a■t■i■o■n■ ■s■t■a■r■t■u■p■.■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■0■9■3■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■b■a■s■e■s■
■
■C■:■\■U■s■e■r■s■\■D■E■L■L■\■D■o■w■n■l■o■a■d■s■\■s■e■r■v■e■r■3■\■b■r■o■k■e■r■_■s■e■r■v■e■r■\■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■0■9■3■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■ ■-■ ■I■N■F■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■0■9■3■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■0■9■3■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■ ■-■ ■I■N■F■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■0■9■3■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■ ■A■p■p■l■i■c■a■t■i■o■n■ ■s■t■a■r■t■u■p■ ■c■o■m■p■l■e■t■e■.■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■ ■U■v■i■c■o■r■n■ ■r■u■n■n■i■n■g■ ■o■n■ ■h■t■t■p■:■/■/■0■.■0■.■0■.■0■:■8■0■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■4■2■2■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■4■2■2■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■h■t■t■p■:■/■/■1■2■7■.■0■.■0■.■1■:■8■0■0■3■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■0■,■4■2■8■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■f■o■r■ ■l■o■g■i■n■ ■5■0■0■8■0■.■.■.■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■4■,■8■0■1■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■2■5■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■ ■p■l■a■t■f■o■r■m■ ■s■y■m■b■o■l■s■.■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■2■6■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
```

■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■2■0■2■6■-■0■8■-■2■1■ ■1■4■:■0■0■:■4■5■,■1■3■9■ ■-■ ■h■i■s■t■o■r■y■_■s■e■r■v■e■r■.■m■t■5■_■f■
■
■

■ requirements.txt

```
fastapi
uvicorn[standard]
asyncpg
pydantic
numpy
orjson
uvloop
pytest
pytest-asyncio
```

■ access_server\access_main.py

```
import os
import time
import json
import logging
import asyncio
import httpx
import redis
import configparser
import websockets
from fastapi import FastAPI, HTTPException, Request, Response, WebSocket, WebSocketDisconnect
from fastapi.responses import ORJSONResponse, Response
from typing import Dict, List, Any

# Setup Logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(name)s - %(levelname)s - %(message)s")
logger = logging.getLogger("access_server")
INTERNAL_SECRET = os.getenv("INTERNAL_SECRET", "default_internal_secret_token_change_in_production")
if INTERNAL_SECRET == "default_internal_secret_token_change_in_production":
    logger.warning("\n" + "="*80 + "\n" +
                   "  ■■  WARNING: DEFAULT INSECURE INTERNAL_SECRET IS ACTIVE! ■■\n" +
                   "  Please set the INTERNAL_SECRET environment variable in production.\n" +
                   "="*80 + "\n")

app = FastAPI(
    title="Broker Access Server Proxy"
)

# Load Configuration
config = configparser.ConfigParser()
config_path = os.path.join(os.path.dirname(__file__), "config", "access.ini")
if os.path.exists(config_path):
    config.read(config_path)
else:
    logger.warning(f"access.ini not found at {config_path}. Using default configuration.")

# Parse Firewall Config
firewall_sec = config["firewall"] if "firewall" in config else {}
allow_ips = [ip.strip() for ip in firewall_sec.get("allow_ips", "127.0.0.1,::1,localhost").split(",") if i
deny_ips = [ip.strip() for ip in firewall_sec.get("deny_ips", "192.168.1.50").split(",") if ip.strip()]

# Parse Rate Limit Config
rate_limit_sec = config["rate_limit"] if "rate_limit" in config else {}
window_seconds = int(rate_limit_sec.get("window_seconds", "60"))
max_requests = int(rate_limit_sec.get("max_requests", "100"))

# Parse Upstreams Config
upstreams_sec = config["upstreams"] if "upstreams" in config else {}
upstreams = {k.upper(): v for k, v in upstreams_sec.items()}
if "DEFAULT" not in upstreams:
    upstreams["DEFAULT"] = "localhost:8000"

# Redis-backed session helper inside Access Server
class RedisSessionReader:
    def __init__(self, redis_host="localhost", redis_port=6379, use_redis=True):
        self.use_redis = use_redis
        self.r = None
        self.last_redis_failure = 0.0
        self.retry_cooldown = 10.0 # seconds
        if self.use_redis:
            self.r = redis.Redis(
                host=redis_host,
                port=redis_port,
                decode_responses=True,
                socket_connect_timeout=0.5,
                socket_timeout=0.5
```

```
            )

    def validate_session(self, token: str) -&gt; int:
        if self.r and (time.time() - self.last_redis_failure &gt; self.retry_cooldown):
            try:
                data = self.r.get(f"session:{token}")
                if data:
                    session_data = json.loads(data)
                    return session_data.get("login")
            except Exception as e:
                logger.error(f"Access Server Redis session lookup failed: {e}. Entering Redis cooldown.")
                self.last_redis_failure = time.time()
        return None

# Sliding window rate limiter
class RateLimiter:
    def __init__(self, max_reqs: int, window: int, redis_host="localhost", redis_port=6379, use_redis=True
        self.max_requests = max_reqs
        self.window_seconds = window
        self.use_redis = use_redis
        self.r = None
        self.last_redis_failure = 0.0
        self.retry_cooldown = 10.0 # seconds
        self.in_memory_hits = {} # key -&gt; list of timestamps
        if self.use_redis:
            self.r = redis.Redis(
                host=redis_host,
                port=redis_port,
                decode_responses=True,
                socket_connect_timeout=0.5,
                socket_timeout=0.5
            )

    def is_rate_limited(self, key: str) -&gt; bool:
        now = time.time()
        cutoff = now - self.window_seconds

        if self.r and (time.time() - self.last_redis_failure &gt; self.retry_cooldown):
            try:
                redis_key = f"rate:{key}"
                pipe = self.r.pipeline()
                pipe.zremrangebyscore(redis_key, 0, cutoff)
                pipe.zadd(redis_key, {str(now): now})
                pipe.zcard(redis_key)
                pipe.expire(redis_key, self.window_seconds)
                res = pipe.execute()
                request_count = res[2]
                return request_count &gt; self.max_requests
            except Exception as e:
                logger.error(f"Access Server Redis rate limiting failed: {e}. Entering Redis cooldown.")
                self.last_redis_failure = time.time()

        # In-memory sliding window fallback
        timestamps = self.in_memory_hits.setdefault(key, [])
        # Clean up old timestamps
        timestamps = [t for t in timestamps if t &gt; cutoff]
        timestamps.append(now)
        self.in_memory_hits[key] = timestamps
        return len(timestamps) &gt; self.max_requests

session_reader = RedisSessionReader()
rate_limiter = RateLimiter(max_requests, window_seconds)

def resolve_upstream(region_param: str, path: str = None) -&gt; str:
    if path and (path.startswith("history") or path.startswith("/history")):
        return upstreams.get("HISTORY", "localhost:8002")
    region_key = region_param.upper() if region_param else "DEFAULT"
    return upstreams.get(region_key, upstreams.get("DEFAULT", "localhost:8000"))
```

```
# Pipeline Verification Helpers
def check_firewall(client_host: str):
    # 1. IP Firewall check
    if client_host in deny_ips:
        raise HTTPException(status_code=403, detail="Forbidden: IP denied by firewall.")
    if allow_ips and "*" not in allow_ips:
        # Allow checking localhost aliases
        aliases = [client_host]
        if client_host == "testclient":
            aliases.append("127.0.0.1")
        if not any(a in allow_ips for a in aliases):
            raise HTTPException(status_code=403, detail="Forbidden: IP not in allow list.")

async def validate_session_token(token: str, upstream: str) -&gt; int:
    # Check Redis fast path
    login = session_reader.validate_session(token)
    if login:
        return login

    # HTTP validation fallback to main_server
    validation_upstream = upstream
    if "8002" in upstream or "history" in upstream:
        validation_upstream = resolve_upstream(None) # fallback to default main_server

    try:
        async with httpx.AsyncClient() as client:
            resp = await client.get(
                f"http://{validation_upstream}/internal/auth/validate",
                params={"token": token},
                headers={"X-Internal-Secret": INTERNAL_SECRET},
                timeout=2.0
            )
            if resp.status_code == 200:
                return resp.json().get("login")
    except Exception as e:
        logger.error(f"HTTP fallback session validation failed: {e}")
    return None

# Catch-all Reverse Proxy Route
@app.api_route("/{path:path}", methods=["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS", "HEAD"])
async def reverse_proxy_handler(request: Request, path: str):
    client_host = request.client.host if request.client else "127.0.0.1"

    # 1. IP Firewall check
    check_firewall(client_host)

    # 2. Rate limiting by IP
    if rate_limiter.is_rate_limited(f"ip:{client_host}"):
        raise HTTPException(status_code=429, detail="Too Many Requests: IP rate limit exceeded.")

    # Determine region from header or query param
    region = request.headers.get("X-Region") or request.query_params.get("region")
    upstream = resolve_upstream(region, path)

    # 3. Session Validation (bypass for public endpoints)
    is_public = path in ("web/auth/login", "admin/setup")
    login = None
    if not is_public:
        auth_header = request.headers.get("Authorization")
        if not auth_header or not auth_header.startswith("Bearer "):
            raise HTTPException(status_code=401, detail="Unauthorized: missing bearer token")
        token = auth_header.split(" ")[1]
        login = await validate_session_token(token, upstream)
        if not login:
            raise HTTPException(status_code=401, detail="Unauthorized: invalid or expired session token")

        # 4. Rate Limiting by Account Login
        if rate_limiter.is_rate_limited(f"login:{login}"):
```

```
            raise HTTPException(status_code=429, detail="Too Many Requests: Account rate limit exceeded.")

    # 5. Forward request to resolved main_server
    body = await request.body()
    # Filter headers to avoid header conflicts
    headers = dict(request.headers)
    headers.pop("host", None)

    upstream_url = f"http://{upstream}/{path}"
    if request.query_params:
        upstream_url += f"?{request.query_params}"

    async with httpx.AsyncClient() as client:
        try:
            resp = await client.request(
                method=request.method,
                url=upstream_url,
                headers=headers,
                content=body,
                timeout=10.0
            )
            return Response(
                content=resp.content,
                status_code=resp.status_code,
                headers=dict(resp.headers)
            )
        except Exception as e:
            logger.error(f"Error forwarding request to upstream {upstream}: {e}")
            raise HTTPException(status_code=502, detail="Bad Gateway: upstream connection failed.")

# WebSocket Reverse Proxy Route
@app.websocket("/{path:path}")
async def ws_proxy_handler(websocket: WebSocket, path: str):
    client_host = websocket.client.host if websocket.client else "127.0.0.1"

    # 1. IP Firewall check
    try:
        check_firewall(client_host)
    except HTTPException as e:
        await websocket.close(code=4003, reason=e.detail)
        return

    # 2. Rate limiting by IP
    if rate_limiter.is_rate_limited(f"ip:{client_host}"):
        await websocket.close(code=4029, reason="IP rate limit exceeded")
        return

    # Determine region from header or query param
    region = websocket.headers.get("X-Region") or websocket.query_params.get("region")
    upstream = resolve_upstream(region, path)

    # 3. Session validation on WebSocket upgrade
    token = websocket.query_params.get("token")
    if not token:
        auth_header = websocket.headers.get("Authorization")
        if auth_header and auth_header.startswith("Bearer "):
            token = auth_header.split(" ")[1]

    if not token:
        await websocket.close(code=4001, reason="Missing token")
        return

    login = await validate_session_token(token, upstream)
    if not login:
        await websocket.close(code=4003, reason="Unauthorized")
        return

    # 4. Rate Limiting by Account Login
```

```
    if rate_limiter.is_rate_limited(f"login:{login}"):
        await websocket.close(code=4029, reason="Account rate limit exceeded")
        return

    # Accept WebSocket client upgrade
    await websocket.accept()

    # Determine upstream WS URL
    query_str = str(websocket.query_params)
    upstream_ws_url = f"ws://{upstream}/{path}"
    if query_str:
        upstream_ws_url += f"?{query_str}"

    # Forward messages bi-directionally
    try:
        async with websockets.connect(upstream_ws_url) as upstream_ws:
            async def client_to_upstream():
                try:
                    async for msg in websocket.iter_text():
                        await upstream_ws.send(msg)
                except Exception as e:
                    if not isinstance(e, WebSocketDisconnect):
                        logger.error(f"Error in client_to_upstream: {e}", exc_info=True)

            async def upstream_to_client():
                try:
                    async for msg in upstream_ws:
                        if isinstance(msg, bytes):
                            msg = msg.decode("utf-8")
                        await websocket.send_text(msg)
                except Exception as e:
                    # Ignore standard connection closed
                    if "ConnectionClosed" not in type(e).__name__:
                        logger.error(f"Error in upstream_to_client: {e}", exc_info=True)

            client_task = asyncio.create_task(client_to_upstream())
            upstream_task = asyncio.create_task(upstream_to_client())

            done, pending = await asyncio.wait(
                [client_task, upstream_task],
                return_when=asyncio.FIRST_COMPLETED
            )

            for task in pending:
                task.cancel()
            if pending:
                await asyncio.gather(*pending, return_exceptions=True)
    except WebSocketDisconnect:
        pass
    except Exception as e:
        logger.error(f"WebSocket proxy exception: {e}")
        try:
            await websocket.close(code=1011, reason="Internal proxy error")
        except Exception:
            pass

@app.on_event("shutdown")
async def shutdown_event():
    logger.info("Access Server shutting down. Closing Redis connection pools...")
    if session_reader.r:
        session_reader.r.close()
    if rate_limiter.r:
        rate_limiter.r.close()
```

■ access_server\config\access.ini

```
[firewall]
allow_ips = 127.0.0.1,::1,localhost
deny_ips = 192.168.1.50

[rate_limit]
window_seconds = 60
max_requests = 100

[upstreams]
default = localhost:8000
US = localhost:8000
EU = localhost:8000
history = localhost:8002
```

■ history_server\bar_aggregator.py

```
import logging
from datetime import datetime, timezone
from decimal import Decimal
from typing import Dict, Tuple, Any

logger = logging.getLogger(__name__)

def floor_to_period(dt: datetime, tf: str) -&gt; datetime:
    # Ensure dt is timezone-aware
    if dt.tzinfo is None:
        dt = dt.replace(tzinfo=timezone.utc)

    if tf == '1m':
        return dt.replace(second=0, microsecond=0)
    elif tf == '5m':
        minute = dt.minute - (dt.minute % 5)
        return dt.replace(minute=minute, second=0, microsecond=0)
    elif tf == '1h':
        return dt.replace(minute=0, second=0, microsecond=0)
    elif tf == '1d':
        return dt.replace(hour=0, minute=0, second=0, microsecond=0)
    else:
        raise ValueError(f"Unsupported timeframe: {tf}")

class Bar:
    def __init__(self, time: datetime, symbol: str, val: Decimal):
        self.time = time
        self.symbol = symbol
        self.open = val
        self.high = val
        self.low = val
        self.close = val
        self.tick_volume = 1

class BarAggregator:
    def __init__(self):
        # Maps (symbol, timeframe) -&gt; Bar
        self.current_bars: Dict[Tuple[str, str], Bar] = {}
        self.timeframes = ['1m', '5m', '1h', '1d']

    async def on_tick(self, tick_time: datetime, symbol: str, bid: Decimal, conn: Any) -&gt; None:
        """
        Process tick price update to aggregate OHLCV bars.
        When period boundary is crossed, completed bars are flushed to bars_{timeframe} tables.
        """
        for tf in self.timeframes:
            key = (symbol, tf)
            period_start = floor_to_period(tick_time, tf)

            if key not in self.current_bars:
                # Start new in-progress bar
                self.current_bars[key] = Bar(period_start, symbol, bid)
            else:
                bar = self.current_bars[key]
                if period_start &gt; bar.time:
                    # Flush old bar to DB
                    await self._flush_bar_to_db(bar, tf, conn)
                    # Start new bar
                    self.current_bars[key] = Bar(period_start, symbol, bid)
                elif period_start == bar.time:
                    # Update current in-progress bar
                    bar.high = max(bar.high, bid)
                    bar.low = min(bar.low, bid)
                    bar.close = bid
                    bar.tick_volume += 1
```

```
            else:
                # Late tick (historical/out-of-order) - ignore or log
                logger.debug(f"Late tick received for {symbol} ({tf}): tick_time={tick_time}, bar_time

async def _flush_bar_to_db(self, bar: Bar, tf: str, conn: Any) -&gt; None:
    """Flushes a completed bar to the bars_{tf} table using database-agnostic insert/update."""
    query = f"""
        INSERT INTO bars_{tf} (time, symbol, open, high, low, close, tick_volume)
        VALUES ($1, $2, $3, $4, $5, $6, $7)
        ON CONFLICT(time, symbol) DO UPDATE SET
            open = EXCLUDED.open,
            high = EXCLUDED.high,
            low = EXCLUDED.low,
            close = EXCLUDED.close,
            tick_volume = EXCLUDED.tick_volume
    """
    try:
        # We must serialize datetime to ISO string for SQLite compatibility if it's SQLite
        # DBConnection connection wrappers handle SQL parsing, but datetime parameters must be formatt
        time_str = bar.time.isoformat()
        await conn.execute(
            query,
            time_str,
            bar.symbol,
            bar.open,
            bar.high,
            bar.low,
            bar.close,
            bar.tick_volume
        )
        logger.debug(f"Flushed completed bar for {bar.symbol} ({tf}) at {bar.time}")
    except Exception as e:
        logger.error(f"Error flushing bar to DB for {bar.symbol} ({tf}): {e}")
```

■ history_server\history_main.py

```
import os
import sys
import json
import secrets
import logging
from datetime import datetime, timezone, timedelta
from decimal import Decimal
from typing import List, Optional, Dict, Any

from fastapi import FastAPI, WebSocket, WebSocketDisconnect, HTTPException, Depends, Query, Request
from fastapi.responses import ORJSONResponse, Response
from pydantic import BaseModel
import redis
import httpx

# Add current folder to path to import local modules
sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from history_server.bases.connection import DBConnection
from bar_aggregator import BarAggregator, floor_to_period
from Datafeeds.mt5_feed import MT5Feed
from Gateways.mt5_gateway import MT5Gateway
# -----------------------------------------------------------------------------
import logging

logging.getLogger("httpx").setLevel(logging.WARNING)
# -----------------------------------------------------------------------------

# Setup logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(name)s - %(levelname)s - %(message)s")
logger = logging.getLogger("history_server")

INTERNAL_SECRET = os.getenv("INTERNAL_SECRET", "default_internal_secret_token_change_in_production")
ADMIN_API_KEY = os.getenv("ADMIN_API_KEY", "default_admin_api_key_token_change_in_production")

app = FastAPI(
    title="Broker History Server"
)

# Global instances
bar_aggregator = BarAggregator()
mt5_feed = MT5Feed()
mt5_gateway = MT5Gateway()

# Redis session validation helper
class RedisSessionReader:
    def __init__(self, redis_host="localhost", redis_port=6379, use_redis=True):
        self.use_redis = use_redis
        self.r = None
        self.last_redis_failure = 0.0
        self.retry_cooldown = 10.0  # seconds
        if self.use_redis:
            self.r = redis.Redis(
                host=redis_host,
                port=redis_port,
                decode_responses=True,
                socket_connect_timeout=0.5,
                socket_timeout=0.5
            )

    def validate_session(self, token: str) -&gt; Optional[int]:
        if self.r and (time.time() - self.last_redis_failure &gt; self.retry_cooldown):
            try:
                data = self.r.get(f"session:{token}")
                if data:
```

```
                    session_data = json.loads(data)
                    expires_at = datetime.fromisoformat(session_data["expires_at"])
                    if datetime.now(timezone.utc) &lt; expires_at or datetime.utcnow() &lt; expires_at.rep
                        return session_data.get("login")
            except Exception as e:
                logger.error(f"History Server Redis session lookup failed: {e}. Entering Redis cooldown.")
                import time
                self.last_redis_failure = time.time()
        return None

# Helper to validate session
import time
session_reader = RedisSessionReader()

async def get_current_login(request: Request) -&gt; int:
    """Dependency that extracts the Bearer token and verifies the session (with main_server HTTP fallback)
    auth_header = request.headers.get("Authorization")
    if not auth_header or not auth_header.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Unauthorized: missing bearer token")
    token = auth_header.split(" ")[1]

    # 1. Try Redis first
    login = session_reader.validate_session(token)
    if login:
        return login

    # 2. Try HTTP validation fallback against main_server (defense-in-depth)
    main_server_url = os.getenv("MAIN_SERVER_URL", "localhost:8000")
    try:
        async with httpx.AsyncClient() as client:
            resp = await client.get(
                f"http://{main_server_url}/internal/auth/validate",
                params={"token": token},
                headers={"X-Internal-Secret": INTERNAL_SECRET},
                timeout=2.0
            )
            if resp.status_code == 200:
                login = resp.json().get("login")
                if login:
                    return login
    except Exception as e:
        logger.error(f"History server HTTP session validation fallback failed: {e}")

    raise HTTPException(status_code=401, detail="Unauthorized: invalid or expired session token")

class BarResponse(BaseModel):
    time: str
    open: Decimal
    high: Decimal
    low: Decimal
    close: Decimal
    tick_volume: int

def parse_datetime(val: str) -&gt; Optional[datetime]:
    if not val:
        return None
    try:
        # Try epoch float
        return datetime.fromtimestamp(float(val), tz=timezone.utc)
    except ValueError:
        pass
    try:
        # Try ISO format
        val_clean = val.replace('Z', '+00:00')
        dt = datetime.fromisoformat(val_clean)
        if dt.tzinfo is None:
            dt = dt.replace(tzinfo=timezone.utc)
        return dt
```

```
    except Exception as e:
        raise HTTPException(status_code=400, detail=f"Invalid datetime format: {val}. Error: {e}")

# ==================== PUBLIC/CLIENT CHART API ====================

@app.get("/history/{symbol}", response_model=List[BarResponse])
async def get_chart_history(
    symbol: str,
    tf: str = Query("1m", regex="^(1m|5m|1h|1d)$"),
    from_time: Optional[str] = Query(None, alias="from"),
    to_time: Optional[str] = Query(None, alias="to"),
    current_login: int = Depends(get_current_login)
):
    """Fetches OHLCV chart history bars for a given symbol and timeframe."""
    # Parse date range
    dt_from = parse_datetime(from_time)
    dt_to = parse_datetime(to_time)

    if not dt_to:
        dt_to = datetime.now(timezone.utc)
    if not dt_from:
        dt_from = dt_to - timedelta(days=7)

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        # Query completed bars from database
        query = f"""
            SELECT time, open, high, low, close, tick_volume
            FROM bars_{tf}
            WHERE symbol = $1 AND time &gt;= $2 AND time &lt;= $3
            ORDER BY time ASC
        """
        rows = await conn.fetch(query, symbol, dt_from.isoformat(), dt_to.isoformat())

        results = []
        for r in rows:
            # Handle float values from DB and format safely to Decimals
            results.append(BarResponse(
                time=str(r["time"]),
                open=Decimal(str(r["open"])),
                high=Decimal(str(r["high"])),
                low=Decimal(str(r["low"])),
                close=Decimal(str(r["close"])),
                tick_volume=int(r["tick_volume"])
            ))

        # Add current in-progress bar to response if it falls within the window
        key = (symbol, tf)
        if key in bar_aggregator.current_bars:
            in_progress = bar_aggregator.current_bars[key]
            if dt_from &lt;= in_progress.time &lt;= dt_to:
                results.append(BarResponse(
                    time=in_progress.time.isoformat(),
                    open=in_progress.open,
                    high=in_progress.high,
                    low=in_progress.low,
                    close=in_progress.close,
                    tick_volume=in_progress.tick_volume
                ))

        return results

# ==================== INTERNAL TICKET INGESTION ====================

@app.websocket("/internal/ticks/ingest")
async def ingest_ticks(websocket: WebSocket):
    """WebSocket endpoint for main_server to push processed ticks."""
    secret = websocket.headers.get("X-Internal-Secret") or websocket.query_params.get("secret")
```

```
    if not secret or not secrets.compare_digest(secret, INTERNAL_SECRET):
        await websocket.close(code=4003, reason="Unauthorized")
        return

    await websocket.accept()
    logger.info("Tick ingestion WebSocket connection established")

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        try:
            async for message in websocket.iter_text():
                data = json.loads(message)
                symbol = data["symbol"]
                bid = Decimal(str(data["bid"]))
                ask = Decimal(str(data["ask"]))

                # Check for timestamp
                tick_time_str = data.get("time")
                if tick_time_str:
                    tick_time = parse_datetime(tick_time_str)
                else:
                    tick_time = datetime.now(timezone.utc)

                # 1. Store raw tick in database
                await conn.execute(
                    "INSERT INTO ticks (time, symbol, bid, ask, source) VALUES ($1, $2, $3, $4, $5)",
                    tick_time.isoformat(),
                    symbol,
                    bid,
                    ask,
                    data.get("source", "INTERNAL")
                )

                # 2. Feed to BarAggregator
                await bar_aggregator.on_tick(tick_time, symbol, bid, conn)

        except WebSocketDisconnect:
            logger.info("Tick ingestion WebSocket disconnected")
        except Exception as e:
            logger.error(f"Error in tick ingestion handler: {e}")

# ==================== A-BOOK GATEWAY ROUTE ====================

class GatewayExecutionRequest(BaseModel):
    ticket: int
    symbol: str
    volume: float
    action: int
    price: Optional[float] = None
    type_filling: Optional[str] = "FOK"
    gateway_url: Optional[str] = None
    gateway_login: Optional[str] = None
    gateway_password: Optional[str] = None
    gateway_server: Optional[str] = None

@app.post("/internal/gateway/execute")
async def execute_gateway_order(request: Request, payload: GatewayExecutionRequest):
    # Gate by internal secret
    secret = request.headers.get("X-Internal-Secret")
    if not secret or not secrets.compare_digest(secret, INTERNAL_SECRET):
        raise HTTPException(status_code=403, detail="Forbidden: Invalid or missing X-Internal-Secret")

    res = await mt5_gateway.execute_order(
        ticket=payload.ticket,
        symbol=payload.symbol,
        volume=payload.volume,
        action=payload.action,
        price=payload.price,
```

```
        type_filling=payload.type_filling,
        gateway_url=payload.gateway_url,
        gateway_login=payload.gateway_login,
        gateway_password=payload.gateway_password,
        gateway_server=payload.gateway_server
    )
    return res

# ==================== APP LIFESPAN EVENTS ====================

@app.on_event("startup")
async def startup_event():
    await DBConnection.initialize()
    logger.info("History Server started and DB initialized.")
    START_FEED = os.getenv("START_FEED", "true").lower() == "true"
    if START_FEED:
        mt5_feed.start()
        logger.info("MT5 price feed started.")

@app.on_event("shutdown")
async def shutdown_event():
    await DBConnection.close()
    logger.info("History Server DB connection closed.")
    START_FEED = os.getenv("START_FEED", "true").lower() == "true"
    if START_FEED:
        await mt5_feed.stop()
        logger.info("MT5 price feed stopped.")
```

■ history_server\bases\connection.py

```
import os
import re
import asyncio
import logging
import sqlite3
from typing import Optional, Any, Dict, List

logger = logging.getLogger(__name__)

# Try to import asyncpg
try:
    import asyncpg
except ImportError:
    asyncpg = None

class SQLiteConnectionWrapper:
    def __init__(self, conn: sqlite3.Connection):
        self._conn = conn

    def _convert_query(self, query: str) -&gt; str:
        # Convert Postgres $1, $2 placeholders to SQLite ? placeholders
        query = re.sub(r'\$\d+', '?', query)
        # Convert PG specific standard functions like NOW() to SQLite datetime('now')
        query = query.replace("NOW()", "datetime('now')")
        return query

    async def fetch(self, query: str, *args) -&gt; List[Dict[str, Any]]:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            return [dict(row) for row in cursor.fetchall()]
        return await loop.run_in_executor(None, _run)

    async def fetchrow(self, query: str, *args) -&gt; Optional[Dict[str, Any]]:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            row = cursor.fetchone()
            return dict(row) if row else None
        return await loop.run_in_executor(None, _run)

    async def fetchval(self, query: str, *args) -&gt; Any:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            row = cursor.fetchone()
            if row:
                return row[0]
            return None
        return await loop.run_in_executor(None, _run)

    async def execute(self, query: str, *args) -&gt; None:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            self._conn.commit()
        await loop.run_in_executor(None, _run)

    class TransactionContext:
        def __init__(self, conn: sqlite3.Connection):
            self._conn = conn
```

```
        async def __aenter__(self):
            return self

        async def __aexit__(self, exc_type, exc_val, exc_tb):
            if exc_type:
                self._conn.rollback()
            else:
                self._conn.commit()

    def transaction(self):
        return self.TransactionContext(self._conn)

class SQLitePoolWrapper:
    """Mock asyncpg Pool wrapper for SQLite."""
    def __init__(self, db_path: str):
        self.db_path = db_path
        self._conn = None

    def initialize(self):
        # Register Decimal adapter and converter
        from decimal import Decimal
        sqlite3.register_adapter(Decimal, lambda d: str(d))
        sqlite3.register_converter("DECIMAL", lambda b: Decimal(b.decode('utf-8')))

        self._conn = sqlite3.connect(
            self.db_path,
            check_same_thread=False,
            detect_types=sqlite3.PARSE_DECLTYPES
        )
        self._conn.row_factory = sqlite3.Row

    def close(self):
        if self._conn:
            self._conn.close()

    class ConnectionContext:
        def __init__(self, conn_wrapper: SQLiteConnectionWrapper):
            self.wrapper = conn_wrapper

        async def __aenter__(self) -&gt; SQLiteConnectionWrapper:
            return self.wrapper

        async def __aexit__(self, exc_type, exc_val, exc_tb):
            pass

    def acquire(self) -&gt; ConnectionContext:
        return self.ConnectionContext(SQLiteConnectionWrapper(self._conn))

class DBConnection:
    pool: Optional[Any] = None
    is_sqlite: bool = False

    @classmethod
    async def initialize(cls):
        """Initialize connection pool (Postgres asyncpg or SQLite fallback)."""
        db_type = os.getenv("DB_TYPE", "sqlite")

        if db_type == "postgres" and asyncpg is not None:
            dsn = os.getenv("HISTORY_DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/history
            try:
                cls.pool = await asyncpg.create_pool(
                    dsn=dsn, min_size=5, max_size=20, timeout=30.0
                )
                cls.is_sqlite = False
                logger.info("Initialized PostgreSQL history pool successfully.")
                return
            except Exception as e:
                logger.warning(f"Failed to connect to PostgreSQL history. Falling back to SQLite. Error: {
```

```
    # Fallback to SQLite
    db_path = os.getenv("HISTORY_SQLITE_DB_PATH", os.path.join(os.path.dirname(__file__), "history.db"
    cls.pool = SQLitePoolWrapper(db_path)
    cls.pool.initialize()
    cls.is_sqlite = True
    logger.info(f"Initialized SQLite history database resolved to: {db_path}")

@classmethod
async def close(cls):
    """Close connection pool."""
    if cls.pool is not None:
        if cls.is_sqlite:
            cls.pool.close()
        else:
            await cls.pool.close()
        cls.pool = None

@classmethod
def get_pool(cls) -&gt; Any:
    if cls.pool is None:
        raise RuntimeError("Database pool has not been initialized. Call initialize() first.")
    return cls.pool
```

■ history_server\bases\history.db

(Empty file)

■ history_server\bases\schema.sql

```
-- PostgreSQL Schema for History Server (Time-Series Market Data)

DROP TABLE IF EXISTS ticks CASCADE;
DROP TABLE IF EXISTS bars_1m CASCADE;
DROP TABLE IF EXISTS bars_5m CASCADE;
DROP TABLE IF EXISTS bars_1h CASCADE;
DROP TABLE IF EXISTS bars_1d CASCADE;

-- Raw Ticks table
CREATE TABLE ticks (
    id BIGSERIAL PRIMARY KEY,
    time TIMESTAMP WITH TIME ZONE NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    bid DECIMAL(18, 8) NOT NULL,
    ask DECIMAL(18, 8) NOT NULL,
    source VARCHAR(32) NOT NULL
);

-- Index for tick lookup by symbol and time range
CREATE INDEX idx_ticks_symbol_time ON ticks (symbol, time DESC);

-- OHLCV Bar Tables
CREATE TABLE bars_1m (
    time TIMESTAMP WITH TIME ZONE NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_1m_lookup ON bars_1m (symbol, time DESC);

CREATE TABLE bars_5m (
    time TIMESTAMP WITH TIME ZONE NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_5m_lookup ON bars_5m (symbol, time DESC);

CREATE TABLE bars_1h (
    time TIMESTAMP WITH TIME ZONE NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_1h_lookup ON bars_1h (symbol, time DESC);

CREATE TABLE bars_1d (
    time TIMESTAMP WITH TIME ZONE NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
```

```
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_1d_lookup ON bars_1d (symbol, time DESC);
```

■ history_server\bases\schema_sqlite.sql

```
-- SQLite Schema for History Server (Time-Series Market Data)

DROP TABLE IF EXISTS ticks;
DROP TABLE IF EXISTS bars_1m;
DROP TABLE IF EXISTS bars_5m;
DROP TABLE IF EXISTS bars_1h;
DROP TABLE IF EXISTS bars_1d;

-- Raw Ticks table
CREATE TABLE ticks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    time TEXT NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    bid DECIMAL(18, 8) NOT NULL,
    ask DECIMAL(18, 8) NOT NULL,
    source VARCHAR(32) NOT NULL
);

-- Index for tick lookup by symbol and time range
CREATE INDEX idx_ticks_symbol_time ON ticks (symbol, time DESC);

-- OHLCV Bar Tables
CREATE TABLE bars_1m (
    time TEXT NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_1m_lookup ON bars_1m (symbol, time DESC);

CREATE TABLE bars_5m (
    time TEXT NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_5m_lookup ON bars_5m (symbol, time DESC);

CREATE TABLE bars_1h (
    time TEXT NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_1h_lookup ON bars_1h (symbol, time DESC);

CREATE TABLE bars_1d (
    time TEXT NOT NULL,
    symbol VARCHAR(32) NOT NULL,
    open DECIMAL(18, 8) NOT NULL,
    high DECIMAL(18, 8) NOT NULL,
    low DECIMAL(18, 8) NOT NULL,
    close DECIMAL(18, 8) NOT NULL,
```

```
    tick_volume BIGINT NOT NULL DEFAULT 0,
    PRIMARY KEY (time, symbol)
);
CREATE INDEX idx_bars_1d_lookup ON bars_1d (symbol, time DESC);
```

■ history_server\Datafeeds\mt5_feed.py

```
import os
import json
import logging
import asyncio
import httpx
import websockets
import msgpack
import secrets
import fnmatch
from datetime import datetime, timezone
from typing import Dict, Any, Optional, List

logger = logging.getLogger("history_server.mt5_feed")

# Load configuration from environment variables
TRADE_SERVER_URL = os.getenv("TRADE_SERVER_URL", "http://localhost:8003")
MAIN_SERVER_URL = os.getenv("MAIN_SERVER_URL", "http://localhost:8000")
INTERNAL_SECRET = os.getenv("INTERNAL_SECRET", "default_internal_secret_token_change_in_production")

# Symbol digits mapping for point and spread calculations
SYMBOL_DIGITS = {
    "EURUSD": 5,
    "GBPUSD": 5,
    "BTCUSD": 2,
    "BTCUSDT": 2
}

def match_symbols_filter(symbol_name: str, symbol_path: str, filters: List[str]) -&gt; bool:
    if not filters:
        return False

    # Normalize path and name
    norm_name = symbol_name.strip().lower()
    norm_path = symbol_path.replace('/', '\\').strip().lower()

    # Check if there are any inclusion rules (rules that do not start with !)
    has_inclusions = any(not f.startswith('!') for f in filters)

    def matches_pattern(pattern: str) -&gt; bool:
        pat = pattern.replace('/', '\\').strip().lower()
        while '\\\\' in pat:
            pat = pat.replace('\\\\', '\\')
        # fnmatch supports wildcards like * and ?
        if fnmatch.fnmatch(norm_path, pat):
            return True
        if fnmatch.fnmatch(norm_name, pat):
            return True

        # Support literal path matching when broker path differs (e.g., forex\EURUSD -&gt; Forex\Major\EUR
        if '\\' in pat:
            pat_parts = pat.split('\\')
            if pat_parts[-1] == norm_name:
                if len(pat_parts) &gt; 1:
                    parent_folder = pat_parts[-2]
                    if parent_folder == '*' or any(parent_folder in c or c.startswith(parent_folder) for c
                        return True
        # If no backslash in pattern, match base name or anywhere in path
        if '\\' not in pat:
            if fnmatch.fnmatch(norm_name, pat):
                return True
            # Also check if it's a simple wild-match on path components
            components = norm_path.split('\\')
            if any(fnmatch.fnmatch(c, pat) for c in components):
                return True
        return False
```

```
        matched = False
        if has_inclusions:
            for f in filters:
                if f.startswith('!'):
                    continue
                if matches_pattern(f):
                    matched = True
                    break
        else:
            # Defaults to true if no inclusions specified
            matched = True

        # Check exclusions
        for f in filters:
            if f.startswith('!'):
                pattern = f[1:]
                if matches_pattern(pattern):
                    return False

        return matched

    class MT5Feed:
        def __init__(self):
            self.last_ticks: Dict[str, Dict[str, Any]] = {}
            self.is_running = False
            self._manager_task = None
            self._tasks: Dict[int, tuple] = {}
            self.symbol_digits: Dict[str, int] = {}

        def start(self):
            self.is_running = True
            self._manager_task = asyncio.create_task(self._manager_loop())
            logger.info("MT5Feed dynamic manager worker started.")

        async def stop(self):
            self.is_running = False
            if self._manager_task:
                self._manager_task.cancel()
                try:
                    await self._manager_task
                except asyncio.CancelledError:
                    pass
            for fid, item in list(self._tasks.items()):
                task = item[0]
                task.cancel()
                try:
                    await task
                except asyncio.CancelledError:
                    pass
            self._tasks.clear()
            logger.info("MT5Feed dynamic manager worker stopped.")

        def is_valid_tick(self, symbol: str, bid: float, ask: float) -&gt; bool:
            """
            Apply strict quote filtration rules per MT5 specification:
            1. Ask must be &gt; Bid.
            2. Spread must not be extreme (&lt;= 500 points).
            3. Price jump must be within bounds (&lt; 5% change from last quote).
            """
            if ask &lt;= bid:
                logger.warning(f"Filtered tick {symbol}: ask {ask} &lt;= bid {bid}")
                return False

            base_symbol = symbol.split('\\')[-1].upper()
            digits = self.symbol_digits.get(symbol.upper()) or self.symbol_digits.get(base_symbol) or SYMBOL_D
            point = 10 ** (-digits)
            spread_points = round((ask - bid) / point)
            max_spread = 20000 if ("BTC" in base_symbol or "ETH" in base_symbol or "CRYPTO" in base_symbol) el
```

```
        if spread_points &gt; max_spread:
            logger.warning(f"Filtered tick {symbol}: extreme spread {spread_points} points (bid={bid}, ask
            return False

        last_tick = self.last_ticks.get(symbol)
        if last_tick:
            last_bid = last_tick["bid"]
            if last_bid &gt; 0:
                pct_change = abs(bid - last_bid) / last_bid
                if pct_change &gt;= 0.05:
                    logger.warning(f"Filtered tick {symbol}: extreme price jump {pct_change*100:.2f}% (las
                    return False

        return True

    async def _manager_loop(self):
        logger.info("Starting MT5Feed config polling manager loop...")
        async with httpx.AsyncClient() as http_client:
            while self.is_running:
                try:
                    resp = await http_client.get(
                        f"{MAIN_SERVER_URL}/internal/gateways",
                        headers={"X-Internal-Secret": INTERNAL_SECRET},
                        timeout=5.0
                    )
                    if resp.status_code == 200:
                        all_gateways = resp.json()
                        active_feeders = [
                            g for g in all_gateways
                            if g.get("type", "").startswith("Feeder_") and g.get("is_active") == 1
                        ]

                        active_ids = set()

                        # Fields that are runtime stats and should NOT trigger a restart
                        _RUNTIME_FIELDS = {"ticks_count", "last_active", "last_tick_age_s"}

                        def _config_key(f):
                            return {k: v for k, v in f.items() if k not in _RUNTIME_FIELDS}

                        for feeder in active_feeders:
                            fid = feeder["id"]
                            active_ids.add(fid)
                            task_is_done = fid in self._tasks and self._tasks[fid][0].done()
                            if fid not in self._tasks or task_is_done:
                                if task_is_done:
                                    # Retrieve exception if any to log it
                                    try:
                                        exc = self._tasks[fid][0].exception()
                                    except Exception:
                                        exc = "cancelled or pending"
                                    logger.warning(f"Feeder task '{name}' (ID: {fid}) terminated (reason/e
                                else:
                                    logger.info(f"Detected new active feeder: {feeder['name']} (ID: {fid})
                                t = asyncio.create_task(self._run_feed_loop(feeder))
                                self._tasks[fid] = (t, feeder)
                            else:
                                _, old_feeder = self._tasks[fid]
                                old_key = _config_key(old_feeder)
                                new_key = _config_key(feeder)
                                if old_key != new_key:
                                    diffs = {k: (old_key.get(k), new_key.get(k)) for k in old_key if old_k
                                    logger.info(f"Detected config change for feeder: {feeder['name']} (ID:
                                    self._tasks[fid][0].cancel()
                                    t = asyncio.create_task(self._run_feed_loop(feeder))
                                    self._tasks[fid] = (t, feeder)
                                else:
                                    # Update the stored feeder dict with latest stats without restarting
```

```
                                    self._tasks[fid] = (self._tasks[fid][0], feeder)

                        # Stop tasks no longer present or active
                        for fid in list(self._tasks.keys()):
                            if fid not in active_ids:
                                logger.info(f"Stopping feeder task ID: {fid} (feeder disabled or deleted)"
                                self._tasks[fid][0].cancel()
                                del self._tasks[fid]
                    else:
                        logger.error(f"Failed to fetch gateways from main server: {resp.status_code}")
                except Exception as e:
                    logger.error(f"Error in MT5Feed manager loop: {e}")

                await asyncio.sleep(10.0)

    async def _run_feed_loop(self, feeder: Dict[str, Any]):
        feeder_id = feeder["id"]
        name = feeder["name"]
        host = feeder.get("host") or "localhost"
        port = feeder.get("port") or 8003
        username = feeder.get("username", "")
        api_key = feeder.get("api_key", "")
        settings_json = feeder.get("settings_json") or "{}"

        trade_server_url = f"http://{host}:{port}"

        # Parse settings
        if isinstance(settings_json, dict):
            settings = settings_json
        else:
            try:
                settings = json.loads(settings_json)
            except Exception:
                settings = {}

        gateway_settings = settings.get("gateway", {})
        gateway_server = gateway_settings.get("gateway_server") or "86.104.251.194:443"

        symbols_filter = settings.get("symbols_filter") or ""

        parameters = settings.get("parameters", {})
        terminal_path = parameters.get("LP_TERMINAL_PATH") or "C:\\Program Files\\MetaTrader 5\\terminal64

        backoff = 1.0
        max_backoff = 30.0

        logger.info(f"Starting connection task for feeder '{name}' (ID: {feeder_id}) targeting trade-serve

        async with httpx.AsyncClient() as http_client:
            while self.is_running:
                try:
                    # 1. Connect to trade-server
                    logger.info(f"[{name}] Authenticating with trade-server MT5 session at {trade_server_u
                    try:
                        login_id = int(username)
                    except ValueError:
                        logger.error(f"[{name}] Invalid login ID '{username}'. Must be an integer.")
                        await asyncio.sleep(10)
                        continue

                    resp = await http_client.post(
                        f"{trade_server_url}/api/v1/connect",
                        json={
                            "login": login_id,
                            "password": api_key,
                            "server": gateway_server,
                            "path": terminal_path
                        },
```

```
    timeout=15.0
)
if resp.status_code != 200:
    raise Exception(f"Connect failed with status {resp.status_code}: {resp.text}")

logger.info(f"[{name}] Successfully logged in to MT5. Resolving symbols...")

# Fetch platform symbols from main server and broker symbols from trade-server
resolved_symbols = []
try:
    # 1. Fetch platform symbols from main server
    plat_resp = await http_client.get(
        f"{MAIN_SERVER_URL}/internal/symbols",
        headers={"X-Internal-Secret": INTERNAL_SECRET},
        timeout=10.0
    )
    # 2. Fetch broker symbols from trade-server
    sym_resp = await http_client.get(f"{trade_server_url}/api/v1/symbols", timeout=10.

    if plat_resp.status_code == 200 and sym_resp.status_code == 200:
        plat_symbols = plat_resp.json()
        broker_symbols = sym_resp.json().get("data", []) if isinstance(sym_resp.json()

        # Get translations list
        translations = settings.get("translations") or []

        # Build a quick set of broker symbol names for fast lookup
        broker_sym_map = {s["name"].upper(): s for s in broker_symbols if isinstance(s

        filters_list = []
        for s in symbols_filter.split(","):
            s = s.strip()
            if not s:
                continue
            while '\\\\' in s:
                s = s.replace('\\\\', '\\')
            filters_list.append(s)

        # Match filter rules against platform symbol directory paths
        for p_sym in plat_symbols:
            if not isinstance(p_sym, dict) or "symbol" not in p_sym:
                continue
            plat_name = p_sym["symbol"] # e.g. "forex\EURUSD" or "Metals\XAGUSD"
            plat_digits = p_sym.get("digits", 5)

            # Run matching logic on platform symbol name
            if match_symbols_filter(plat_name, plat_name, filters_list):
                # Translate platform symbol name to broker symbol name
                # Check translation rules (support matching either full path or base n
                broker_name = None
                for rule in translations:
                    rule_sym = rule.get("symbol")
                    if rule_sym == plat_name or rule_sym == plat_name.split('\\')[-1]:
                        broker_name = rule.get("source")
                        break

                # Fallback to last segment of platform path
                if not broker_name:
                    broker_name = plat_name.split('\\')[-1]

                # Check if the translated broker symbol exists on the broker side
                broker_upper = broker_name.upper()
                if broker_upper in broker_sym_map:
                    actual_broker_name = broker_sym_map[broker_upper]["name"]
                    if actual_broker_name not in resolved_symbols:
                        resolved_symbols.append(actual_broker_name)

                    # Calibrate dynamic digits using platform digits
```

```
                    self.symbol_digits[plat_name.upper()] = int(plat_digits)
                    broker_digits = broker_sym_map[broker_upper].get("digits", plat_di
                    self.symbol_digits[actual_broker_name.upper()] = int(broker_digits
                    base = actual_broker_name.split('\\')[-1].upper()
                    self.symbol_digits[base] = int(broker_digits)

        logger.info(f"[{name}] Filter '{symbols_filter}' matched {len(resolved_symbols
    else:
        logger.warning(f"[{name}] Failed to fetch symbols lists: plat_status={plat_res
except Exception as e:
    logger.warning(f"[{name}] Exception fetching symbols for resolution: {e}. Using fa

# Fallback: if wildcard matching gave 0 results, try treating filter items as literal
if not resolved_symbols and symbols_filter:
    fallback_items = [s.strip() for s in symbols_filter.split(",") if s.strip()]
    resolved_symbols = [s for s in fallback_items if "*" not in s and not s.startswith
    if resolved_symbols:
        logger.info(f"[{name}] No broker symbols matched filter. Using filter items as

if not resolved_symbols:
    logger.warning(f"[{name}] No symbols to subscribe — symbols filter is empty or mat
                   f"Please configure symbols in the Data Feeds UI (Symbols tab). Retr
    await asyncio.sleep(30)
    continue

logger.info(f"[{name}] Connecting WebSocket stream...")

# 2. WebSocket listener
ws_url = trade_server_url.replace("http://", "ws://") + "/ws/marketdata"
async with websockets.connect(ws_url) as ws:
    backoff = 1.0  # Reset backoff on success

    # Subscribe to resolved symbols
    for sym in resolved_symbols:
        await ws.send(json.dumps({"action": "sub", "symbol": sym}))
        logger.info(f"[{name}] Subscribed to {sym}")

    # Receive ticks loop
    async for message in ws:
        tv_tick = msgpack.unpackb(message, raw=False)
        if not isinstance(tv_tick, dict) or "s" not in tv_tick:
            continue

        symbol = tv_tick["s"]
        bid = float(tv_tick["b"])
        ask = float(tv_tick["a"])
        timestamp = float(tv_tick.get("ts", 0)) / 1000.0
        if timestamp == 0:
            timestamp = datetime.now(timezone.utc).timestamp()

        # Apply validation
        if not self.is_valid_tick(symbol, bid, ask):
            continue

        last_tick = self.last_ticks.get(symbol)
        if last_tick and last_tick["bid"] == bid and last_tick["ask"] == ask:
            continue

        tick_payload = {
            "symbol": symbol,
            "bid": bid,
            "ask": ask,
            "time": timestamp
        }
        self.last_ticks[symbol] = tick_payload

        # Forward to main_server
        try:
```

```
                    await http_client.post(
                        f"{MAIN_SERVER_URL}/internal/ticks",
                        json={
                            "symbol": symbol,
                            "bid": bid,
                            "ask": ask,
                            "time": datetime.fromtimestamp(timestamp, tz=timezone.utc).isoform
                            "source": name
                        },
                        headers={"X-Internal-Secret": INTERNAL_SECRET},
                        timeout=2.0
                    )
                except Exception as e:
                    logger.warning(f"[{name}] Failed to post tick for {symbol} to main_server:

        except asyncio.CancelledError:
            logger.info(f"[{name}] Connection task canceled.")
            break
        except Exception as e:
            logger.error(f"[{name}] Error in feed connection loop: {e}. Retrying in {backoff}s..."
            await asyncio.sleep(backoff)
            backoff = min(max_backoff, backoff * 2.0)
```

■ history_server\Gateways\mt5_gateway.py

```
import os
import logging
import httpx
from typing import Dict, Any, Optional

logger = logging.getLogger("history_server.mt5_gateway")

TRADE_SERVER_URL = os.getenv("TRADE_SERVER_URL", "http://localhost:8003")

class MT5Gateway:
    def __init__(self):
        pass

    async def execute_order(self, ticket: int, symbol: str, volume: float, action: int, price: Optional[fl
                            gateway_url: Optional[str] = None, gateway_login: Optional[str] = None, gatewa
        """
        Execute order on MT5 LP sandbox terminal via trade-server.
        Maps symbol, side, volume and returns status + execution price.
        """
        # side: 0 = BUY, 1 = SELL
        side = "buy" if action == 0 else "sell"
        trade_url = gateway_url or TRADE_SERVER_URL

        # Map symbol translations or strip folder prefixes so liquidity provider matches correctly
        target_symbol = symbol
        try:
            from history_server.bases.connection import DBConnection
            import json
            pool = DBConnection.get_pool()
            async with pool.acquire() as conn:
                gw_row = await conn.fetchrow(
                    "SELECT settings_json FROM gateways WHERE username = $1",
                    str(gateway_login) if gateway_login else ""
                )
                if gw_row and gw_row["settings_json"]:
                    settings = json.loads(gw_row["settings_json"])
                    translations = settings.get("translations", [])
                    for trans in translations:
                        local_sym = trans.get("symbol", "")
                        if local_sym.lower() == symbol.lower():
                            target_symbol = trans.get("source", symbol)
                            break
        except Exception as e:
            logger.warning(f"MT5Gateway: Failed to lookup symbol translation from DB: {e}")

        # Fallback: Strip folders from symbol path if not explicitly translated
        if target_symbol == symbol:
            if "\\" in target_symbol:
                target_symbol = target_symbol.split("\\")[-1]
            elif "/" in target_symbol:
                target_symbol = target_symbol.split("/")[-1]

        logger.info(f"MT5Gateway: Forwarding ticket {ticket} ({side} {volume} {target_symbol} [original: {
        try:
            async with httpx.AsyncClient() as client:
                # If credentials are provided, establish terminal session connection first
                if gateway_login and gateway_password:
                    try:
                        login_id = int(gateway_login)
                        target_server = gateway_server or "86.104.251.194:443"

                        # Check existing connection status to avoid redundant disconnect/reconnect cycles
                        already_connected = False
                        try:
                            status_resp = await client.get(f"{trade_url}/api/v1/status", timeout=2.0)
```

```
                    if status_resp.status_code == 200:
                        status_data = status_resp.json()
                        data = status_data.get("data", {})
                        if data.get("status") == "connected" and str(data.get("login")) == str(log
                            already_connected = True
                            logger.info(f"MT5Gateway: Terminal at {trade_url} is already connected
                except Exception as e:
                    logger.warning(f"MT5Gateway: Failed to fetch connection status: {e}")

                if not already_connected:
                    logger.info(f"MT5Gateway: Connecting terminal session at {trade_url} for login
                    conn_resp = await client.post(
                        f"{trade_url}/api/v1/connect",
                        json={
                            "login": login_id,
                            "password": gateway_password,
                            "server": target_server,
                            "path": "C:\\Program Files\\MetaTrader 5\\terminal64.exe"
                        },
                        timeout=15.0
                    )
                    if conn_resp.status_code != 200:
                        logger.error(f"MT5Gateway: Connect session failed: {conn_resp.text}")
            except ValueError:
                logger.error(f"MT5Gateway: Invalid login ID '{gateway_login}'.")

        resp = await client.post(
            f"{trade_url}/api/v1/place-order",
            json={
                "symbol": target_symbol,
                "side": side,
                "volume": float(volume),
                "price": float(price) if price else None,
                "type_filling": type_filling
            },
            timeout=10.0
        )

        if resp.status_code == 200:
            res_body = resp.json()
            data = res_body.get("data", {})
            if data.get("success"):
                exec_price = float(data.get("price", price or 0.0))
                order_id = data.get("order")
                logger.info(f"MT5Gateway: Ticket {ticket} successfully executed on LP! ID: {order_
                return {
                    "status": "FILLED",
                    "price_execution": exec_price,
                    "order_id": order_id
                }
            else:
                err_reason = data.get("error", "Execution failed on gateway")
                logger.warning(f"MT5Gateway: Ticket {ticket} rejected by LP. Reason: {err_reason}"
                return {
                    "status": "REJECTED",
                    "reason": err_reason
                }
        else:
            err_reason = f"Gateway HTTP error status {resp.status_code}: {resp.text}"
            logger.warning(f"MT5Gateway: Ticket {ticket} failed. Reason: {err_reason}")
            return {
                "status": "REJECTED",
                "reason": err_reason
            }
    except Exception as e:
        err_reason = f"LP gateway connection error: {e}"
        logger.error(f"MT5Gateway: Ticket {ticket} failed. Reason: {err_reason}")
        return {
```

```
    "status": "REJECTED",
    "reason": err_reason
}
```

■ history_server\scratch\test_feeder_scenarios.py

```
import sys
import os
import time
import fnmatch
from typing import List, Dict

# Ensure we can import match_symbols_filter from mt5_feed
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
try:
    from Datafeeds.mt5_feed import match_symbols_filter
except ImportError:
    # Inline fallback implementation for standalone run
    def match_symbols_filter(symbol_name: str, symbol_path: str, filters: List[str]) -&gt; bool:
        if not filters:
            return False
        norm_name = symbol_name.strip().lower()
        norm_path = symbol_path.replace('/', '\\').strip().lower()
        while '\\\\' in norm_path:
            norm_path = norm_path.replace('\\\\', '\\')
        has_inclusions = any(not f.startswith('!') for f in filters)

        def matches_pattern(pattern: str) -&gt; bool:
            pat = pattern.replace('/', '\\').strip().lower()
            while '\\\\' in pat:
                pat = pat.replace('\\\\', '\\')
            if fnmatch.fnmatch(norm_path, pat):
                return True
            if fnmatch.fnmatch(norm_name, pat):
                return True
            if '\\' in pat:
                pat_parts = pat.split('\\')
                if pat_parts[-1] == norm_name:
                    if len(pat_parts) &gt; 1:
                        parent_folder = pat_parts[-2]
                        if parent_folder == '*' or any(parent_folder in c or c.startswith(parent_folder) f
                            return True
            if '\\' not in pat:
                if fnmatch.fnmatch(norm_name, pat):
                    return True
                components = norm_path.split('\\')
                if any(fnmatch.fnmatch(c, pat) for c in components):
                    return True
            return False

        for f in filters:
            if f.startswith('!'):
                exclude_pattern = f[1:]
                if matches_pattern(exclude_pattern):
                    return False
        if has_inclusions:
            for f in filters:
                if f.startswith('!'):
                    continue
                if matches_pattern(f):
                    return True
            return False
        return True

# 1. Define Mock Platform Symbols (different groups, sub-groups, custom names)
mock_platform_symbols = [
    {"symbol": "crypto\\BTCUSD", "digits": 5},
    {"symbol": "forex\\EURUSD", "digits": 5},
    {"symbol": "GBPUSD", "digits": 5},
    {"symbol": "Metals\\XAUUSD", "digits": 2},
    {"symbol": "Metals\\XAGUSD", "digits": 2},
```

```
    {"symbol": "forex\\Major\\USDJPY", "digits": 3},
    {"symbol": "forex\\Exotics\\USDTRY", "digits": 5},
    {"symbol": "custom\\HISUSD", "digits": 5},
    {"symbol": "energy\\Crude\\CL_OIL", "digits": 2},
    {"symbol": "indices\\US30", "digits": 2},
    {"symbol": "forex\\EURUSD.ai", "digits": 5},
]

# Mock MT5 Broker symbols
mock_broker_symbols = [
    {"name": "BTCUSD", "digits": 5},
    {"name": "EURUSD", "digits": 5},
    {"name": "GBPUSD", "digits": 5},
    {"name": "XAUUSD", "digits": 2},
    {"name": "XAGUSD", "digits": 2},
    {"name": "USDJPY", "digits": 3},
    {"name": "USDTRY", "digits": 5},
    {"name": "HISUSD", "digits": 5},
    {"name": "CL_OIL", "digits": 2},
    {"name": "US30", "digits": 2},
]

def run_resolution_test(filter_string: str, translations: List[Dict] = None):
    if translations is None:
        translations = []

    filters_list = []
    for s in filter_string.split(","):
        s = s.strip()
        if not s:
            continue
        while '\\\\' in s:
            s = s.replace('\\\\', '\\')
        filters_list.append(s)

    broker_sym_map = {s["name"].upper(): s for s in mock_broker_symbols}
    matched_platform_paths = []
    subscribed_broker_symbols = []

    for p_sym in mock_platform_symbols:
        plat_name = p_sym["symbol"]

        if match_symbols_filter(plat_name, plat_name, filters_list):
            broker_name = None
            for rule in translations:
                if rule.get("symbol") == plat_name:
                    broker_name = rule.get("source")
                    break
            if not broker_name:
                broker_name = plat_name.split('\\')[-1]

            broker_upper = broker_name.upper()
            if broker_upper in broker_sym_map:
                actual_name = broker_sym_map[broker_upper]["name"]
                if plat_name not in matched_platform_paths:
                    matched_platform_paths.append(plat_name)
                if actual_name not in subscribed_broker_symbols:
                    subscribed_broker_symbols.append(actual_name)

    return matched_platform_paths, subscribed_broker_symbols

def test_scenarios():
    print("=" * 60)
    print("STARTING DATA FEED MATCHING SCENARIOS TESTS")
    print("=" * 60)

    # Print the available symbols list showing the groups, subgroups, and base symbols
    print("AVAILABLE PLATFORM SYMBOLS FOR TESTING:")
```

```
for sym in mock_platform_symbols:
    path_parts = sym["symbol"].split('\\')
    if len(path_parts) &gt; 1:
        group = path_parts[0]
        subgroup = "\\".join(path_parts[1:-1]) if len(path_parts) &gt; 2 else ""
        base = path_parts[-1]
        print(f"  - Group: {group:&lt;10} Subgroup: {subgroup:&lt;12} Symbol: {base}")
    else:
        print(f"  - Group: {'[None]':&lt;10} Subgroup: {'':&lt;12} Symbol: {sym['symbol']}")
print("=" * 60)

scenarios = [
    ("Match Everything (*)", "*",
     ["crypto\\BTCUSD", "forex\\EURUSD", "GBPUSD", "Metals\\XAUUSD", "Metals\\XAGUSD", "forex\\Major\\
     ["BTCUSD", "EURUSD", "GBPUSD", "XAUUSD", "XAGUSD", "USDJPY", "USDTRY", "HISUSD", "CL_OIL", "US30"

    ("Match Forex Only (forex\\*)", "forex\\*",
     ["forex\\EURUSD", "forex\\Major\\USDJPY", "forex\\Exotics\\USDTRY"],
     ["EURUSD", "USDJPY", "USDTRY"]),

    ("Match Metals Only (Metals\\*)", "Metals\\*",
     ["Metals\\XAUUSD", "Metals\\XAGUSD"],
     ["XAUUSD", "XAGUSD"]),

    ("Literal Symbol Path (forex\\EURUSD)", "forex\\EURUSD",
     ["forex\\EURUSD"],
     ["EURUSD"]),

    ("Simple Base Symbol (GBPUSD)", "GBPUSD",
     ["GBPUSD"],
     ["GBPUSD"]),

    ("Wildcard Folder with Exclusion (forex\\*,!forex\\Exotics\\*)", "forex\\*,!forex\\Exotics\\*",
     ["forex\\EURUSD", "forex\\Major\\USDJPY"],
     ["EURUSD", "USDJPY"]),

    ("Literal Sub-group Path (forex\\Major\\*)", "forex\\Major\\*",
     ["forex\\Major\\USDJPY"],
     ["USDJPY"]),

    ("Custom Symbol Path without matching (custom\\*)", "custom\\*",
     ["custom\\HISUSD"],
     ["HISUSD"]),
]

passed = 0
for title, filter_str, expected_plat, expected_broker in scenarios:
    res_plat, res_broker = run_resolution_test(filter_str)
    success = (sorted(res_plat) == sorted(expected_plat)) and (sorted(res_broker) == sorted(expected_b
    print(f"Scenario: {title}")
    print(f"  Filter: {filter_str}")
    print(f"  Matched Platform Paths: {res_plat}")
    print(f"  Subscribed Broker Symbols: {res_broker}")
    print(f"  Status: {'PASSED' if success else 'FAILED'}")
    print("-" * 40)
    if success:
        passed += 1

# Translation scenario (HISUSD maps to source EURUSD)
print("Scenario: Custom Translation (HISUSD maps to source EURUSD)")
trans = [{"symbol": "custom\\HISUSD", "source": "EURUSD"}]
res_plat, res_broker = run_resolution_test("custom\\*", trans)
expected_plat_trans = ["custom\\HISUSD"]
expected_broker_trans = ["EURUSD"]
success_trans = (sorted(res_plat) == sorted(expected_plat_trans)) and (sorted(res_broker) == sorted(ex
print(f"  Matched Platform Paths: {res_plat}")
print(f"  Subscribed Broker Symbols: {res_broker}")
print(f"  Status: {'PASSED' if success_trans else 'FAILED'}")
```

```
print("-" * 40)
if success_trans:
    passed += 1

# New Translation scenario (EURUSD.ai maps to source EURUSD)
print("Scenario: Custom Translation (EURUSD.ai maps to source EURUSD)")
trans_ai = [{"symbol": "forex\\EURUSD.ai", "source": "EURUSD"}]
res_plat_ai, res_broker_ai = run_resolution_test("forex\\EURUSD.ai", trans_ai)

# Simulate quote tick translation matching resolve_and_translate_tick
incoming_tick_symbol = "EURUSD"
incoming_bid, incoming_ask = 1.0912, 1.0915
translated_symbol = None
for rule in trans_ai:
    if rule.get("source") == incoming_tick_symbol:
        translated_symbol = rule.get("symbol")
        break

success_ai = (sorted(res_plat_ai) == sorted(["forex\\EURUSD.ai"])) and (sorted(res_broker_ai) == sorte
print(f"  Matched Platform Paths: {res_plat_ai}")
print(f"  Subscribed Broker Symbols: {res_broker_ai}")
print(f"  Simulating Tick Translation: {incoming_tick_symbol} ({incoming_bid}/{incoming_ask}) -&gt; {t
print(f"  Status: {'PASSED' if success_ai else 'FAILED'}")
print("-" * 40)
if success_ai:
    passed += 1

# User's Custom Symbol Translation scenario (EUTUSD maps to source EURUSD)
print("Scenario: User Custom Translation (EUTUSD maps to source EURUSD)")
# Platform symbol list has 'forex\\EUTUSD' (we mock adding it if not exists)
mock_platform_symbols.append({"symbol": "forex\\EUTUSD", "digits": 5})
trans_user = [{"symbol": "EUTUSD", "source": "EURUSD"}]

# Run resolution matching
filters_list_user = ["forex\\*"]
res_plat_user = []
res_broker_user = []
broker_sym_map = {s["name"].upper(): s for s in mock_broker_symbols}
for p_sym in mock_platform_symbols:
    plat_name = p_sym["symbol"]
    if match_symbols_filter(plat_name, plat_name, filters_list_user):
        broker_name = None
        for rule in trans_user:
            rule_sym = rule.get("symbol")
            if rule_sym == plat_name or rule_sym == plat_name.split('\\')[-1]:
                broker_name = rule.get("source")
                break
        if not broker_name:
            broker_name = plat_name.split('\\')[-1]
        broker_upper = broker_name.upper()
        if broker_upper in broker_sym_map:
            actual_name = broker_sym_map[broker_upper]["name"]
            if plat_name not in res_plat_user:
                res_plat_user.append(plat_name)
            if actual_name not in res_broker_user:
                res_broker_user.append(actual_name)

# Simulate tick resolution matching resolve_and_translate_tick DB fallback logic
local_symbol_user = None
for rule in trans_user:
    target_pat = rule.get("symbol")
    source_pat = rule.get("source")
    translated = target_pat if "EURUSD" == source_pat else None
    if translated:
        # check if translated matches EUTUSD exactly or as folder prefix
        for p_sym in mock_platform_symbols:
            if p_sym["symbol"] == translated or p_sym["symbol"].endswith("\\" + translated):
                local_symbol_user = p_sym["symbol"]
```

```
                    break
            break

    success_user = ("forex\\EUTUSD" in res_plat_user) and ("EURUSD" in res_broker_user) and (local_symbol_
    print(f"  Matched Platform Paths: {res_plat_user}")
    print(f"  Subscribed Broker Symbols: {res_broker_user}")
    print(f"  Simulating Tick Translation: EURUSD -&gt; {local_symbol_user}")
    print(f"  Status: {'PASSED' if success_user else 'FAILED'}")
    print("-" * 40)
    if success_user:
        passed += 1

    # Stress Test Scenario: matching over 10,000 platform symbols to check performance
    print("Scenario: Stress Test (Matching rules against 10,000 platform symbols)")
    large_platform_symbols = []
    for idx in range(10000):
        group = ["forex", "crypto", "Metals", "indices", "stocks"][idx % 5]
        large_platform_symbols.append({"symbol": f"{group}\\SYM_{idx}", "digits": 5})

    start_time = time.time()
    # Match large list against 'forex\\*' filter
    filters_list = ["forex\\*"]
    matches_count = 0
    # Capture local function for benchmark speed
    local_match = match_symbols_filter
    for p_sym in large_platform_symbols:
        if local_match(p_sym["symbol"], p_sym["symbol"], filters_list):
            matches_count += 1

    duration = time.time() - start_time
    print(f"  Matched {matches_count} symbols out of 10,000 in {duration:.4f} seconds.")
    # Check if matched count is 2000 (since idx % 5 matches forex 1/5 of the time)
    success_stress = matches_count == 2000 and duration &lt; 0.1 # Should complete in less than 100ms
    print(f"  Status:   {'PASSED' if success_stress else 'FAILED'}")
    print("=" * 60)
    if success_stress:
        passed += 1

    total_tests = len(scenarios) + 3
    print(f"TEST RESULTS: {passed}/{total_tests} SCENARIOS PASSED")
    print("=" * 60)

if __name__ == "__main__":
    test_scenarios()
```

■ history_server\scratch\test_frontend_logic.py

```
# test_frontend_logic.py
import re

def simulate_tree_select(current_filter: str, selected_path: str, is_folder: bool) -&gt; str:
    # 1. Determine the path value just like React logic
    val = selected_path
    if is_folder:
        val = "*" if selected_path == "*" else f"{selected_path}\\*"

    # 2. Simulate the React setSymbolsFilter updates
    trimmed = current_filter.strip()
    if not trimmed:
        return val

    parts = [p.strip() for p in trimmed.split(',')]
    if val in parts:
        return current_filter  # Prevent duplicate appends

    return f"{trimmed},{val}"

# Test Cases
print("=" * 60)
print("TESTING FRONTEND STRING MANIPULATION LOGIC (SIMULATED IN PYTHON)")
print("=" * 60)

# Test 1: User types in raw text directly (no rows created, just a single string)
user_input = "forex\*,!forex\Exotics\*"
print(f"Test 1 - Direct Textarea Input: '{user_input}'")
print(f"  Result: '{user_input}'")
print("  Status: PASSED (No rows list exists to split it)")
print("-" * 60)

# Test 2: Tree select starting from empty state
f_empty = ""
f_res1 = simulate_tree_select(f_empty, "forex", True)
print(f"Test 2 - Select folder 'forex' on empty filter:")
print(f"  Result: '{f_res1}'")
assert f_res1 == "forex\\*"
print("  Status: PASSED")
print("-" * 60)

# Test 3: Tree select selecting a folder when filter already has entries
f_current = "forex\\*,!forex\\Exotics\\*"
f_res2 = simulate_tree_select(f_current, "Metals", True)
print(f"Test 3 - Select folder 'Metals' when filter is '{f_current}':")
print(f"  Result: '{f_res2}'")
assert f_res2 == "forex\\*,!forex\\Exotics\\*,Metals\\*"
print("  Status: PASSED")
print("-" * 60)

# Test 4: Prevent duplicate selections from tree
f_duplicate = "forex\\*,!forex\\Exotics\\*"
f_res3 = simulate_tree_select(f_duplicate, "forex", True)
print(f"Test 4 - Select folder 'forex' again (duplicate prevention):")
print(f"  Result: '{f_res3}'")
assert f_res3 == f_duplicate
print("  Status: PASSED")
print("=" * 60)
print("FRONTEND LOGIC TESTS: ALL PASSED")
print("=" * 60)
```

■ main_server\main.py

```
import os
import uvicorn

def main():
    """
    Main entry point for starting the MT5 Trade Server web API.
    """
    host = os.getenv("HOST", "127.0.0.1")
    port = int(os.getenv("PORT", "8000"))
    reload = os.getenv("RELOAD", "false").lower() == "true"

    print("=" * 70)
    print("STARTING METATRADER 5 REBUILD TRADE SERVER (mt5trade64.exe)")
    print(f"Server host: {host}")
    print(f"Server port: {port}")
    print("=" * 70)

    uvicorn.run("webapi.manager_api:app", host=host, port=port, reload=reload)

if __name__ == "__main__":
    main()
```

■ main_server\main_server.log

```
=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■
■
■S■T■A■R■T■I■N■G■ ■M■E■T■A■T■R■A■D■E■R■ ■5■ ■R■E■B■U■I■L■D■ ■T■R■A■D■E■ ■S■E■R■V■E■R■ ■(■m■t■
■
■S■e■r■v■e■r■ ■h■o■s■t■:■ ■1■2■7■.■0■.■0■.■1■
■
■S■e■r■v■e■r■ ■p■o■r■t■:■ ■8■0■0■0■
■
■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■=■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
```

■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■

■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■

■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■

```
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:56538 - "GET /api/v1/positi
INFO:     127.0.0.1:56538 - "GET /api/v1/order
INFO:     127.0.0.1:56538 - "GET /api/v1/balan
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:60460 - "GET /internal/gat
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:56538 - "GET /admin/ticks
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:56538 - "GET /api/v1/positi
INFO:     127.0.0.1:56538 - "GET /api/v1/order
INFO:     127.0.0.1:56538 - "GET /api/v1/balan
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
INFO:     127.0.0.1:57741 - "POST /internal/ti
```

```
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■P■O■S■T■ ■/■a■d■m■i■n■/■g■r■o■u■p■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■2■5■0■6■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■g■a■t■e■w■a■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■g■r■o■u■p■s■
```

```
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
```

■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■

```
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
```

```
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
```

```
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■d■m■i■n■/■t■i■c■k■s■ ■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■p■o■s■i■t■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■o■r■d■e■r■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■6■5■3■8■ ■-■ ■"■G■E■T■ ■/■a■p■i■/■v■1■/■b■a■l■a■n■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
■
■I■N■F■O■:■ ■ ■ ■ ■ ■1■2■7■.■0■.■0■.■1■:■5■7■7■4■1■ ■-■ ■"■P■O■S■T■ ■/■i■n■t■e■r■n■a■l■/■t■i■
```

... [truncated for PDF size]

■ main_server\requirements.txt

```
fastapi
uvicorn[standard]
asyncpg
pydantic
numpy
orjson
uvloop
pytest
pytest-asyncio
```

■ main_server\test_accounts.db

```
SQLite format 3■■■■■■■@  ■■■+■■■■■■■■■■■■■■■■%■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■&amp;■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■tabledealsdeals■CREATE TABLE deals (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    order_ticket INTEGER REFERENCES orders(ticket),
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price DECIMAL(18, 8) NOT NULL,
    commission DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    action INT NOT NULL,
    entry INT NOT NULL,
    reason INT NOT NULL DEFAULT 0,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)J■■■■■■otableordersorders■CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)A■■■■■■Utableaccountsaccounts■CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)■■■■■■■■]tablesymbolssymbols■CREATE TABLE symbols (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0,
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)■■■■■■■■■■tablegroupsgroups■CREATE TABLE groups (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
```

```
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)P■■■++■Ytablesqlite_sequencesqlite_sequence■CREATE TABLE sqlite_sequence(name,seq)■■c?■■■indexsqlite_a
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_overri■■■■■■■■■■-■■■A■■■indexsqlite_autoindex_symbols_1symbols■■■■■■■■■■+■■■?■■■indexsqlite_
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■!■demo_group
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■f■■■■■■■■■
■Y■*■■■D■m■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE TABLE positions (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price_open DECIMAL(18, 8) NOT NULL,
    price_current DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    action INT NOT NULL,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, U■■■■■■■}tableaccountsaccounts■CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
, settings_json TEXT)J■■■■■■■otableordersorders■CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
)■■■■■■■Utableaccountsaccounts■CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)-■■■A■■■indexsqlite_autoindex_symbols_1symbols■■■■■■■■]tablesymbolssymbols■CREATE TABLE symbols (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0,
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)+■■■?■■■indexsqlite_autoindex_groups_1groups■■■■■■■■■tablegroupsgroups■CREATE TABLE groups (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)P■■■++■Ytablesqlite_sequencesqlite_sequence■CREATE TABLE sqlite_sequence(name,seq)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■
■u■


4■■■■■■■u■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

... [truncated for PDF size]
```

■ main_server\test_phase7_server.db

```
SQLite format 3■■■■■■■@  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■T■■■■■■■wtablepositionspositions
CREATE TABLE positions (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price_open DECIMAL(18, 8) NOT NULL,
    price_current DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    action INT NOT NULL,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    time_create TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)R■■■■■■■■■tabledealsdeals■CREATE TABLE deals (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    order_ticket INTEGER REFERENCES orders(ticket),
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price DECIMAL(18, 8) NOT NULL,
    commission DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    action INT NOT NULL,
    entry INT NOT NULL,
    reason INT NOT NULL DEFAULT 0,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)J■■■■■■■otableordersorders■CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)A■■■■■■■Utableaccountsaccounts■CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)^■■■■■■■■■tablesymbolssymbols■CREATE TABLE symbols (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0,
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)-■■■A■■■indexsqlite_autoindex_symbols_1symbols■P■■■++■Ytablesqlite_sequencesqlite_sequence■CREATE TABL
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)+■■■?■■■indexsqlite_autoindex_groups_1groups■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■!■test_group
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■orders■■■■■■positions■■■
■deals■■■■■deals■
■■■■symbols■■■■■groups
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FRI,00:00-23:592026-08-19 10:29:48
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■ ■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■N■
■l
■■■■■]■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■T■■■■■■wtableposP
■■-■■iindexidx_orders_loginorders■CREATE INDEX idx_orders_login ON orders(login)T■■■■■■wtablepositionsp
CREATE TABLE positions (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price_open DECIMAL(18, 8) NOT NULL,
    price_current DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    action INT NOT NULL,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    time_create TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)R■■■■■■■■tabledealsdeals■CREATE TABLE deals (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    order_ticket INTEGER REFERENCES orders(ticket),
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price DECIMAL(18, 8) NOT NULL,
    commission DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    action INT NOT NULL,
    entry INT NOT NULL,
    reason INT NOT NULL DEFAULT 0,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
)J■■■■■■otableordersorders■CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)A■■■■■■Utableaccountsaccounts■CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)^■■■■■■■tablesymbolssymbols■CREATE TABLE symbols (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0,
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)-■■■A■■■indexsqlite_autoindex_symbols_1symbols■P■■■++■Ytablesqlite_sequencesqlite_sequence■CREATE TABL
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)+■■■?■■■indexsqlite_autoindex_groups_1groups■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

... [truncated for PDF size]
```

■ main_server\trade_server.db

(Empty file)

■ main_server\bases\broker.db

```
SQLite format 3■■■■■■■@  ■■■@■■■3■■■■■■■■■■■■■D■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■U■■■■■■■■■■P■■■-■■iindexidx_orders_loginorders■CT■■■■■■■wtablepositionspositions
CREATE TABLE positions (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price_open DECIMAL(18, 8) NOT NULL,
    price_current DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    action INT NOT NULL,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    time_create TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)R
■■■■■■tabledealsdeals■CREATE TABLE deals (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    order_ticket INTEGER REFERENCES orders(ticket),
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price DECIMAL(18, 8) NOT NULL,
    commission DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    action INT NOT NULL,
    entry INT NOT NULL,
    reason INT NOT NULL DEFAULT 0,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)J■■■■■■■otableordersorders■CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)A■■■■■■■Utableaccountsaccounts■CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
)^■■■■■■■tablesymbolssymbols■CREATE TABLE symbols (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0,
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)-■■■A■■■indexsqlite_autoindex_symbols_1symbols■z■■■■■■Otablegroupsgroups■CREATE TABLE groups (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)+■■■?■■■indexsqlite_autoindex_groups_1groups■■■■■■■■■P■■■++■Ytablesqlite_sequencesqlite_sequence■CRE
■■■■■■3_real\reald2■2026-08-19 18:12:05{"currency":"USD","digits":2,"trade_server":"MetaQuotes-Demo","au
■■■■■■3_rahul\dmod2■2026-08-19 18:11:43{"currency":"USD","digits":2,"trade_server":"MetaQuotes-Demo","au
■■■■h■■h■■■■■{■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■real\test■
■■■real\real■
■■■rahul\dmo■■■1■routing_test_group■
■!■demo_group
■■■■u■■■■■■■■■u■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■positions(
■■■■groups■■■■■■symbol
■■■■deals■■■■■■■■■■■■0■■■■■■■■■■■■■■k■■■%■■■■■■+3[forex\XAUUSD■■USD
MON,00:00-24:00;TUE,00:00-24:00;WED,00:00-24:00;THU,00:00-24:00;FRI,00:00-24:002026-08-20 15:35:11{"descri
MON,00:00-24:00;TUE,00:00-24:00;WED,00:00-24:00;THU,00:00-24:00;FRI,00:00-24:002026-08-20 15:35:03{"descri
MON,00:00-23:592026-08-20 15:06:22{"is_dummy":true}S■■■%■■■■■■+3/forex\.dummy■■USD
MON,00:00-23:592026-08-19 19:54:15{"is_dummy":true}T■■■'■■■■■■+3/crypto\.dummy■■USD
MON,00:00-2■■■■*■■■ (■■■#,■■■■$■■■"■■■■■■■■■■■■■■■■■
■S■■■g■■H■■■g■y■■■■■■■p■^■■■■/■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■Custom\EU(■■'■Custom\.dummy'■■3■forex\GBPUSD\.dummy$
■■■EURUSD-a
■■■EURNOK,■■5■forex\Exotics\NZDUSD!■■'■Metals\.dummy■■■5■forex\Exotics\USDJPY ■■5■forex\Exotics\.dummy
■■■AUDUSD■■■■GBPUSD■■■'■crypto\ETHUSD■■■'■crypto\BTCUSD■■■%■forex\GBPUSD&amp;■■'■Metals\XAGUSD■■■%■
■■■■■;■
o■;■A■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■3■33■AEURUSD?zG■{?zG■{?,&lt;■2026-08-19 12:48:302026-08-19 12:48:302026-08-19 12:48:30l■
■■■■■■■■■■■■■3■33■AEURUSD?zG■{?zG■{?,&lt;■2026-08-19 12:47:072026-08-19 12:47:072026-08-19 12:47:07w■
■■■(y■■■!a■■■■A■■■'K■■■■1■■■■■■■■■■■■■■■■2■■■■■O■■


U
■■■■]■!■■■l■'


q
,■■■]■■■■■I■■■■■D■■■■■T■■■■■E■■■■y■&lt;■■■}■;■■■s■/■■■S■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
m&amp;h2026-08-21 09:00:57J9■■■■%■■■■■■■■■3■■Bforex\GBPUSD??0■#
m&amp;h2026-08-21 09:00:56B8■■■■%■■■■■■■■■3■■Bforex\GBPUSD??]ce2026-08-21 08:59:36B7■■■■%■■■■■■■■■3■■
Ri2026-08-19 13:45:31:
■■■■■■■■■■■■■3+'■EURUSD?zG■{?ud02026-08-19 13:45:30B■■■■■■■■■■■■■■■3*'■EURUSD?zG■{?ud0zG■{2026-08-
■■■■■■■■■■■■■3!'■EURUSD?zG■{?$6e■2026-08-19 13:37:07;■■■■■■■■■■■■■■■3■■BEURUSD?zG■{?■■2026-08-19 1
■■■■■■■■#■■
t■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
=p2026-08-21 06:14:112026-08-21 09:04:18
■c■'■■■■■■■■■■33■AMetals\XAGUSD?zG■{@Q6■■1'@Qs■■y@
=q2026-08-21 06:13:492026-08-21 nm■■■■%■■■■■■■■■■33■Cforex\EURUSD??Y|?S|@]B\(2026-08-21 08:56:592026-0
■■g■■■■■g■■■■■■■■■■■■■■■33■AEURNOK?zG■{@%'=["@%(M■@1■R@[@q■2026-08-21 06:56:032026-08-21 09:08:33g■
0j@eY@■■■I^2026-08-21 00:37:462026-08-21 09:08:52g■■■■■■■■■■■■■■■33■AEURUSD??pJim?`■■@U■■■@[?u
4■2026-08-19 10:29:142026-08-21 09:10:43g■■■■■■■■■■■■■■■33■BEURNOK?zG■{@%■3H+@%&amp;@/■■■■■■@[@[■62
■■■%■l■■■■%■■■■■■■■■■33'■forex\EURUSD?zG■{?k;&lt;?■`■■\(@'eF
```

L02026-08-21 07:01:142026-08-21 09:04:40■<■■%■■■■■■■■■■^■■■■'■■■■■■■■■■■33■ACustom\GBPUSD?zG■{?9?
;%?5■t
@t@'c"2026-08-21 12:28:272026-08-21 12:32:36m'■■■%■■■■■■■■■■■33■Bforex\EURUSD?Q■?■■?9X■bN?zG■{@A=■K^2
■■■■■■■■■
■■_■W■G■?■■■■■■■■■■2■■■■■■■■■■■■

■■■■■■■■■■■
K
1
)
!
■■■■■■■■■■■■■■■■■■■■■■■■■■■w■o■g■O■7■/■'■■■■■■■■■■■■■■■■■■■■■■■■■■■y■q■i■a■Y■Q■I■A■9■■■■■
■■■■■■■■■■■■{■s■k■c■[■S■K■C■;■3■+■#■■
x
p
h
`
X
P■■■■

w
n
e
\
S
B
9
■
■■■■|■s■j■■*■"

■w■o■g■_■W■

■■■■■■■■■a■X■O■F■=■4■+■"■■■■■■■■■■■■■

H
@
8
0
(

■
■
■
■■■■■■■■■■■O■G■?■7■/■'■■■■■■■■■■■■■■■■■■z■q■h■_■V■M■D■;■2■)■ ■■■■■■

■■

```
■■■■■A■■■■■■A■■■■■■A
■■■■■A■■■■■■A■■■■■■A■■■■■■A■■■■■■B■■■■■■A■■■■■■A■■■■■■A■■■■■■A
■■■■■■■J■y■
■■■■■]■■■■■■■■■■P■■■■-■■iindexidx_orders_loginorders■CT■■■■■■■wtableposP■■■■-■■iindexidx_orders_logino
CREATE TABLE positions (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price_open DECIMAL(18, 8) NOT NULL,
    price_currenr■■■■■■■;tablesymbolssymbols■CREATE TABLE symbols (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0,
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
, settings_json TEXT)■■■■■■■■wtablegroupsgroups■CREATE TABLE groups (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
, settings_json TEXT)
■4■■■■otableordersorders■CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFc■■■■■■!tableordersorders■CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
, assigned_dealer INTEGER)U■■■■■■■}tableaccountsaccounts■CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
```

```
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
, settings_json TEXT)-■■■A■■■indexsqlite_autoindex_symbols_1symbols■■■}■■■■Otablegroupsgroups■CREATE T
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)+■■■?■■■indexsqlite_autoindex_groups_1groups■■■■■■■■■P■■■++■Ytablesqlite_sequencesqlite_sequence■CRE
■■■■■■■{■u
■t■m■_■X■■■
■■■■■■■■■■■■■■■■■{■d■E■&gt;■■■■■■■■■■

n
g
■
■■■■■■■■■■■■■■■■■■■■{■f■Q■J■B■:■2■*■"■■■■■■■■\■.■&amp;■■■■■■■■■■■■■=■5■-■%■■■■■
■■

u
_
W
O
G■■■■T■L■6■■■■■■e■]■U■M■E

}
?
7
/
'
■
■
■■■■■■■s■k■m
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

... [truncated for PDF size]
```

■ main_server\bases\connection.py

```
import os
import re
import asyncio
import logging
import sqlite3
from typing import Optional, Any, Dict, List

logger = logging.getLogger(__name__)

# Try to import asyncpg
try:
    import asyncpg
except ImportError:
    asyncpg = None

class SQLiteConnectionWrapper:
    def __init__(self, conn: sqlite3.Connection):
        self._conn = conn

    def _convert_query(self, query: str) -&gt; str:
        # Convert Postgres $1, $2 placeholders to SQLite ? placeholders
        query = re.sub(r'\$\d+', '?', query)
        # Convert PG specific standard functions like NOW() to SQLite datetime('now')
        query = query.replace("NOW()", "datetime('now')")
        return query

    async def fetch(self, query: str, *args) -&gt; List[Dict[str, Any]]:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            return [dict(row) for row in cursor.fetchall()]
        return await loop.run_in_executor(None, _run)

    async def fetchrow(self, query: str, *args) -&gt; Optional[Dict[str, Any]]:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            row = cursor.fetchone()
            return dict(row) if row else None
        return await loop.run_in_executor(None, _run)

    async def fetchval(self, query: str, *args) -&gt; Any:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            row = cursor.fetchone()
            if row:
                return row[0]
            return None
        return await loop.run_in_executor(None, _run)

    async def execute(self, query: str, *args) -&gt; None:
        loop = asyncio.get_event_loop()
        def _run():
            cursor = self._conn.cursor()
            cursor.execute(self._convert_query(query), args)
            self._conn.commit()
        await loop.run_in_executor(None, _run)

    class TransactionContext:
        def __init__(self, conn: sqlite3.Connection):
            self._conn = conn
```

```
        async def __aenter__(self):
            return self

        async def __aexit__(self, exc_type, exc_val, exc_tb):
            if exc_type:
                self._conn.rollback()
            else:
                self._conn.commit()

    def transaction(self):
        return self.TransactionContext(self._conn)

class SQLitePoolWrapper:
    """Mock asyncpg Pool wrapper for SQLite."""
    def __init__(self, db_path: str):
        self.db_path = db_path
        self._conn = None

    def initialize(self):
        # Register Decimal adapter and converter
        from decimal import Decimal
        sqlite3.register_adapter(Decimal, lambda d: str(d))
        sqlite3.register_converter("DECIMAL", lambda b: Decimal(b.decode('utf-8')))

        self._conn = sqlite3.connect(
            self.db_path,
            check_same_thread=False,
            detect_types=sqlite3.PARSE_DECLTYPES
        )
        self._conn.row_factory = sqlite3.Row

    def close(self):
        if self._conn:
            self._conn.close()

    class ConnectionContext:
        def __init__(self, conn_wrapper: SQLiteConnectionWrapper):
            self.wrapper = conn_wrapper

        async def __aenter__(self) -&gt; SQLiteConnectionWrapper:
            return self.wrapper

        async def __aexit__(self, exc_type, exc_val, exc_tb):
            pass

    def acquire(self) -&gt; ConnectionContext:
        return self.ConnectionContext(SQLiteConnectionWrapper(self._conn))

class DBConnection:
    pool: Optional[Any] = None
    is_sqlite: bool = False

    @classmethod
    async def initialize(cls):
        """Initialize connection pool (Postgres asyncpg or SQLite fallback)."""
        db_type = os.getenv("DB_TYPE", "sqlite") # Default to SQLite for local ease

        if db_type == "postgres" and asyncpg is not None:
            dsn = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/broker_db")
            try:
                cls.pool = await asyncpg.create_pool(
                    dsn=dsn, min_size=5, max_size=20, timeout=30.0
                )
                cls.is_sqlite = False
                logger.info("Initialized PostgreSQL connection pool successfully.")
                return
            except Exception as e:
                logger.warning(f"Failed to connect to PostgreSQL. Falling back to SQLite. Error: {e}")
```

```
    # Fallback to SQLite - resolve to the bases/ folder dynamically next to connection.py
    db_path = os.getenv("SQLITE_DB_PATH", os.path.join(os.path.dirname(__file__), "broker.db"))
    cls.pool = SQLitePoolWrapper(db_path)
    cls.pool.initialize()
    cls.is_sqlite = True
    logger.warning("=" * 70)
    logger.warning("■■  WARNING: DEVELOPER-ONLY SQLITE DATABASE ACTIVE! ■■")
    logger.warning("SQLite is strictly for local developer testing. Do not use in production.")
    logger.warning(f"Database file resolved to: {db_path}")
    logger.warning("=" * 70)

@classmethod
async def close(cls):
    """Close connection pool."""
    if cls.pool is not None:
        if cls.is_sqlite:
            cls.pool.close()
        else:
            await cls.pool.close()
        cls.pool = None

@classmethod
def get_pool(cls) -&gt; Any:
    if cls.pool is None:
        raise RuntimeError("Database pool has not been initialized. Call initialize() first.")
    return cls.pool
```

■ main_server\bases\schema.sql

```
-- PostgreSQL Schema matching MT5-style Database Structures

DROP TABLE IF EXISTS positions CASCADE;
DROP TABLE IF EXISTS deals CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS accounts CASCADE;
DROP TABLE IF EXISTS group_symbols CASCADE;
DROP TABLE IF EXISTS symbols CASCADE;
DROP TABLE IF EXISTS routing_rules CASCADE;
DROP TABLE IF EXISTS gateways CASCADE;
DROP TABLE IF EXISTS groups CASCADE;

-- Groups Table (Defines trading tiers, limits, margin calls)
CREATE TABLE groups (
    id SERIAL PRIMARY KEY,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,       -- Call level (e.g. 50.00%)
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,   -- Stop-out level (e.g. 30.00%)
    spread_override INT NOT NULL DEFAULT 0,                 -- Override spread markup in points
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Symbols Table (Defines instruments, sizes, margins)
CREATE TABLE symbols (
    id SERIAL PRIMARY KEY,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,                          -- Decimal places (e.g. 5 for EURUSD)
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0, -- Lot contract size (e.g. 100k)
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,     -- Margin multiplier
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,                    -- Base spread markup in points
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Accounts Table (Accounts associated with a group setting)
CREATE TABLE accounts (
    login INT PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name) ON DELETE RESTRICT,
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    status INT NOT NULL DEFAULT 0,                          -- 0: Normal, 1: ReadOnly, 2: Blocked
    so_activation INT NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Orders Table (Holds active and completed orders)
CREATE TABLE orders (
    ticket SERIAL PRIMARY KEY,
    login INT REFERENCES accounts(login) ON DELETE CASCADE,
    symbol VARCHAR(32) REFERENCES symbols(symbol) ON DELETE RESTRICT,
    volume DECIMAL(18, 8) NOT NULL,                         -- Initial lot volume
    volume_current DECIMAL(18, 8) NOT NULL,                 -- Remaining unfilled volume
    price_order DECIMAL(18, 8) NOT NULL,                    -- Requested price
```

```
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,                                      -- EnOrderType: 0=OP_BUY, 1=OP_SELL, 2=OP_BUY_
    state INT NOT NULL DEFAULT 0,                           -- EnOrderState: 0=STARTED, 4=FILLED, 5=REJECT
    reason INT NOT NULL DEFAULT 0,                          -- EnOrderReason: 0=ORDER_REASON_CLIENT
    activation_mode INT NOT NULL DEFAULT 0,
    time_setup TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Deals Table (Immutable execution history logs)
CREATE TABLE deals (
    ticket SERIAL PRIMARY KEY,
    order_ticket INT REFERENCES orders(ticket) ON DELETE SET NULL,
    login INT REFERENCES accounts(login) ON DELETE CASCADE,
    symbol VARCHAR(32) REFERENCES symbols(symbol) ON DELETE RESTRICT,
    volume DECIMAL(18, 8) NOT NULL,
    price DECIMAL(18, 8) NOT NULL,                          -- Realized fill price
    commission DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,     -- Swaps (Accumulated storage charges)
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,      -- P&amp;amp;L realized
    action INT NOT NULL,                                    -- EnDealAction: 0=DEAL_BUY, 1=DEAL_SELL, 2=D
    entry INT NOT NULL,                                     -- EnDealEntry: 0=ENTRY_IN, 1=ENTRY_OUT, 2=ENT
    reason INT NOT NULL DEFAULT 0,                          -- EnDealReason: 0=DEAL_REASON_CLIENT
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Positions Table (Holds net open position state for calculations)
CREATE TABLE positions (
    ticket SERIAL PRIMARY KEY,
    login INT REFERENCES accounts(login) ON DELETE CASCADE,
    symbol VARCHAR(32) REFERENCES symbols(symbol) ON DELETE RESTRICT,
    volume DECIMAL(18, 8) NOT NULL,
    price_open DECIMAL(18, 8) NOT NULL,
    price_current DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    action INT NOT NULL,                                    -- EnPositionAction: 0=POSITION_BUY, 1=POSITI
    activation_mode INT NOT NULL DEFAULT 0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.00000000,     -- Swap
    time_create TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    time_update TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Indexes for performance
CREATE INDEX idx_orders_login ON orders(login);
CREATE INDEX idx_deals_login ON deals(login);
CREATE INDEX idx_positions_login ON positions(login);

-- Group Symbols Table (per-group symbol settings overrides)
CREATE TABLE group_symbols (
    id SERIAL PRIMARY KEY,
    group_name VARCHAR(64) NOT NULL REFERENCES groups(name) ON DELETE CASCADE,
    symbol VARCHAR(32) NOT NULL REFERENCES symbols(symbol) ON DELETE CASCADE,
    spread_diff INT DEFAULT 0,
    commission_rate DECIMAL(18, 8) DEFAULT 0.00000000,
    margin_rate DECIMAL(18, 8) DEFAULT 1.00000000,
    trade_allowed INT DEFAULT 1,
    UNIQUE (group_name, symbol)
);

-- Gateways Table
CREATE TABLE gateways (
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(64) UNIQUE NOT NULL,
    type VARCHAR(32) NOT NULL,
    host VARCHAR(256),
    port INT,
    username VARCHAR(128),
    api_key TEXT,
    is_active INT DEFAULT 1,
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Routing Rules Table
CREATE TABLE routing_rules (
    id SERIAL PRIMARY KEY,
    name VARCHAR(64) NOT NULL,
    priority INT NOT NULL,
    is_enabled INT DEFAULT 1,
    match_groups TEXT, -- JSON string (list of wildcards/names)
    match_symbols TEXT, -- JSON string
    match_order_types TEXT, -- JSON string
    match_volume_min DECIMAL(18, 8),
    match_volume_max DECIMAL(18, 8),
    action VARCHAR(32) NOT NULL, -- INSTANT_EXECUTE, TO_DEALER, TO_GATEWAY, REJECT
    gateway_id INT REFERENCES gateways(id) ON DELETE SET NULL,
    delay_seconds INT DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

■ main_server\bases\schema_sqlite.sql

```
-- SQLite Schema matching MT5-style Database Structures
DROP TABLE IF EXISTS positions;
DROP TABLE IF EXISTS deals;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS accounts;
DROP TABLE IF EXISTS group_symbols;
DROP TABLE IF EXISTS symbols;
DROP TABLE IF EXISTS routing_rules;
DROP TABLE IF EXISTS gateways;
DROP TABLE IF EXISTS groups;

-- Groups Table
CREATE TABLE groups (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    max_leverage INT NOT NULL DEFAULT 100,
    margin_call DECIMAL(6, 2) NOT NULL DEFAULT 50.00,
    margin_stop_out DECIMAL(6, 2) NOT NULL DEFAULT 30.00,
    spread_override INT NOT NULL DEFAULT 0,
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Symbols Table
CREATE TABLE symbols (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol VARCHAR(32) UNIQUE NOT NULL,
    digits INT NOT NULL DEFAULT 5,
    contract_size DECIMAL(18, 8) NOT NULL DEFAULT 100000.0,
    currency VARCHAR(8) NOT NULL DEFAULT 'USD',
    margin_initial DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    margin_maintenance DECIMAL(18, 8) NOT NULL DEFAULT 1.0,
    spread_base INT NOT NULL DEFAULT 10,
    session_hours VARCHAR(256) DEFAULT 'MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FR
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Accounts Table
CREATE TABLE accounts (
    login INTEGER PRIMARY KEY,
    group_name VARCHAR(64) REFERENCES groups(name),
    balance DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    credit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    leverage INT NOT NULL DEFAULT 100,
    equity DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    free_margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin_level DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    status INT NOT NULL DEFAULT 0,
    so_activation INTEGER NOT NULL DEFAULT 0,
    password_hash VARCHAR(128) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Orders Table
CREATE TABLE orders (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    volume_current DECIMAL(18, 8) NOT NULL,
    price_order DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
```

```
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    type INT NOT NULL,
    state INT NOT NULL DEFAULT 0,
    reason INT NOT NULL DEFAULT 0,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    time_setup TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_done TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Deals Table
CREATE TABLE deals (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    order_ticket INTEGER REFERENCES orders(ticket),
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price DECIMAL(18, 8) NOT NULL,
    commission DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    action INT NOT NULL,
    entry INT NOT NULL,
    reason INT NOT NULL DEFAULT 0,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Positions Table
CREATE TABLE positions (
    ticket INTEGER PRIMARY KEY AUTOINCREMENT,
    login INTEGER REFERENCES accounts(login),
    symbol VARCHAR(32) REFERENCES symbols(symbol),
    volume DECIMAL(18, 8) NOT NULL,
    price_open DECIMAL(18, 8) NOT NULL,
    price_current DECIMAL(18, 8) NOT NULL,
    price_sl DECIMAL(18, 8) DEFAULT 0.0,
    price_tp DECIMAL(18, 8) DEFAULT 0.0,
    action INT NOT NULL,
    activation_mode INTEGER NOT NULL DEFAULT 0,
    profit DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    margin DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    storage DECIMAL(18, 8) NOT NULL DEFAULT 0.0,
    time_create TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    time_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_orders_login ON orders(login);
CREATE INDEX idx_deals_login ON deals(login);
CREATE INDEX idx_positions_login ON positions(login);

-- Group Symbols Table (per-group symbol settings overrides)
CREATE TABLE group_symbols (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    group_name VARCHAR(64) NOT NULL REFERENCES groups(name) ON DELETE CASCADE,
    symbol VARCHAR(32) NOT NULL REFERENCES symbols(symbol) ON DELETE CASCADE,
    spread_diff INT DEFAULT 0,
    commission_rate DECIMAL(18, 8) DEFAULT 0.00000000,
    margin_rate DECIMAL(18, 8) DEFAULT 1.00000000,
    trade_allowed INT DEFAULT 1,
    UNIQUE (group_name, symbol)
);

-- Gateways Table
CREATE TABLE gateways (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) UNIQUE NOT NULL,
    type VARCHAR(32) NOT NULL,
    host VARCHAR(256),
```

```
    port INT,
    username VARCHAR(128),
    api_key TEXT,
    is_active INT DEFAULT 1,
    settings_json TEXT DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Routing Rules Table
CREATE TABLE routing_rules (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(64) NOT NULL,
    priority INT NOT NULL,
    is_enabled INT DEFAULT 1,
    match_groups TEXT, -- JSON string (list of wildcards/names)
    match_symbols TEXT, -- JSON string
    match_accounts TEXT, -- JSON string
    match_order_types TEXT, -- JSON string
    match_volume_min DECIMAL(18, 8),
    match_volume_max DECIMAL(18, 8),
    action VARCHAR(32) NOT NULL, -- INSTANT_EXECUTE, TO_DEALER, TO_GATEWAY, REJECT
    gateway_id INTEGER REFERENCES gateways(id) ON DELETE SET NULL,
    delay_seconds INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

■ main_server\core\deal_writer.py

```
import logging
from typing import Dict, Any
from decimal import Decimal
from bases.connection import DBConnection

logger = logging.getLogger(__name__)

class DealWriter:
    @staticmethod
    async def write_deal(
        order_ticket: int,
        login: int,
        symbol: str,
        volume: Decimal,
        price: Decimal,
        action: int,
        entry: int,
        profit: Decimal = Decimal("0.0"),
        commission: Decimal = Decimal("0.0"),
        storage: Decimal = Decimal("0.0"),
        reason: int = 0,
        conn = None
    ) -&gt; int:
        """
        Inserts a deal row into PostgreSQL and updates the account balance atomically.
        - ENTRY_IN: Deducts commission from balance.
        - ENTRY_OUT: Realizes profit, deducts commission, and adds storage (swaps) to balance.
        """
        if conn is not None:
            return await DealWriter._write_deal_impl(
                conn, order_ticket, login, symbol, volume, price, commission, storage, profit, action, ent
            )
        else:
            pool = DBConnection.get_pool()
            async with pool.acquire() as conn:
                async with conn.transaction():
                    return await DealWriter._write_deal_impl(
                        conn, order_ticket, login, symbol, volume, price, commission, storage, profit, act
                    )

    @staticmethod
    async def _write_deal_impl(
        conn, order_ticket: int, login: int, symbol: str, volume: Decimal, price: Decimal,
        commission: Decimal, storage: Decimal, profit: Decimal, action: int, entry: int, reason: int
    ) -&gt; int:
        # 1. Insert Deal log
        ticket = await conn.fetchval(
            """INSERT INTO deals (order_ticket, login, symbol, volume, price, commission, storage, profit,
               VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11)
               RETURNING ticket""",
            order_ticket, login, symbol, volume, price, commission, storage, profit, action, entry, reason
        )

        # 2. Update Account Balance atomically
        if entry == 1: # ENTRY_OUT (Position close)
            # Realize Profit + Apply Commission + Apply Swap (storage)
            await conn.execute(
                """UPDATE accounts
                   SET balance = balance + $1 - $2 + $3, updated_at = datetime('now')
                   WHERE login = $4""",
                profit, commission, storage, login
            )
        else: # ENTRY_IN (Position open)
            # Apply Commission immediately if charged on open
            await conn.execute(
```

```
        """UPDATE accounts
           SET balance = balance - $1, updated_at = datetime('now')
           WHERE login = $2""",
        commission, login
    )

logger.info(f"Deal {ticket} logged for order {order_ticket}: login={login}, profit={profit}, balan
return ticket
```

■ main_server\core\position_keeper.py

```
import logging
from typing import List, Dict, Any
from decimal import Decimal
from datetime import datetime
from bases.connection import DBConnection
from core.interfaces.i_position_keeper import IPositionKeeper

logger = logging.getLogger(__name__)

class PositionKeeper(IPositionKeeper):
    def __init__(self):
        # ticket -&gt; position_dict
        self._positions: Dict[int, Dict[str, Any]] = {}
        # login -&gt; set of tickets
        self._login_tickets: Dict[int, set] = {}

    async def get_positions(self, login: int) -&gt; List[Dict[str, Any]]:
        tickets = self._login_tickets.get(login, set())
        return [self._positions[ticket] for ticket in tickets if ticket in self._positions]

    async def get_all_positions(self) -&gt; List[Dict[str, Any]]:
        return list(self._positions.values())

    async def update_position(
        self, login: int, symbol: str, volume: float, price: float, action: int, deal_entry: int,
        price_sl: float = 0.0, price_tp: float = 0.0, conn = None
    ) -&gt; Dict[str, Any]:
        """
        Updates position in memory and database.
        volume is Decimal or float (we convert to Decimal).
        deal_entry: 0=ENTRY_IN, 1=ENTRY_OUT.
        action: 0=POSITION_BUY, 1=POSITION_SELL (corresponds to deal action).
        """
        vol_dec = Decimal(str(volume))
        price_dec = Decimal(str(price))
        sl_dec = Decimal(str(price_sl))
        tp_dec = Decimal(str(price_tp))

        if conn is not None:
            return await self._update_position_impl(conn, login, symbol, vol_dec, price_dec, action, deal_
        else:
            pool = DBConnection.get_pool()
            async with pool.acquire() as conn:
                async with conn.transaction():
                    return await self._update_position_impl(conn, login, symbol, vol_dec, price_dec, actio

    async def _update_position_impl(
        self, conn, login: int, symbol: str, vol_dec: Decimal, price_dec: Decimal, action: int, deal_entry
        sl_dec: Decimal, tp_dec: Decimal
    ) -&gt; Dict[str, Any]:
        # Check if there is an existing open position for this login + symbol in hedging/netting.
        row = await conn.fetchrow(
            "SELECT * FROM positions WHERE login = $1 AND symbol = $2",
            login, symbol
        )
        existing = dict(row) if row else None

        if deal_entry == 0:  # ENTRY_IN
            if existing:
                # Existing position: check if same direction or opposite
                if existing["action"] == action:
                    # Same direction: increase volume, update average open price and SL/TP
                    new_vol = Decimal(str(existing["volume"])) + vol_dec
                    # Average price = (old_vol * old_price + new_vol * new_price) / (old_vol + new_vol)
                    old_cost = Decimal(str(existing["volume"])) * Decimal(str(existing["price_open"]))
```

```
                new_cost = vol_dec * price_dec
                new_price_open = (old_cost + new_cost) / new_vol

                await conn.execute(
                    """UPDATE positions
                       SET volume = $1, price_open = $2, price_current = $3, price_sl = $4, price_tp =
                       WHERE ticket = $6""",
                    new_vol, new_price_open, price_dec, sl_dec, tp_dec, existing["ticket"]
                )

                # Update memory
                ticket = existing["ticket"]
                self._positions[ticket]["volume"] = new_vol
                self._positions[ticket]["price_open"] = new_price_open
                self._positions[ticket]["price_current"] = price_dec
                self._positions[ticket]["price_sl"] = sl_dec
                self._positions[ticket]["price_tp"] = tp_dec
            else:
                # Opposite direction: reduce volume (netting)
                old_vol = Decimal(str(existing["volume"]))
                if old_vol &gt; vol_dec:
                    # Partial close
                    new_vol = old_vol - vol_dec
                    await conn.execute(
                        "UPDATE positions SET volume = $1, price_current = $2, time_update = datetime(
                        new_vol, price_dec, existing["ticket"]
                    )
                    ticket = existing["ticket"]
                    self._positions[ticket]["volume"] = new_vol
                    self._positions[ticket]["price_current"] = price_dec
                elif old_vol == vol_dec:
                    # Full close
                    await conn.execute("DELETE FROM positions WHERE ticket = $1", existing["ticket"])
                    ticket = existing["ticket"]
                    if ticket in self._positions:
                        del self._positions[ticket]
                    if login in self._login_tickets and ticket in self._login_tickets[login]:
                        self._login_tickets[login].remove(ticket)
                else:
                    # Reverse position
                    new_vol = vol_dec - old_vol
                    new_action = action # opposite action
                    await conn.execute(
                        """UPDATE positions
                           SET volume = $1, price_open = $2, price_current = $3, action = $4, price_sl
                           WHERE ticket = $7""",
                        new_vol, price_dec, price_dec, new_action, sl_dec, tp_dec, existing["ticket"]
                    )
                    ticket = existing["ticket"]
                    self._positions[ticket]["volume"] = new_vol
                    self._positions[ticket]["price_open"] = price_dec
                    self._positions[ticket]["price_current"] = price_dec
                    self._positions[ticket]["action"] = new_action
                    self._positions[ticket]["price_sl"] = sl_dec
                    self._positions[ticket]["price_tp"] = tp_dec
                    self._positions[ticket]["activation_mode"] = 0
        else:
            # Create new position
            ticket = await conn.fetchval(
                """INSERT INTO positions (login, symbol, volume, price_open, price_current, action, pr
                   VALUES ($1, $2, $3, $4, $5, $6, $7, $8, 0) RETURNING ticket""",
                login, symbol, vol_dec, price_dec, price_dec, action, sl_dec, tp_dec
            )

            pos_data = {
                "ticket": ticket,
                "login": login,
                "symbol": symbol,
```

```
                    "volume": vol_dec,
                    "price_open": price_dec,
                    "price_current": price_dec,
                    "action": action,
                    "price_sl": sl_dec,
                    "price_tp": tp_dec,
                    "activation_mode": 0,
                    "profit": Decimal("0.0"),
                    "margin": Decimal("0.0"),
                    "storage": Decimal("0.0")
                }
                self._positions[ticket] = pos_data
                if login not in self._login_tickets:
                    self._login_tickets[login] = set()
                self._login_tickets[login].add(ticket)

        elif deal_entry == 1:  # ENTRY_OUT (Direct close request)
            if existing:
                old_vol = Decimal(str(existing["volume"]))
                if old_vol &gt; vol_dec:
                    new_vol = old_vol - vol_dec
                    await conn.execute(
                        "UPDATE positions SET volume = $1, price_current = $2, time_update = datetime('now
                        new_vol, price_dec, existing["ticket"]
                    )
                    ticket = existing["ticket"]
                    self._positions[ticket]["volume"] = new_vol
                    self._positions[ticket]["price_current"] = price_dec
                else:
                    # Full close
                    await conn.execute("DELETE FROM positions WHERE ticket = $1", existing["ticket"])
                    ticket = existing["ticket"]
                    if ticket in self._positions:
                        del self._positions[ticket]
                    if login in self._login_tickets and ticket in self._login_tickets[login]:
                        self._login_tickets[login].remove(ticket)
            else:
                logger.warning(f"ENTRY_OUT received but no position exists for login={login}, symbol={symb
                raise ValueError("Position not found")
        else:
            raise ValueError(f"Invalid deal_entry: {deal_entry}")

        # Return updated position state
        return self._positions.get(ticket, {})

    async def update_position_activation(self, ticket: int, activation_mode: int, conn = None) -&gt; None:
        """
        Updates the activation mode of an existing position in DB and memory.
        """
        if conn is not None:
            await conn.execute("UPDATE positions SET activation_mode = $1 WHERE ticket = $2", activation_m
        else:
            pool = DBConnection.get_pool()
            async with pool.acquire() as conn:
                await conn.execute("UPDATE positions SET activation_mode = $1 WHERE ticket = $2", activati

        if ticket in self._positions:
            self._positions[ticket]["activation_mode"] = activation_mode

    async def restore_state(self) -&gt; None:
        """Loads active positions from PostgreSQL/SQLite into memory on startup."""
        self._positions.clear()
        self._login_tickets.clear()

        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            rows = await conn.fetch("SELECT * FROM positions")
            for row in rows:
```

```
            pos = dict(row)
            ticket = pos["ticket"]
            login = pos["login"]

            self._positions[ticket] = {
                "ticket": ticket,
                "login": login,
                "symbol": pos["symbol"],
                "volume": Decimal(str(pos["volume"])),
                "price_open": Decimal(str(pos["price_open"])),
                "price_current": Decimal(str(pos["price_current"])),
                "action": pos["action"],
                "price_sl": Decimal(str(pos.get("price_sl") or 0.0)),
                "price_tp": Decimal(str(pos.get("price_tp") or 0.0)),
                "activation_mode": pos.get("activation_mode", 0),
                "profit": Decimal(str(pos["profit"])),
                "margin": Decimal(str(pos["margin"])),
                "storage": Decimal(str(pos["storage"]))
            }

            if login not in self._login_tickets:
                self._login_tickets[login] = set()
            self._login_tickets[login].add(ticket)

        logger.info(f"PositionKeeper state restored: loaded {len(self._positions)} positions into memory."
```

■ main_server\core\tick_center.py

```
import logging
import asyncio
from decimal import Decimal
from typing import Dict, Any, List
from bases.connection import DBConnection
from shared.schemas.v1 import (
    ORDER_REASON_SL,
    ORDER_REASON_TP,
    ORDER_REASON_SO,
    ACTIVATION_NONE,
    ACTIVATION_SL,
    ACTIVATION_TP,
    ACTIVATION_PENDING,
    ACTIVATION_STOPOUT,
    OP_BUY,
    OP_SELL
)

logger = logging.getLogger(__name__)

class TickCenter:
    def __init__(self, trade_center, position_keeper, rms_engine, matching_engine, event_bus, tick_publish
        self.trade_center = trade_center
        self.position_keeper = position_keeper
        self.rms = rms_engine
        self.matching = matching_engine
        self.event_bus = event_bus
        self.tick_publisher = tick_publisher

    async def on_tick(self, symbol: str, bid: float, ask: float) -&gt; None:
        """
        Processes incoming quote ticks for a symbol in-process.
        Per MetaTrader5SDK/Server-API/Request-Processing-on-the-Server.md section 'Processing Ticks and Or
        """
        bid_dec = Decimal(str(bid))
        ask_dec = Decimal(str(ask))

        # Update Top of Book quote in matching engine
        self.matching.set_quote(symbol, bid, ask)

        # Publish tick event to event bus
        await self.event_bus.publish("ticks", {"symbol": symbol, "bid": float(bid_dec), "ask": float(ask_d

        # Forward to History Server via non-blocking queue
        if self.tick_publisher:
            self.tick_publisher.publish_tick(symbol, bid, ask)

        pool = DBConnection.get_pool()
        tasks_to_run = []

        async with pool.acquire() as conn:
            # 1. Update PriceCurrent and Profit of positions
            # per Processing Ticks and Orders step 2
            positions = await self.position_keeper.get_all_positions()
            symbol_positions = [p for p in positions if p["symbol"] == symbol]

            impacted_logins = set()

            for pos in symbol_positions:
                login = pos["login"]
                ticket = pos["ticket"]
                action = pos["action"] # 0: POSITION_BUY, 1: POSITION_SELL
                volume = Decimal(str(pos["volume"]))
                price_sl = Decimal(str(pos["price_sl"]))
                price_tp = Decimal(str(pos["price_tp"]))
```

```
activation = pos["activation_mode"]

# Current price: Buy evaluates at Bid, Sell evaluates at Ask
price_current = bid_dec if action == 0 else ask_dec

# Fetch symbol contract size
sym_row = await conn.fetchrow("SELECT contract_size FROM symbols WHERE symbol = $1", symbo
contract_size = Decimal(str(sym_row["contract_size"])) if sym_row else Decimal("100000.0")

# Profit calculation:
# Buy profit = (price_current - price_open) * contract_size * volume
# Sell profit = (price_open - price_current) * contract_size * volume
price_open = Decimal(str(pos["price_open"]))
if action == 0:
    profit = (price_current - price_open) * contract_size * volume
else:
    profit = (price_open - price_current) * contract_size * volume

# Update database
await conn.execute(
    "UPDATE positions SET price_current = $1, profit = $2, time_update = datetime('now') W
    price_current, profit, ticket
)
# Update memory
self.position_keeper._positions[ticket]["price_current"] = price_current
self.position_keeper._positions[ticket]["profit"] = profit

# Publish position update to event bus
await self.event_bus.publish("positions.updated", {
    "login": login,
    "ticket": ticket,
    "price_current": float(price_current),
    "profit": float(profit)
})

impacted_logins.add(login)

# 2. Check Stop Loss trigger (Processing Ticks and Orders step 3, 4, 5)
if price_sl &gt; 0:
    sl_hit = False
    if action == 0 and price_current &lt;= price_sl: # BUY position: Bid falls below SL
        sl_hit = True
    elif action == 1 and price_current &gt;= price_sl: # SELL position: Ask rises above SL
        sl_hit = True

    if sl_hit and activation == ACTIVATION_NONE:
        # Set activation state to ACTIVATION_SL BEFORE dispatching order to prevent duplic
        await self.position_keeper.update_position_activation(ticket, ACTIVATION_SL, conn=

        # Trigger position close request using the same place_order serialization lock
        close_type = 1 if action == 0 else 0  # opposite action closes position
        close_req = {
            "login": login,
            "symbol": symbol,
            "volume": volume,
            "price_request": price_current,
            "type": close_type,
            "reason": ORDER_REASON_SL
        }
        logger.warning(f"Stop Loss triggered for position {ticket} (login {login}). Sendin
        tasks_to_run.append(("close", close_req))

# 3. Check Take Profit trigger (Processing Ticks and Orders step 6, 7, 8)
if price_tp &gt; 0:
    tp_hit = False
    if action == 0 and price_current &gt;= price_tp: # BUY position: Bid rises above TP
        tp_hit = True
    elif action == 1 and price_current &lt;= price_tp: # SELL position: Ask falls below TP
```

```
                tp_hit = True

            if tp_hit and activation == ACTIVATION_NONE:
                # Set activation state to ACTIVATION_TP BEFORE dispatching close order
                await self.position_keeper.update_position_activation(ticket, ACTIVATION_TP, conn=

                close_type = 1 if action == 0 else 0
                close_req = {
                    "login": login,
                    "symbol": symbol,
                    "volume": volume,
                    "price_request": price_current,
                    "type": close_type,
                    "reason": ORDER_REASON_TP
                }
                logger.warning(f"Take Profit triggered for position {ticket} (login {login}). Send
                tasks_to_run.append(("close", close_req))

    # 4. Check pending orders trigger (Processing Ticks and Orders step 9, 10, 11)
    pending_rows = await conn.fetch(
        "SELECT ticket, login, price_order, type, activation_mode FROM orders WHERE symbol = $1 AN
        symbol
    )
    for row in pending_rows:
        ticket = row["ticket"]
        login = row["login"]
        price_order = Decimal(str(row["price_order"]))
        order_type = row["type"]
        ord_activation = row["activation_mode"]

        triggered = False
        if order_type == 2: # BUY_LIMIT: Triggered when Ask &lt;= price_order
            if ask_dec &lt;= price_order:
                triggered = True
        elif order_type == 3: # SELL_LIMIT: Triggered when Bid &gt;= price_order
            if bid_dec &gt;= price_order:
                triggered = True
        elif order_type == 4: # BUY_STOP: Triggered when Ask &gt;= price_order
            if ask_dec &gt;= price_order:
                triggered = True
        elif order_type == 5: # SELL_STOP: Triggered when Bid &lt;= price_order
            if bid_dec &lt;= price_order:
                triggered = True

        if triggered and ord_activation == ACTIVATION_NONE:
            # Update order activation to ACTIVATION_PENDING before dispatching
            await conn.execute(
                "UPDATE orders SET activation_mode = $1 WHERE ticket = $2",
                ACTIVATION_PENDING, ticket
            )

            logger.warning(f"Pending order {ticket} triggered for login {login}. Activating.")
            activation_price = ask_dec if order_type in (2, 4) else bid_dec
            tasks_to_run.append(("activate", {"ticket": ticket, "price": float(activation_price)})
            impacted_logins.add(login)

    # 5. Recalculate margins for accounts with position/order updates
    # per Processing Ticks and Orders step 12
    if impacted_logins:
        await self.rms.recalculate_accounts(list(impacted_logins), conn=conn)

    # 6. Stop Out check (Processing Ticks and Orders step 13)
    stop_outs = await self.rms.check_stop_outs(conn=conn)
    for login in stop_outs:
        acc_row = await conn.fetchrow("SELECT so_activation FROM accounts WHERE login = $1", login
        if acc_row and acc_row["so_activation"] == ACTIVATION_NONE:
            # Set account activation to stop out
            await self.rms.update_account_so_activation(login, ACTIVATION_STOPOUT, conn=conn)
```

```
                # Search for pending order with largest margin requirement
                pending_orders = await conn.fetch(
                    "SELECT ticket, volume, type FROM orders WHERE login = $1 AND state = 0 AND type I
                    login
                )
                largest_ord_ticket = None
                largest_ord_margin = Decimal("-1.0")
                for po in pending_orders:
                    po_ticket = po["ticket"]
                    po_volume = Decimal(str(po["volume"]))
                    po_details = await conn.fetchrow(
                        """SELECT o.price_order, s.contract_size, s.margin_initial
                           FROM orders o
                           JOIN symbols s ON o.symbol = s.symbol
                           WHERE o.ticket = $1""",
                        po_ticket
                    )
                    if po_details:
                        po_margin = (po_volume * Decimal(str(po_details["contract_size"])) * Decimal(s
                        if po_margin &gt; largest_ord_margin:
                            largest_ord_margin = po_margin
                            largest_ord_ticket = po_ticket

                if largest_ord_ticket:
                    logger.warning(f"Stop Out: cancelling pending order {largest_ord_ticket} for login
                    # Cancel order
                    await conn.execute("UPDATE orders SET activation_mode = $1 WHERE ticket = $2", ACT
                    tasks_to_run.append(("cancel", {"ticket": largest_ord_ticket, "login": login}))
                else:
                    # Search for position with largest loss
                    user_positions = await self.position_keeper.get_positions(login)
                    largest_loss_pos = None
                    largest_loss = Decimal("1.0") # We want largest negative profit (loss)

                    for up in user_positions:
                        up_profit = Decimal(str(up["profit"]))
                        if up_profit &lt; largest_loss:
                            largest_loss = up_profit
                            largest_loss_pos = up

                    if largest_loss_pos:
                        logger.warning(f"Stop Out: liquidating position {largest_loss_pos['ticket']} f
                        # Set position activation mode to ACTIVATION_STOPOUT
                        await self.position_keeper.update_position_activation(largest_loss_pos["ticket

                        # Trigger close order
                        close_type = 1 if largest_loss_pos["action"] == 0 else 0
                        close_req = {
                            "login": login,
                            "symbol": largest_loss_pos["symbol"],
                            "volume": Decimal(str(largest_loss_pos["volume"])),
                            "price_request": Decimal(str(largest_loss_pos["price_current"])),
                            "type": close_type,
                            "reason": ORDER_REASON_SO
                        }
                        tasks_to_run.append(("close", close_req))

        # Execute collected tasks sequentially outside the main on_tick transaction block
        for task_type, payload in tasks_to_run:
            try:
                if task_type == "close":
                    await self.trade_center.place_order(payload)
                elif task_type == "activate":
                    await self.trade_center.activate_pending_order(payload["ticket"], payload["price"])
                elif task_type == "cancel":
                    await self.trade_center.cancel_order(payload["ticket"], payload["login"])
            except Exception as e:
                logger.error(f"Error executing tick task {task_type} with payload {payload}: {e}")
```

■ main_server\core\tick_publisher.py

```
import os
import json
import logging
import asyncio
import secrets
import websockets
import time
from datetime import datetime, timezone
from typing import Dict, Any

logger = logging.getLogger(__name__)

INTERNAL_SECRET = os.getenv("INTERNAL_SECRET", "default_internal_secret_token_change_in_production")
HISTORY_SERVER_WS_URL = os.getenv("HISTORY_SERVER_WS_URL", "ws://localhost:8002/internal/ticks/ingest")

class TickPublisher:
    def __init__(self, max_size: int = 10000):
        self.queue = asyncio.Queue(maxsize=max_size)
        self.max_size = max_size
        self.running = False
        self.consumer_task = None
        self.last_warning_time = 0.0

    def start(self):
        """Starts the background worker task to drain the queue and push to history_server."""
        self.running = True
        self.consumer_task = asyncio.create_task(self._worker())
        logger.info("TickPublisher background worker started.")

    async def stop(self):
        """Stops the background worker gracefully."""
        self.running = False
        if self.consumer_task:
            self.consumer_task.cancel()
            try:
                await self.consumer_task
            except asyncio.CancelledError:
                pass
            self.consumer_task = None
        logger.info("TickPublisher background worker stopped.")

    def publish_tick(self, symbol: str, bid: float, ask: float):
        """
        Pushes a tick to the internal queue.
        This is completely non-blocking (fire-and-forget) from the caller's thread context.
        Uses drop-oldest policy if the buffer fills up.
        """
        tick = {
            "symbol": symbol,
            "bid": float(bid),
            "ask": float(ask),
            "time": datetime.now(timezone.utc).isoformat(),
            "source": "INTERNAL"
        }
        try:
            if self.queue.full():
                try:
                    discarded = self.queue.get_nowait()
                    # Log a loud warning to alert operators, throttled to once per 5 seconds
                    now = time.time()
                    if now - self.last_warning_time &gt; 5.0:
                        logger.warning(
                            f"■■ WARNING: TickPublisher queue overflow! Bounded size ({self.max_size}) ex
                            f"Dropping oldest ticks to prevent memory bloat."
                        )
```

```
                    self.last_warning_time = now
            except asyncio.QueueEmpty:
                pass
        self.queue.put_nowait(tick)
    except Exception as e:
        logger.error(f"Error publishing tick to background queue: {e}")

async def _worker(self):
    """Persistent worker loop that manages the WebSocket client connection and drains the queue."""
    backoff = 1.0
    while self.running:
        try:
            headers = {"X-Internal-Secret": INTERNAL_SECRET}
            logger.info(f"Attempting connection to History Server at {HISTORY_SERVER_WS_URL}...")

            async with websockets.connect(HISTORY_SERVER_WS_URL, extra_headers=headers) as ws:
                logger.info("Successfully connected to History Server tick ingest WS.")
                # Reset backoff upon successful connection
                backoff = 1.0

                while self.running:
                    # Wait for the next tick to publish
                    tick = await self.queue.get()
                    try:
                        await ws.send(json.dumps(tick))
                        self.queue.task_done()
                    except Exception as send_err:
                        # Re-enqueue the tick so we don't lose it if we disconnected,
                        # but make sure not to block if the queue filled up
                        logger.warning(f"Failed to send tick: {send_err}. Re-enqueuing.")
                        self.queue.task_done()

                        # Put back into queue if possible, dropping oldest if full
                        if self.queue.full():
                            try:
                                self.queue.get_nowait()
                            except asyncio.QueueEmpty:
                                pass
                        self.queue.put_nowait(tick)
                        raise send_err

        except Exception as e:
            if not self.running:
                break
            logger.warning(
                f"Connection to History Server failed or dropped: {e}. "
                f"Retrying in {backoff:.1f} seconds."
            )
            await asyncio.sleep(backoff)
            # Exponential backoff up to 30 seconds
            backoff = min(backoff * 2.0, 30.0)
```

■ main_server\core\trade_center.py

```
import logging
import asyncio
import os
from datetime import datetime
from typing import Dict, Any
from decimal import Decimal
from core.interfaces.i_oms import IOMS
from core.interfaces.i_rms import IRMS
from core.interfaces.i_matching import IMatching
from core.interfaces.i_position_keeper import IPositionKeeper
from core.interfaces.i_event_bus import IEventBus
from core.deal_writer import DealWriter
from bases.connection import DBConnection
from shared.schemas.v1 import (
    ORDER_REASON_SL, ORDER_REASON_TP, ORDER_REASON_SO,
    ORDER_STATE_REJECTED, ORDER_STATE_PENDING_DEALER, ORDER_STATE_PENDING_GATEWAY
)
from routing.engine import route_order

logger = logging.getLogger(__name__)

class TradeCenter(IOMS):
    def __init__(
        self,
        rms: IRMS,
        matching: IMatching,
        position_keeper: IPositionKeeper,
        event_bus: IEventBus
    ):
        self.rms = rms
        self.matching = matching
        self.position_keeper = position_keeper
        self.event_bus = event_bus
        self._account_locks: Dict[int, asyncio.Lock] = {}

    async def cleanup_orders_on_startup(self) -&gt; None:
        """
        Runs on trade server boot. Cancels frozen market orders and resets activation attributes.
        Per MetaTrader5SDK/Server-API/Request-Processing-on-the-Server.md section 'Removing unprocessed or
        """
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            # 1. Cancel frozen market orders (OP_BUY=0, OP_SELL=1)
            rows = await conn.fetch(
                "SELECT ticket, login FROM orders WHERE state IN (0, 1) AND type IN (0, 1)"
            )
            for row in rows:
                ticket = row["ticket"]
                login = row["login"]
                logger.warning(f"unfilled order {ticket} (account {login}) canceled")

                await conn.execute(
                    "UPDATE orders SET state = 5, time_done = datetime('now') WHERE ticket = $1",
                    ticket
                )

            # 2. Reset activation mode on all pending orders inSTARTED state
            await conn.execute(
                "UPDATE orders SET activation_mode = 0 WHERE state = 0 AND type NOT IN (0, 1)"
            )

            # 3. Reset activation mode on all positions
            await conn.execute(
                "UPDATE positions SET activation_mode = 0"
            )
```

```
        # 4. Reset stop out activation mode on all accounts
        await conn.execute(
            "UPDATE accounts SET so_activation = 0"
        )
    logger.info("Startup order and activation attribute cleanup completed successfully.")

async def place_order(self, order_request: Dict[str, Any]) -&gt; Dict[str, Any]:
    """
    Processes and executes a client trade order (market or pending).
    """
    login = order_request["login"]
    symbol = order_request["symbol"]
    volume = Decimal(str(order_request["volume"]))
    price_req = Decimal(str(order_request["price_request"]))
    order_type = order_request["type"] # 0: OP_BUY, 1: OP_SELL, 2: OP_BUY_LIMIT, etc.
    price_sl = Decimal(str(order_request.get("price_sl") or 0.0))
    price_tp = Decimal(str(order_request.get("price_tp") or 0.0))
    reason = order_request.get("reason", 0) # 0=CLIENT, 4=SL, 5=TP, 6=SO
    type_filling = order_request.get("type_filling", "FOK")

    is_pending = order_type in (2, 3, 4, 5) # OP_BUY_LIMIT=2, OP_SELL_LIMIT=3, OP_BUY_STOP=4, OP_SELL_

    # Acquire lock per account to serialize execution
    # per Request-Processing-on-the-Server.md step 4/5: MT5 serializes request verification and execut
    lock = self._account_locks.setdefault(login, asyncio.Lock())
    async with lock:
        # Check quote age for market orders
        if not is_pending:
            quote_age = self.matching.get_quote_age(symbol)
            if quote_age != float('inf') and quote_age &gt; 10.0:
                raise ValueError(f"STALE_QUOTE: Quote for symbol {symbol} is stale ({quote_age:.1f}s o

        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            # 1. Create Order row with state = STARTED (0) outside the execution transaction
            ticket = await conn.fetchval(
                """INSERT INTO orders (login, symbol, volume, volume_current, price_order, price_sl, p
                   VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10) RETURNING ticket""",
                login, symbol, volume, volume, price_req, price_sl, price_tp, order_type, 0, reason
            )

            try:
                # Retrieve group name for routing
                acc = await conn.fetchrow("SELECT group_name FROM accounts WHERE login = $1", login)
                group_name = acc["group_name"] if acc else "demo_group"

                # Validate requested symbol against group permitted symbol patterns
                group_row = await conn.fetchrow("SELECT settings_json FROM groups WHERE name = $1", gr
                if group_row and group_row["settings_json"]:
                    import json
                    import fnmatch
                    try:
                        settings = json.loads(group_row["settings_json"])
                        symbol_rules = settings.get("symbol_rules", [])
                        if symbol_rules:
                            matching_rules = []
                            for rule in symbol_rules:
                                pat = rule.get("symbol", "")
                                clean_sym = symbol.replace("\\", "/").lower()
                                clean_pat = pat.replace("\\", "/").lower()
                                if fnmatch.fnmatchcase(clean_sym, clean_pat):
                                    matching_rules.append(rule)

                            if matching_rules:
                                matching_rules.sort(key=lambda r: len(r.get("symbol", "")), reverse=Tr
                                allowed = matching_rules[0].get("trade_allowed", True)
                                if not allowed:
                                    raise ValueError(f"SYMBOL_BLOCKED: Symbol {symbol} is not allowed
```

```
                else:
                    raise ValueError(f"SYMBOL_BLOCKED: Symbol {symbol} is not allowed for
        except ValueError:
            raise
        except Exception as e:
            logger.error(f"Error validating group symbol rules: {e}")

    # Evaluate routing rules
    route_res = await route_order(order_request, group_name, conn)
    action = route_res["action"]

    if action == "REJECT":
        raise ValueError(f"Order rejected by routing rule: {route_res['rule_name']}")

    if action == "TO_DEALER":
        await conn.execute(
            "UPDATE orders SET state = $1, assigned_dealer = $2 WHERE ticket = $3",
            ORDER_STATE_PENDING_DEALER, route_res.get("gateway_id"), ticket
        )
        return {
            "ticket": ticket,
            "login": login,
            "symbol": symbol,
            "volume": volume,
            "price_execution": Decimal("0.0"),
            "profit": Decimal("0.0"),
            "action": order_type,
            "entry": 0,
            "status": "PENDING_DEALER",
            "message": f"Order queued for dealer approval (Rule: {route_res['rule_name']})
        }

    if action == "TO_GATEWAY" and os.getenv("REAL_LP", "true").lower() != "true":
        await conn.execute(
            "UPDATE orders SET state = $1 WHERE ticket = $2",
            ORDER_STATE_PENDING_GATEWAY, ticket
        )
        return {
            "ticket": ticket,
            "login": login,
            "symbol": symbol,
            "volume": volume,
            "price_execution": Decimal("0.0"),
            "profit": Decimal("0.0"),
            "action": order_type,
            "entry": 0,
            "status": "PENDING_GATEWAY",
            "message": f"Gateway routing is not yet available (Rule: {route_res['rule_name
        }

    is_abook = (action == "TO_GATEWAY" and os.getenv("REAL_LP", "true").lower() == "true")
    exec_price_override = None

    if is_abook:
        # Update order state to pending gateway before calling LP
        await conn.execute(
            "UPDATE orders SET state = $1 WHERE ticket = $2",
            ORDER_STATE_PENDING_GATEWAY, ticket
        )

        # Query LP gateway details
        gateway_id = route_res.get("gateway_id")
        gw_host = "localhost"
        gw_port = 8003
        gw_username = ""
        gw_api_key = ""
        gw_server_ip = "86.104.251.194:443"
```

```
if gateway_id:
    gw_row = await conn.fetchrow(
        "SELECT host, port, username, api_key, settings_json FROM gateways WHERE i
        gateway_id
    )
    if gw_row:
        gw_host = gw_row["host"] or "localhost"
        gw_port = gw_row["port"] or 8003
        gw_username = gw_row["username"] or ""
        gw_api_key = gw_row["api_key"] or ""

        # Parse settings_json to get gateway_server
        try:
            settings = json.loads(gw_row["settings_json"] or "{}")
            gw_server_ip = settings.get("gateway", {}).get("gateway_server") or "8
        except Exception:
            pass

# Query LP gateway via history_server
try:
    import httpx
    import secrets

    hist_server_url = os.getenv("HISTORY_SERVER_URL", "http://localhost:8002")
    internal_secret = os.getenv("INTERNAL_SECRET", "default_internal_secret_token_

    async with httpx.AsyncClient() as http_client:
        lp_resp = await http_client.post(
            f"{hist_server_url}/internal/gateway/execute",
            json={
                "ticket": ticket,
                "symbol": symbol,
                "volume": float(volume),
                "action": order_type,
                "price": float(price_req) if is_pending else None,
                "type_filling": type_filling,
                "gateway_url": f"http://{gw_host}:{gw_port}",
                "gateway_login": gw_username,
                "gateway_password": gw_api_key,
                "gateway_server": gw_server_ip
            },
            headers={"X-Internal-Secret": internal_secret},
            timeout=20.0
        )

        if lp_resp.status_code == 200:
            lp_data = lp_resp.json()
            if lp_data.get("status") == "FILLED":
                exec_price_override = Decimal(str(lp_data["price_execution"]))
            else:
                err_msg = lp_data.get("reason", "Gateway execution rejected")
                await conn.execute(
                    "UPDATE orders SET state = $1, reason = $2 WHERE ticket = $3",
                    ORDER_STATE_REJECTED, f"LP Rejected: {err_msg}", ticket
                )
                raise ValueError(f"Gateway Order Rejected: {err_msg}")
        else:
            err_msg = f"Gateway HTTP error status {lp_resp.status_code}"
            await conn.execute(
                "UPDATE orders SET state = $1, reason = $2 WHERE ticket = $3",
                ORDER_STATE_REJECTED, err_msg, ticket
            )
            raise ValueError(f"Gateway Communication Error: {err_msg}")
except Exception as e:
    err_msg = f"LP connection error: {e}"
    logger.error(f"A-Book Execution failed for ticket {ticket}: {err_msg}")
    await conn.execute(
        "UPDATE orders SET state = $1, reason = $2 WHERE ticket = $3",
```

```
                    ORDER_STATE_REJECTED, err_msg, ticket
                )
                if isinstance(e, ValueError):
                    raise e
                raise ValueError(f"LP connection error: {e}")

        # Start the single atomic transaction block
        async with conn.transaction():
            if is_pending:
                # For pending orders, we just verify they are queued and return
                # state = 0 (STARTED) is already set
                pass
            else:
                # Market order execution path
                # 2. Check open positions for close/netting detection
                positions = await self.position_keeper.get_positions(login)
                opp_pos = None
                for p in positions:
                    if p["symbol"] == symbol:
                        if (order_type == 0 and p["action"] == 1) or (order_type == 1 and p["a
                            opp_pos = p
                            break

                is_close = opp_pos is not None

                # Guard: If a specific position ticket is targeted, verify it exists and is op
                position_ticket = order_request.get("position")
                if position_ticket is not None:
                    pos = self.position_keeper._positions.get(int(position_ticket))
                    if not pos or pos["login"] != login:
                        raise ValueError("Position not found or already closed.")

                # Guard: If close request is triggered by SL/TP/Stop Out, ensure we are actual
                if reason in (ORDER_REASON_SL, ORDER_REASON_TP, ORDER_REASON_SO):
                    if not is_close:
                        raise ValueError("Position already closed or not found.")

                if not is_close:
                    # Pass active conn to pre_trade_check to reuse transaction connection
                    allowed = await self.rms.pre_trade_check(login, symbol, volume, price_req,
                    if not allowed:
                        raise ValueError("Insufficient free margin.")

                # 3. Match Order against TOB quotes (B-Book) or use LP execution price (A-Book
                if is_abook:
                    price_exec = exec_price_override
                else:
                    match_res = await self.matching.match_order(symbol, volume, price_req, ord
                    price_exec = match_res["price_execution"]
                deal_entry = 0 # default ENTRY_IN (0)

                # Fetch symbol parameters (using transaction conn)
                sym = await conn.fetchrow("SELECT contract_size FROM symbols WHERE symbol = $1
                contract_size = Decimal(str(sym["contract_size"])) if sym else Decimal("100000

                profit = Decimal("0.0")
                if is_close:
                    deal_entry = 1 # ENTRY_OUT (1)
                    price_open = Decimal(str(opp_pos["price_open"]))
                    if opp_pos["action"] == 0: # Closing a BUY position (with a SELL deal)
                        profit = (price_exec - price_open) * contract_size * volume
                    else: # Closing a SELL position (with a BUY deal)
                        profit = (price_open - price_exec) * contract_size * volume

                # 4. Write Deal Log (using conn)
                deal_action = order_type # DEAL_BUY = 0, DEAL_SELL = 1
                deal_ticket = await DealWriter.write_deal(
                    order_ticket=ticket,
```

```
                    login=login,
                    symbol=symbol,
                    volume=volume,
                    price=price_exec,
                    action=deal_action,
                    entry=deal_entry,
                    profit=profit,
                    commission=Decimal("0.0"),
                    storage=Decimal("0.0"),
                    reason=reason,
                    conn=conn
                )

                # 5. Update Position in memory and DB (using conn)
                await self.position_keeper.update_position(
                    login=login,
                    symbol=symbol,
                    volume=volume,
                    price=price_exec,
                    action=order_type,
                    deal_entry=deal_entry,
                    price_sl=float(price_sl),
                    price_tp=float(price_tp),
                    conn=conn
                )

                # 6. Recalculate account risk parameters (using conn)
                await self.rms.recalculate_accounts([login], conn=conn)

                # 7. Complete Order status to FILLED (4)
                await conn.execute(
                    "UPDATE orders SET state = 4, volume_current = 0, time_done = datetime('no
                    ticket
                )

            # 8. Publish event to EventBus (done only after successful commit)
            if not is_pending:
                event_payload = {
                    "order_ticket": ticket,
                    "deal_ticket": deal_ticket,
                    "login": login,
                    "symbol": symbol,
                    "volume": str(volume),
                    "price": str(price_exec),
                    "profit": str(profit),
                    "timestamp": datetime.utcnow().isoformat()
                }
                await self.event_bus.publish("orders.filled", event_payload)

            return {
                "ticket": ticket,
                "login": login,
                "symbol": symbol,
                "volume": volume,
                "price_execution": Decimal("0.0") if is_pending else price_exec,
                "profit": Decimal("0.0") if is_pending else profit,
                "action": order_type,
                "entry": 0 if is_pending else deal_entry,
                "status": "STARTED" if is_pending else "FILLED",
                "message": "Pending order placed successfully." if is_pending else "Order executed
            }

        except Exception as e:
            logger.error(f"Error executing order {ticket}: {e}")
            # Transaction rolls back automatically on exception block exit.
            # We write the REJECTED state on a new/existing connection outside the transaction blo
            try:
                await conn.execute(
```

```
                        "UPDATE orders SET state = 5, time_done = datetime('now') WHERE ticket = $1",
                        ticket
                    )
                except Exception as db_err:
                    logger.error(f"Failed to mark order as rejected: {db_err}")

                # Wipe any half-mutated memory state and reload from the database
                await self.position_keeper.restore_state()

                return {
                    "ticket": ticket,
                    "login": login,
                    "symbol": symbol,
                    "volume": volume,
                    "price_execution": Decimal("0.0"),
                    "profit": Decimal("0.0"),
                    "action": order_type,
                    "entry": 0,
                    "status": "REJECTED",
                    "message": str(e)
                }

async def activate_pending_order(self, ticket: int, activation_price: float) -&gt; Dict[str, Any]:
    """
    Activates a pending order: matches it, writes a deal log, updates position, and updates order stat
    Runs under the lock of the account.
    Per Request-Processing-on-the-Server.md step 10/11.
    """
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        # 1. Fetch order details
        order = await conn.fetchrow(
            "SELECT login, symbol, volume, type, price_sl, price_tp FROM orders WHERE ticket = $1",
            ticket
        )
        if not order:
            raise ValueError("Order not found")

        login = order["login"]
        symbol = order["symbol"]
        volume = Decimal(str(order["volume"]))
        order_type = order["type"]
        price_sl = Decimal(str(order["price_sl"] or 0.0))
        price_tp = Decimal(str(order["price_tp"] or 0.0))

        # Map pending order type to market execution type
        # OP_BUY_LIMIT (2) / OP_BUY_STOP (4) -&gt; market BUY (0)
        # OP_SELL_LIMIT (3) / OP_SELL_STOP (5) -&gt; market SELL (1)
        exec_action = 0 if order_type in (2, 4) else 1

        lock = self._account_locks.setdefault(login, asyncio.Lock())
        async with lock:
            try:
                async with conn.transaction():
                    # Check open positions for close/netting
                    positions = await self.position_keeper.get_positions(login)
                    opp_pos = None
                    for p in positions:
                        if p["symbol"] == symbol:
                            if (exec_action == 0 and p["action"] == 1) or (exec_action == 1 and p["act
                                opp_pos = p
                                break

                    is_close = opp_pos is not None
                    if not is_close:
                        # Verify margin before activation
                        allowed = await self.rms.pre_trade_check(login, symbol, volume, activation_pri
                        if not allowed:
```

```
                raise ValueError("Insufficient free margin.")

            # Use activation price as execution price
            price_exec = Decimal(str(activation_price))
            deal_entry = 0

            sym = await conn.fetchrow("SELECT contract_size FROM symbols WHERE symbol = $1", s
            contract_size = Decimal(str(sym["contract_size"])) if sym else Decimal("100000.0")

            profit = Decimal("0.0")
            if is_close:
                deal_entry = 1
                price_open = Decimal(str(opp_pos["price_open"]))
                if opp_pos["action"] == 0:
                    profit = (price_exec - price_open) * contract_size * volume
                else:
                    profit = (price_open - price_exec) * contract_size * volume

            # Write deal log
            deal_ticket = await DealWriter.write_deal(
                order_ticket=ticket,
                login=login,
                symbol=symbol,
                volume=volume,
                price=price_exec,
                action=exec_action,
                entry=deal_entry,
                profit=profit,
                reason=0,  # ORDER_REASON_CLIENT
                conn=conn
            )

            # Update position
            await self.position_keeper.update_position(
                login=login,
                symbol=symbol,
                volume=volume,
                price=price_exec,
                action=exec_action,
                deal_entry=deal_entry,
                price_sl=float(price_sl),
                price_tp=float(price_tp),
                conn=conn
            )

            # Recalculate margins
            await self.rms.recalculate_accounts([login], conn=conn)

            # Update order state to FILLED (4)
            await conn.execute(
                "UPDATE orders SET state = 4, volume_current = 0, time_done = datetime('now')
                ticket
            )

        # Publish event
        event_payload = {
            "order_ticket": ticket,
            "deal_ticket": deal_ticket,
            "login": login,
            "symbol": symbol,
            "volume": str(volume),
            "price": str(price_exec),
            "profit": str(profit),
            "timestamp": datetime.utcnow().isoformat()
        }
        await self.event_bus.publish("orders.filled", event_payload)

        return {"ticket": ticket, "status": "FILLED", "price_execution": price_exec, "profit":
```

```
            except Exception as e:
                logger.error(f"Error activating pending order {ticket}: {e}")
                # Mark order rejected on exception
                try:
                    await conn.execute(
                        "UPDATE orders SET state = 5, time_done = datetime('now') WHERE ticket = $1",
                        ticket
                    )
                except Exception as db_err:
                    logger.error(f"Failed to reject pending order {ticket}: {db_err}")

                await self.position_keeper.restore_state()
                return {"ticket": ticket, "status": "REJECTED", "message": str(e)}

async def cancel_order(self, ticket: int, login: int) -&gt; Dict[str, Any]:
    """
    Cancels a pending order.
    """
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        lock = self._account_locks.setdefault(login, asyncio.Lock())
        async with lock:
            await conn.execute(
                "UPDATE orders SET state = 5, time_done = datetime('now') WHERE ticket = $1 AND login
                ticket, login
            )
    return {"ticket": ticket, "status": "CANCELLED"}

async def get_order(self, ticket: int) -&gt; Dict[str, Any]:
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        row = await conn.fetchrow("SELECT * FROM orders WHERE ticket = $1", ticket)
        return dict(row) if row else {}
```

■ main_server\core\interfaces\i_event_bus.py

```
from abc import ABC, abstractmethod
from typing import Callable, Any, Dict

class IEventBus(ABC):
    @abstractmethod
    async def publish(self, topic: str, payload: Dict[str, Any]) -&gt; None:
        """
        Publishes an event to a topic.
        """
        pass

    @abstractmethod
    async def subscribe(self, topic: str, callback: Callable[[Dict[str, Any]], Any]) -&gt; None:
        """
        Subscribes to a topic with a callback function.
        """
        pass
```

■ main_server\core\interfaces\i_matching.py

```
from abc import ABC, abstractmethod
from typing import Dict, Any

class IMatching(ABC):
    @abstractmethod
    async def match_order(
        self, symbol: str, volume: float, price_request: float, order_type: int
    ) -&gt; Dict[str, Any]:
        """
        Executes order matching logic.
        Returns execution price, execution volume, entry direction, and gateway ID.
        """
        pass
```

■ main_server\core\interfaces\i_oms.py

```
from abc import ABC, abstractmethod
from typing import Dict, Any

class IOMS(ABC):
    @abstractmethod
    async def place_order(self, order_request: Dict[str, Any]) -&gt; Dict[str, Any]:
        """
        Validates, checks margin (RMS), matches, writes deals, and updates positions.
        Returns the execution result or raises an exception.
        """
        pass

    @abstractmethod
    async def cancel_order(self, ticket: int, login: int) -&gt; Dict[str, Any]:
        """
        Cancels an active pending order.
        """
        pass

    @abstractmethod
    async def get_order(self, ticket: int) -&gt; Dict[str, Any]:
        """
        Retrieves active or history order details.
        """
        pass
```

■ main_server\core\interfaces\i_position_keeper.py

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any

class IPositionKeeper(ABC):
    @abstractmethod
    async def get_positions(self, login: int) -&gt; List[Dict[str, Any]]:
        """
        Retrieves active open positions for a specific client login.
        """
        pass

    @abstractmethod
    async def get_all_positions(self) -&gt; List[Dict[str, Any]]:
        """
        Retrieves all open positions currently in memory.
        """
        pass

    @abstractmethod
    async def update_position(
        self, login: int, symbol: str, volume: float, price: float, action: int, deal_entry: int
    ) -&gt; Dict[str, Any]:
        """
        Updates the net position state in memory and database based on a new deal.
        Handles: ENTRY_IN (new position/adding volume), ENTRY_OUT (partial/full close).
        """
        pass

    @abstractmethod
    async def restore_state(self) -&gt; None:
        """
        Loads all active positions from the database into memory during boot/restart.
        """
        pass
```

■ main_server\core\interfaces\i_rms.py

```
from abc import ABC, abstractmethod
from typing import List

class IRMS(ABC):
    @abstractmethod
    async def pre_trade_check(
        self, login: int, symbol: str, volume: float, price: float, action: int
    ) -&gt; bool:
        """
        Runs pre-trade risk checks:
        - Validates account status (not blocked)
        - Calculates required margin and compares against free margin
        Returns True if check passes, False or raises error if it fails.
        """
        pass

    @abstractmethod
    async def recalculate_accounts(self, logins: List[int]) -&gt; None:
        """
        Recalculates floating P&amp;L, equity, margin, free margin, and margin levels
        for the given logins based on current quotes.
        """
        pass

    @abstractmethod
    async def check_stop_outs(self) -&gt; List[int]:
        """
        Scans all active accounts to detect Stop Out breaches.
        Returns the list of logins that triggered Stop Out.
        """
        pass
```

■ main_server\feeds\mock_feed.py

```
import asyncio
import random
import logging
from decimal import Decimal

logger = logging.getLogger(__name__)

class MockTickFeed:
    def __init__(self, tick_center, symbols=None):
        self.tick_center = tick_center
        self.symbols = symbols or {
            "EURUSD": {"price": 1.1000, "spread": 0.0002, "step": 0.0001},
            "GBPUSD": {"price": 1.2500, "spread": 0.0003, "step": 0.00015},
            "XAUUSD": {"price": 2000.00, "spread": 0.50, "step": 0.20}
        }
        self.is_running = False
        self._task = None

    def start(self, interval_seconds: float = 0.5):
        self.is_running = True
        self._task = asyncio.create_task(self._run_loop(interval_seconds))
        logger.info("MockTickFeed started.")

    def stop(self):
        self.is_running = False
        if self._task:
            self._task.cancel()
        logger.info("MockTickFeed stopped.")

    async def _run_loop(self, interval: float):
        while self.is_running:
            try:
                # Pick a random symbol
                symbol = random.choice(list(self.symbols.keys()))
                config = self.symbols[symbol]

                # Perform a random walk step
                direction = random.choice([-1, 0, 1])
                config["price"] = max(0.0001, config["price"] + direction * config["step"])

                bid = round(config["price"], 5)
                ask = round(config["price"] + config["spread"], 5)

                # Directly call TickCenter in-process
                await self.tick_center.on_tick(symbol, bid, ask)
            except asyncio.CancelledError:
                break
            except Exception as e:
                logger.error(f"Error in MockTickFeed run loop: {e}")
            await asyncio.sleep(interval)
```

■ main_server\matching\engine.py

```
import logging
from typing import Dict, Any, Tuple
from decimal import Decimal
from core.interfaces.i_matching import IMatching

logger = logging.getLogger(__name__)

class MatchingEngine(IMatching):
    def __init__(self):
        # symbol -&gt; (bid, ask)
        self._quotes: Dict[str, Tuple[Decimal, Decimal]] = {}
        self._quote_times: Dict[str, float] = {}

    def set_quote(self, symbol: str, bid: float, ask: float):
        """Sets mock quotes for testing."""
        import time
        self._quotes[symbol] = (Decimal(str(bid)), Decimal(str(ask)))
        self._quote_times[symbol] = time.time()

    def get_quote_age(self, symbol: str) -&gt; float:
        """Returns the age of the last quote in seconds."""
        import time
        quote_time = self._quote_times.get(symbol)
        if quote_time is None:
            return float('inf')
        return time.time() - quote_time

    async def match_order(
        self, symbol: str, volume: float, price_request: float, order_type: int
    ) -&gt; Dict[str, Any]:
        """
        Matches market orders against current Top of Book (TOB) quotes.
        0: OP_BUY matches at Ask.
        1: OP_SELL matches at Bid.
        """
        vol_dec = Decimal(str(volume))
        req_price = Decimal(str(price_request))

        if symbol in self._quotes:
            bid, ask = self._quotes[symbol]
            # Match against quote
            exec_price = ask if order_type == 0 else bid
            logger.info(f"Matched {symbol} order type={order_type} against TOB quote: request={req_price},
        else:
            import os
            if os.getenv("ALLOW_UNQUOTED_FILLS") == "true":
                exec_price = req_price
                logger.warning(f"DEV FALLBACK: No active quote for {symbol}, matching at request price: {e
            else:
                logger.error(f"Order execution failed: No active quotes available for {symbol}")
                raise ValueError("NO_QUOTE_AVAILABLE")

        # EnDealEntry: ENTRY_IN is 0 (Entering market)
        # Entry/Action mapping
        return {
            "price_execution": exec_price,
            "volume_execution": vol_dec,
            "entry": 0,  # ENTRY_IN
            "gateway_id": "B-BOOK"
        }
```

■ main_server\rms\accounting.py

```
from typing import List, Dict, Any
from decimal import Decimal

def calculate_positions_math(positions: List[Dict[str, Any]], leverage: int) -&gt; Dict[str, Any]:
    """
    Computes P&amp;L and margins for a list of positions using exact Decimal calculations.
    leverage: account leverage (e.g. 100).
    Returns:
        {
            "position_updates": List of dicts containing calculated P&amp;L and margins,
            "total_profit": Decimal,
            "total_margin": Decimal
        }
    """
    if not positions:
        return {
            "position_updates": [],
            "total_profit": Decimal("0.0"),
            "total_margin": Decimal("0.0")
        }

    updates = []
    total_profit = Decimal("0.0")
    total_margin = Decimal("0.0")
    leverage_dec = Decimal(str(leverage))

    for pos in positions:
        action = pos["action"]  # 0: BUY, 1: SELL
        volume = Decimal(str(pos["volume"]))
        price_open = Decimal(str(pos["price_open"]))
        price_current = Decimal(str(pos["price_current"]))
        contract_size = Decimal(str(pos["contract_size"]))

        # Buy: profit = (price_current - price_open) * contract_size * volume
        # Sell: profit = (price_open - price_current) * contract_size * volume
        if action == 0:  # BUY
            profit = (price_current - price_open) * contract_size * volume
        else:            # SELL
            profit = (price_open - price_current) * contract_size * volume

        # Margin: (volume * contract_size * price_open * margin_initial * margin_rate) / leverage
        margin_initial = Decimal(str(pos.get("margin_initial", 1.0)))
        margin_rate = Decimal(str(pos.get("margin_rate", 1.0)))
        margin = (volume * contract_size * price_open * margin_initial * margin_rate) / leverage_dec

        total_profit += profit
        total_margin += margin

        updates.append({
            "ticket": pos["ticket"],
            "profit": profit,
            "margin": margin,
            "price_current": price_current
        })

    return {
        "position_updates": updates,
        "total_profit": total_profit,
        "total_margin": total_margin
    }
```

■ main_server\rms\engine.py

```
import logging
from typing import List, Dict, Any
from decimal import Decimal
from core.interfaces.i_rms import IRMS
from core.interfaces.i_position_keeper import IPositionKeeper
from rms.accounting import calculate_positions_math
from bases.connection import DBConnection

logger = logging.getLogger(__name__)

class RMSEngine(IRMS):
    def __init__(self, position_keeper: IPositionKeeper, matching_engine: Any = None):
        self.position_keeper = position_keeper
        self.matching_engine = matching_engine

    async def _get_symbol_contract_sizes(self, conn, symbols: List[str]) -&gt; Dict[str, Decimal]:
        """
        Batch-fetches symbol contract sizes in a single query to prevent N+1 query patterns.
        """
        if not symbols:
            return {}
        unique_symbols = list(set(symbols))
        placeholders = ", ".join(f"${i+1}" for i in range(len(unique_symbols)))
        query = f"SELECT symbol, contract_size FROM symbols WHERE symbol IN ({placeholders})"
        rows = await conn.fetch(query, *unique_symbols)
        return {r["symbol"]: Decimal(str(r["contract_size"])) for r in rows}

    async def pre_trade_check(
        self, login: int, symbol: str, volume: float, price: float, action: int, conn = None
    ) -&gt; bool:
        """
        Runs pre-trade risk validation.
        Check if account exists, is normal (not blocked), and has enough free margin.
        """
        vol_dec = Decimal(str(volume))
        price_dec = Decimal(str(price))

        if conn is not None:
            return await self._pre_trade_check_impl(conn, login, symbol, vol_dec, price_dec, action)
        else:
            pool = DBConnection.get_pool()
            async with pool.acquire() as conn:
                return await self._pre_trade_check_impl(conn, login, symbol, vol_dec, price_dec, action)

    async def _pre_trade_check_impl(
        self, conn, login: int, symbol: str, vol_dec: Decimal, price_dec: Decimal, action: int
    ) -&gt; bool:
        # 1. Fetch account
        acc = await conn.fetchrow(
            "SELECT balance, credit, leverage, status, group_name FROM accounts WHERE login = $1",
            login
        )
        if not acc:
            raise ValueError(f"Account {login} not found.")
        if acc["status"] != 0:
            raise ValueError(f"Account {login} is blocked or read-only.")

        # 2. Fetch symbol parameters
        sym = await conn.fetchrow(
            "SELECT contract_size, margin_initial FROM symbols WHERE symbol = $1",
            symbol
        )
        if not sym:
            raise ValueError(f"Symbol {symbol} not found.")
```

```
# 3. Calculate required margin for the new trade
leverage = acc["leverage"]
contract_size = Decimal(str(sym["contract_size"]))
margin_initial = Decimal(str(sym["margin_initial"]))

# Check group overrides for the new symbol
group_override = await conn.fetchrow(
    "SELECT margin_rate, trade_allowed FROM group_symbols WHERE group_name = $1 AND symbol = $2",
    acc["group_name"], symbol
)
if group_override:
    if not group_override["trade_allowed"]:
        raise ValueError(f"Trading is disabled for symbol {symbol} in group {acc['group_name']}.")
    margin_initial = margin_initial * Decimal(str(group_override["margin_rate"]))

new_margin = (vol_dec * contract_size * price_dec * margin_initial) / Decimal(str(leverage))

# 4. Fetch open positions and compute current equity &amp; margin
positions = await self.position_keeper.get_positions(login)

# Batch-fetch contract sizes and margin_initial for open positions
open_symbols = [p["symbol"] for p in positions]
symbol_params = {}
if open_symbols:
    placeholders = ", ".join(f"${i+1}" for i in range(len(open_symbols)))
    rows = await conn.fetch(
        f"SELECT symbol, contract_size, margin_initial FROM symbols WHERE symbol IN ({placeholders
        *open_symbols
    )
    symbol_params = {
        r["symbol"]: (Decimal(str(r["contract_size"])), Decimal(str(r["margin_initial"])))
        for r in rows
    }

# Batch-fetch overrides for open positions
overrides = {}
if open_symbols:
    placeholders = ", ".join(f"${i+2}" for i in range(len(open_symbols)))
    rows = await conn.fetch(
        f"SELECT symbol, margin_rate FROM group_symbols WHERE group_name = $1 AND symbol IN ({plac
        acc["group_name"], *open_symbols
    )
    overrides = {r["symbol"]: Decimal(str(r["margin_rate"])) for r in rows}

# Enrich positions with contract size, margin_initial, and overrides for P&amp;L math
enriched_positions = []
for p in positions:
    sym_p = symbol_params.get(p["symbol"], (Decimal("100000.0"), Decimal("1.0")))
    margin_rate = overrides.get(p["symbol"], Decimal("1.0"))
    enriched_positions.append({
        **p,
        "contract_size": sym_p[0],
        "margin_initial": sym_p[1],
        "margin_rate": margin_rate
    })

math_res = calculate_positions_math(enriched_positions, leverage)

balance = Decimal(str(acc["balance"]))
credit = Decimal(str(acc["credit"]))
total_profit = math_res["total_profit"]
total_margin = math_res["total_margin"]

current_equity = balance + credit + total_profit
post_trade_margin = total_margin + new_margin

# Free Margin must be &gt;= required new margin
free_margin = current_equity - post_trade_margin
```

```
    if free_margin &lt; 0:
        logger.warning(f"Pre-trade check failed for login={login}: Insufficient free margin ({free_mar
        return False

    return True

async def recalculate_accounts(self, logins: List[int], conn = None) -&gt; None:
    """
    Recalculates equity, margin, free margin, and margin levels for clients.
    """
    if conn is not None:
        await self._recalculate_accounts_impl(conn, logins)
    else:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await self._recalculate_accounts_impl(conn, logins)

async def _recalculate_accounts_impl(self, conn, logins: List[int]) -&gt; None:
    for login in logins:
        acc = await conn.fetchrow(
            "SELECT balance, credit, leverage, group_name FROM accounts WHERE login = $1",
            login
        )
        if not acc:
            continue

        # Get open positions
        positions = await self.position_keeper.get_positions(login)

        # Batch-fetch contract sizes and margin_initial for open positions
        open_symbols = [p["symbol"] for p in positions]
        symbol_params = {}
        if open_symbols:
            placeholders = ", ".join(f"${i+1}" for i in range(len(open_symbols)))
            rows = await conn.fetch(
                f"SELECT symbol, contract_size, margin_initial FROM symbols WHERE symbol IN ({placehol
                *open_symbols
            )
            symbol_params = {
                r["symbol"]: (Decimal(str(r["contract_size"])), Decimal(str(r["margin_initial"])))
                for r in rows
            }

        # Batch-fetch overrides for open positions
        overrides = {}
        if open_symbols:
            placeholders = ", ".join(f"${i+2}" for i in range(len(open_symbols)))
            rows = await conn.fetch(
                f"SELECT symbol, margin_rate FROM group_symbols WHERE group_name = $1 AND symbol IN ({
                acc["group_name"], *open_symbols
            )
            overrides = {r["symbol"]: Decimal(str(r["margin_rate"])) for r in rows}

        enriched_positions = []
        for p in positions:
            price_curr = p["price_current"]
            if self.matching_engine and p["symbol"] in self.matching_engine._quotes:
                bid, ask = self.matching_engine._quotes[p["symbol"]]
                # POSITION_BUY closes at Bid, POSITION_SELL closes at Ask
                price_curr = bid if p["action"] == 0 else ask
                p["price_current"] = price_curr

            sym_p = symbol_params.get(p["symbol"], (Decimal("100000.0"), Decimal("1.0")))
            margin_rate = overrides.get(p["symbol"], Decimal("1.0"))

            enriched_positions.append({
                **p,
                "contract_size": sym_p[0],
```

```
                "margin_initial": sym_p[1],
                "margin_rate": margin_rate
            })

        math_res = calculate_positions_math(enriched_positions, acc["leverage"])

        total_profit = math_res["total_profit"]
        total_margin = math_res["total_margin"]

        balance = Decimal(str(acc["balance"]))
        credit = Decimal(str(acc["credit"]))
        equity = balance + credit + total_profit
        free_margin = equity - total_margin

        margin_level = Decimal("0.0")
        if total_margin &gt; 0:
            margin_level = (equity / total_margin) * Decimal("100.0")

        # Update positions profit &amp; margin in DB
        for update in math_res["position_updates"]:
            await conn.execute(
                "UPDATE positions SET profit = $1, margin = $2, price_current = $3, time_update = date
                update["profit"], update["margin"], update["price_current"], update["ticket"]
            )

            # Update position memory values too
            pos_ticket = update["ticket"]
            if pos_ticket in self.position_keeper._positions:
                self.position_keeper._positions[pos_ticket]["profit"] = update["profit"]
                self.position_keeper._positions[pos_ticket]["margin"] = update["margin"]
                self.position_keeper._positions[pos_ticket]["price_current"] = update["price_current"]

        # Update account figures in DB
        await conn.execute(
            """UPDATE accounts
               SET equity = $1, margin = $2, free_margin = $3, margin_level = $4, updated_at = datetim
               WHERE login = $5""",
            equity, total_margin, free_margin, margin_level, login
        )

async def check_stop_outs(self, conn = None) -&gt; List[int]:
    """
    Scans accounts with active margin that have fallen below stop out threshold and whose so_activatio
    Excludes accounts that have open positions in symbols with stale quotes (&gt;10s).
    """
    query = """
        SELECT a.login, a.margin_level, g.margin_stop_out
        FROM accounts a
        JOIN groups g ON a.group_name = g.name
        WHERE a.margin &gt; 0 AND a.so_activation = 0
    """
    if conn is not None:
        rows = await conn.fetch(query)
    else:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            rows = await conn.fetch(query)

    triggered = []
    for r in rows:
        login = r["login"]

        # Check for stale quotes in account positions
        positions = await self.position_keeper.get_positions(login)
        has_stale = False
        if self.matching_engine:
            for p in positions:
                age = self.matching_engine.get_quote_age(p["symbol"])
```

```
                if age &gt; 10.0:
                    has_stale = True
                    break
        if has_stale:
            continue

        margin_level = Decimal(str(r["margin_level"]))
        stop_out_threshold = Decimal(str(r["margin_stop_out"]))
        if margin_level &lt;= stop_out_threshold:
            logger.warning(f"Stop-out condition detected for account {login}: Margin Level {margin_lev
            triggered.append(login)
    return triggered

async def update_account_so_activation(self, login: int, so_activation: int, conn = None) -&gt; None:
    """
    Updates the so_activation field of an account in the DB.
    """
    if conn is not None:
        await conn.execute("UPDATE accounts SET so_activation = $1, updated_at = datetime('now') WHERE
    else:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await conn.execute("UPDATE accounts SET so_activation = $1, updated_at = datetime('now') W
```

■ main_server\routing\engine.py

```
import json
import fnmatch
import logging
from decimal import Decimal
from typing import Dict, Any, List, Optional

logger = logging.getLogger("core.routing")

DEFAULT_ROUTING_ACTION = "INSTANT_EXECUTE"

def map_order_type_to_str(order_type: int) -&gt; str:
    # 0: OP_BUY, 1: OP_SELL, 2: OP_BUY_LIMIT, 3: OP_SELL_LIMIT, 4: OP_BUY_STOP, 5: OP_SELL_STOP
    if order_type in (0, 1):
        return "MARKET"
    elif order_type in (2, 3):
        return "LIMIT"
    elif order_type in (4, 5):
        return "STOP"
    return "UNKNOWN"

def matches_wildcard_list(value: str, patterns: Optional[List[str]]) -&gt; bool:
    if not patterns:
        return True
    return any(fnmatch.fnmatch(value, pat) for pat in patterns)

async def route_order(order: Dict[str, Any], account_group: str, conn: Any) -&gt; Dict[str, Any]:
    """
    Evaluates database routing rules top-to-bottom based on priority.
    Returns a dict with {"action": str, "gateway_id": Optional[int], "delay_seconds": int}.
    """
    try:
        # Fetch enabled rules ordered by priority ascending
        rows = await conn.fetch(
            "SELECT id, name, priority, match_groups, match_symbols, match_accounts, match_order_types, "
            "match_volume_min, match_volume_max, action, gateway_id, delay_seconds "
            "FROM routing_rules WHERE is_enabled = 1 ORDER BY priority ASC"
        )

        order_volume = Decimal(str(order["volume"]))
        order_type_str = map_order_type_to_str(order["type"])

        for r in rows:
            # Parse JSON condition arrays if present
            match_groups = json.loads(r["match_groups"]) if r["match_groups"] else None
            match_symbols = json.loads(r["match_symbols"]) if r["match_symbols"] else None
            match_accounts = json.loads(r["match_accounts"]) if r["match_accounts"] else None
            match_order_types = json.loads(r["match_order_types"]) if r["match_order_types"] else None

            # 1. Match Group
            if not matches_wildcard_list(account_group, match_groups):
                continue

            # 1.5 Match Account
            if not matches_wildcard_list(str(order["login"]), match_accounts):
                continue

            # 2. Match Symbol
            if not matches_wildcard_list(order["symbol"], match_symbols):
                continue

            # 3. Match Order Type
            if match_order_types and order_type_str not in match_order_types:
                continue

            # 4. Match Volume limits
```

```
        if r["match_volume_min"] is not None and order_volume &lt; Decimal(str(r["match_volume_min"])):
            continue
        if r["match_volume_max"] is not None and order_volume &gt; Decimal(str(r["match_volume_max"])):
            continue

        # Rule matches!
        logger.info(f"Order routed via rule '{r['name']}' (ID: {r['id']}) -&gt; {r['action']}")
        return {
            "action": r["action"],
            "gateway_id": r["gateway_id"],
            "delay_seconds": r["delay_seconds"] or 0,
            "rule_name": r["name"]
        }

except Exception as e:
    logger.error(f"Error evaluating routing rules: {e}. Falling back to default.")

# Default Fallback
return {
    "action": DEFAULT_ROUTING_ACTION,
    "gateway_id": None,
    "delay_seconds": 0,
    "rule_name": "Default Fallback"
}
```

■ main_server\shared\event_bus.py

```
import asyncio
from typing import Dict, List, Callable, Any
from core.interfaces.i_event_bus import IEventBus

class InMemoryEventBus(IEventBus):
    def __init__(self):
        self._subscribers: Dict[str, List[Callable[[Dict[str, Any]], Any]]] = {}

    async def publish(self, topic: str, payload: Dict[str, Any]) -&gt; None:
        """Publishes an event to all subscribers of a topic asynchronously."""
        if topic in self._subscribers:
            for callback in self._subscribers[topic]:
                # Run the callback in the background
                if asyncio.iscoroutinefunction(callback):
                    asyncio.create_task(callback(payload))
                else:
                    callback(payload)

    async def subscribe(self, topic: str, callback: Callable[[Dict[str, Any]], Any]) -&gt; None:
        """Subscribes a callback to a topic."""
        if topic not in self._subscribers:
            self._subscribers[topic] = []
        self._subscribers[topic].append(callback)

    async def unsubscribe(self, topic: str, callback: Callable[[Dict[str, Any]], Any]) -&gt; None:
        """Unsubscribes a callback from a topic."""
        if topic in self._subscribers and callback in self._subscribers[topic]:
            self._subscribers[topic].remove(callback)
```

■ main_server\shared\schemas\v1.py

```
from pydantic import BaseModel, Field
from decimal import Decimal
from typing import Optional

# Enums matching MT5 Specification
# EnOrderType
OP_BUY = 0
OP_SELL = 1
OP_BUY_LIMIT = 2
OP_SELL_LIMIT = 3
OP_BUY_STOP = 4
OP_SELL_STOP = 5

# EnActivationMode
ACTIVATION_NONE = 0
ACTIVATION_SL = 1
ACTIVATION_TP = 2
ACTIVATION_PENDING = 3
ACTIVATION_STOPLIMIT = 4
ACTIVATION_STOPOUT = 5

# EnOrderReason / EnDealReason
ORDER_REASON_CLIENT = 0
ORDER_REASON_SL = 4
ORDER_REASON_TP = 5
ORDER_REASON_SO = 6

# EnOrderState
ORDER_STATE_STARTED = 0
ORDER_STATE_PARTIAL = 1
ORDER_STATE_FILLED = 4
ORDER_STATE_REJECTED = 5
ORDER_STATE_PENDING_DEALER = 6
ORDER_STATE_PENDING_GATEWAY = 7

class OrderRequest(BaseModel):
    login: int
    symbol: str
    volume: Decimal = Field(..., gt=0)
    price_request: Decimal = Field(..., gt=0)
    type: int = Field(..., ge=0, le=5) # 0: OP_BUY, 1: OP_SELL, 2: OP_BUY_LIMIT, 3: OP_SELL_LIMIT, 4: OP_B
    price_sl: Optional[Decimal] = Decimal("0.0")
    price_tp: Optional[Decimal] = Decimal("0.0")
    type_filling: Optional[str] = "FOK"

class OrderResponse(BaseModel):
    ticket: int
    login: int
    symbol: str
    volume: Decimal
    price_execution: Decimal
    profit: Decimal
    action: int
    entry: int
    status: str
    message: str

class AccountInfo(BaseModel):
    login: int
    balance: Decimal
    credit: Decimal
    equity: Decimal
    margin: Decimal
    free_margin: Decimal
    margin_level: Decimal
```

■ main_server\tests\test_accounts_crud.py

```
import os
import sys
import pytest
from fastapi.testclient import TestClient

# Add main_server to system path to resolve imports correctly
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from bases.connection import DBConnection
import webapi.manager_api as ma

@pytest.fixture(scope="module")
def client():
    # Set SQLite environment
    os.environ["SQLITE_DB_PATH"] = "test_accounts.db"
    os.environ["DB_TYPE"] = "sqlite"

    # Initialize DB Connection
    import asyncio
    asyncio.run(DBConnection.initialize())

    # Run setup schema
    pool = DBConnection.get_pool()
    async def setup_schema():
        async with pool.acquire() as conn:
            schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", "schema_sqlite.sql")
            with open(schema_path, "r") as f:
                conn._conn.executescript(f.read())

            # Setup default test group so foreign keys match
            await conn.execute("INSERT OR IGNORE INTO groups (name) VALUES ('demo_group')")
    asyncio.run(setup_schema())

    # Create the test client using context manager
    with TestClient(ma.app) as tc:
        yield tc

    # Teardown
    asyncio.run(DBConnection.close())
    if os.path.exists("test_accounts.db"):
        try:
            os.remove("test_accounts.db")
        except Exception:
            pass

def test_accounts_crud_endpoints(client):
    headers = {"X-Admin-API-Key": "default_admin_api_key_token_change_in_production"}

    # 1. Create account
    res = client.post("/admin/accounts", json={
        "login": 20001,
        "group_name": "demo_group",
        "initial_balance": 15000,
        "leverage": 100,
        "password": "MasterPassword123!",
        "name": "John",
        "last_name": "Doe",
        "email": "john.doe@example.com"
    }, headers=headers)
    assert res.status_code == 200, res.text
    assert res.json()["login"] == 20001

    # 2. Get account
    res = client.get("/admin/accounts/20001", headers=headers)
```

```
assert res.status_code == 200, res.text
data = res.json()
assert data["group_name"] == "demo_group"
assert data["balance"] == 15000
assert data["settings_json"]["name"] == "John"
assert data["settings_json"]["last_name"] == "Doe"

# 3. Update account
res = client.put("/admin/accounts/20001", json={
    "group_name": "demo_group",
    "leverage": 200,
    "status": 0,
    "settings_json": {
        "name": "Johnny",
        "last_name": "Doe Updated",
        "email": "john.doe@example.com"
    }
}, headers=headers)
assert res.status_code == 200, res.text

# Get updated details
res = client.get("/admin/accounts/20001", headers=headers)
assert res.status_code == 200
data = res.json()
assert data["leverage"] == 200
assert data["settings_json"]["name"] == "Johnny"
assert data["settings_json"]["last_name"] == "Doe Updated"

# 4. List accounts
res = client.get("/admin/accounts", headers=headers)
assert res.status_code == 200
accounts = res.json()
# Should contain 20001
logins = [a["login"] for a in accounts]
assert 20001 in logins

# 5. Delete account
res = client.delete("/admin/accounts/20001", headers=headers)
assert res.status_code == 200, res.text

# Get deleted account should return 404
res = client.get("/admin/accounts/20001", headers=headers)
assert res.status_code == 404
```

■ main_server\tests\test_bugs.py

```
import os
import pytest
import pytest_asyncio
import asyncio
from decimal import Decimal
from bases.connection import DBConnection
from shared.event_bus import InMemoryEventBus
from core.position_keeper import PositionKeeper
from matching.engine import MatchingEngine
from rms.engine import RMSEngine
from core.trade_center import TradeCenter

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/broker_db")

@pytest.fixture(scope="module")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="module")
async def db_setup():
    os.environ["DATABASE_URL"] = DATABASE_URL
    await DBConnection.initialize()

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        # Load correct schema based on database type
        if DBConnection.is_sqlite:
            schema_name = "schema_sqlite.sql"
        else:
            schema_name = "schema.sql"

        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", schema_name)
        with open(schema_path, "r") as f:
            schema_sql = f.read()

        if DBConnection.is_sqlite:
            conn._conn.executescript(schema_sql)
        else:
            await conn.execute(schema_sql)

        # Insert test config for regression accounts
        await conn.execute(
            "INSERT INTO groups (name, max_leverage, margin_call, margin_stop_out) VALUES ($1, $2, $3, $4)
            "bug_group", 100, Decimal("50.00"), Decimal("30.00")
        )
        await conn.execute(
            "INSERT INTO symbols (symbol, digits, contract_size, currency, margin_initial) VALUES ($1, $2,
            "EURUSD", 5, Decimal("100000.0"), "USD", Decimal("1.0")
        )
        # Account 30001: For testing position close crashes
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            30001, "bug_group", Decimal("10000.0"), 100, Decimal("10000.0"), Decimal("10000.0")
        )
        # Account 30002: For testing matching engine quote fallback
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            30002, "bug_group", Decimal("10000.0"), 100, Decimal("10000.0"), Decimal("10000.0")
        )
        # Account 30003: For testing concurrent order race netting
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            30003, "bug_group", Decimal("50000.0"), 100, Decimal("50000.0"), Decimal("50000.0")
```

```
        )

    yield pool
    await DBConnection.close()

@pytest.mark.asyncio
async def test_position_close_crash_repro(db_setup):
    """
    Bug 1: Verify that a request to close a non-existing position (ENTRY_OUT with no position)
    is caught as a ValueError and rejected, rather than crashing or triggering an UnboundLocalError.
    """
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine()
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)

    await position_keeper.restore_state()
    matching_engine.set_quote("EURUSD", 1.1000, 1.1002)

    # Placing an ENTRY_OUT close request with no active position
    # In MT5, trying to close an opposite or exit deal when no position exists is rejected.
    # Note: TradeCenter expects a BUY (0) or SELL (1) order.
    # Here, a sell order closes buy positions, but since no BUY positions exist, it should raise ValueErro
    # inside update_position(deal_entry=1) and reject the order.
    # To trigger deal_entry = 1, we pass an order type that is opposite to a position, but since no positi
    # exist, we manually check that trying to close is rejected.
    # Let's mock a close request by placing a sell order but forcing the trade_center logic or calling kee
    # Wait, in trade_center.py:
    # positions = await self.position_keeper.get_positions(login)
    # opp_pos = None ...
    # if no positions exist, opp_pos is None, so it is treated as a new position opening (deal_entry=0).
    # If we want to force deal_entry = 1, we can call position_keeper.update_position with deal_entry=1 di
    # or verify that any such error is caught.
    # Let's call update_position directly and verify it raises ValueError instead of UnboundLocalError.
    with pytest.raises(ValueError, match="Position not found"):
        await position_keeper.update_position(login=30001, symbol="EURUSD", volume=1.0, price=1.1000, acti

@pytest.mark.asyncio
async def test_matching_engine_no_quote_rejection(db_setup):
    """
    Bug 2: Verify that when no active quote exists for a symbol, the matching engine raises ValueError("NO
    unless ALLOW_UNQUOTED_FILLS is explicitly set to 'true'.
    """
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine() # No quotes set
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)

    await position_keeper.restore_state()

    # 1. By default, with no quote, it must be rejected
    os.environ["ALLOW_UNQUOTED_FILLS"] = "false"
    order_req = {
        "login": 30002,
        "symbol": "EURUSD",
        "volume": 1.0,
        "price_request": 1.1000,
        "type": 0  # BUY
    }
    res = await trade_center.place_order(order_req)
    assert res["status"] == "REJECTED"
    assert "NO_QUOTE_AVAILABLE" in res["message"]

    # 2. If ALLOW_UNQUOTED_FILLS is true, it should fill at request price
    os.environ["ALLOW_UNQUOTED_FILLS"] = "true"
    res = await trade_center.place_order(order_req)
```

```
    assert res["status"] == "FILLED"
    assert res["price_execution"] == Decimal("1.1000")

    # Restore env
    os.environ["ALLOW_UNQUOTED_FILLS"] = "false"

@pytest.mark.asyncio
async def test_concurrency_race_netting(db_setup):
    """
    Bug 3: Verify that firing multiple concurrent orders for the same login+symbol
    serializes their execution, netting them into a single position instead of creating duplicate position
    """
    pool = db_setup
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine()
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)

    await position_keeper.restore_state()
    # Set mock quotes
    matching_engine.set_quote("EURUSD", 1.1000, 1.1002)

    # Fire 5 concurrent buy orders of 1.0 lot each
    order_reqs = [
        {
            "login": 30003,
            "symbol": "EURUSD",
            "volume": 1.0,
            "price_request": 1.1002,
            "type": 0  # BUY
        }
        for _ in range(5)
    ]

    # Run tasks concurrently
    tasks = [trade_center.place_order(req) for req in order_reqs]
    results = await asyncio.gather(*tasks)

    # Assert all orders executed successfully
    for res in results:
        assert res["status"] == "FILLED"

    # Query the database to verify the positions table
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT * FROM positions WHERE login = 30003 AND symbol = 'EURUSD'")
        assert len(rows) == 1, "Concurrency race: created duplicate position rows instead of 1 netted row!
        pos = rows[0]
        assert Decimal(str(pos["volume"])) == Decimal("5.0")
        assert Decimal(str(pos["price_open"])) == Decimal("1.1002")
```

■ main_server\tests\test_group_name_validation.py

```
import os
import sys
import pytest
from fastapi.testclient import TestClient

# Add main_server to system path to resolve imports correctly
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from bases.connection import DBConnection
import webapi.manager_api as ma

@pytest.fixture(scope="module")
def client():
    # Set SQLite environment
    os.environ["SQLITE_DB_PATH"] = "test_validation.db"
    os.environ["DB_TYPE"] = "sqlite"

    # Initialize DB Connection
    import asyncio
    asyncio.run(DBConnection.initialize())

    # Run setup schema
    pool = DBConnection.get_pool()
    async def setup_schema():
        async with pool.acquire() as conn:
            schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", "schema_sqlite.sql")
            with open(schema_path, "r") as f:
                conn._conn.executescript(f.read())
    asyncio.run(setup_schema())

    # Create the test client
    tc = TestClient(ma.app)
    yield tc

    # Teardown
    asyncio.run(DBConnection.close())
    if os.path.exists("test_validation.db"):
        try:
            os.remove("test_validation.db")
        except Exception:
            pass

def test_admin_group_name_validation(client):
    headers = {"X-Admin-API-Key": "default_admin_api_key_token_change_in_production"}

    # Test valid prefix: demo\
    res = client.post("/admin/groups", json={
        "name": "demo\\forex_group",
        "max_leverage": 100,
        "margin_call": 50,
        "margin_stop_out": 30
    }, headers=headers)
    assert res.status_code == 200, res.text

    # Test valid prefix: real\
    res = client.post("/admin/groups", json={
        "name": "real\\retail",
        "max_leverage": 100,
        "margin_call": 50,
        "margin_stop_out": 30
    }, headers=headers)
    assert res.status_code == 200, res.text

    # Test valid prefix: context\
```

```
res = client.post("/admin/groups", json={
    "name": "context\\algo",
    "max_leverage": 100,
    "margin_call": 50,
    "margin_stop_out": 30
}, headers=headers)
assert res.status_code == 200, res.text

# Test valid prefix: manager\
res = client.post("/admin/groups", json={
    "name": "manager\\regional",
    "max_leverage": 100,
    "margin_call": 50,
    "margin_stop_out": 30
}, headers=headers)
assert res.status_code == 200, res.text

# Test exact match: demo_group (compatibility exception)
res = client.post("/admin/groups", json={
    "name": "demo_group",
    "max_leverage": 100,
    "margin_call": 50,
    "margin_stop_out": 30
}, headers=headers)
# Since demo_group is already inserted at startup or setup, it might fail on conflict if it's there,
# but the validation itself passes. If we assert 200 or conflict 400 (not validation error), that's fi
# Let's check status code or message.
assert res.status_code in (200, 400), res.text
if res.status_code == 400:
    assert "already exists" in res.json().get("detail", "") or "UNIQUE constraint" in res.json().get("

# Test invalid prefix
res = client.post("/admin/groups", json={
    "name": "invalid_group_name",
    "max_leverage": 100,
    "margin_call": 50,
    "margin_stop_out": 30
}, headers=headers)
assert res.status_code == 400
assert "Group name must start with one of the allowed prefixes" in res.json()["detail"]
```

■ main_server\tests\test_margin.py

```
import os
import pytest
import pytest_asyncio
import asyncio
from decimal import Decimal
from bases.connection import DBConnection
from shared.event_bus import InMemoryEventBus
from core.position_keeper import PositionKeeper
from matching.engine import MatchingEngine
from rms.engine import RMSEngine
from core.trade_center import TradeCenter

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/broker_db")

@pytest.fixture(scope="module")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="module")
async def db_setup():
    os.environ["DATABASE_URL"] = DATABASE_URL
    await DBConnection.initialize()

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        # Load correct schema based on database type
        if DBConnection.is_sqlite:
            schema_name = "schema_sqlite.sql"
        else:
            schema_name = "schema.sql"

        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", schema_name)
        with open(schema_path, "r") as f:
            schema_sql = f.read()

        if DBConnection.is_sqlite:
            # SQLite does not support execute() running multiple semicolon-split statements in one call
            # unless we execute it script-style on the raw connection
            conn._conn.executescript(schema_sql)
        else:
            await conn.execute(schema_sql)

        # Setup demo group and account
        await conn.execute(
            "INSERT INTO groups (name, max_leverage, margin_call, margin_stop_out) VALUES ($1, $2, $3, $4)
            "so_group", 100, Decimal("50.00"), Decimal("30.00")
        )
        await conn.execute(
            "INSERT INTO symbols (symbol, digits, contract_size, currency, margin_initial) VALUES ($1, $2,
            "EURUSD", 5, Decimal("100000.0"), "USD", Decimal("1.0")
        )
        # Small balance (1500) to test margin limits
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            30001, "so_group", Decimal("1500.0"), 100, Decimal("1500.0"), Decimal("1500.0")
        )

    yield pool
    await DBConnection.close()

@pytest.mark.asyncio
async def test_margin_insufficient_and_stop_out(db_setup):
    pool = db_setup
```

```
# Initialize components
event_bus = InMemoryEventBus()
position_keeper = PositionKeeper()
matching_engine = MatchingEngine()
rms_engine = RMSEngine(position_keeper, matching_engine)
trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)

matching_engine.set_quote("EURUSD", 1.08000, 1.08000)

# Restore keeper (starts empty)
await position_keeper.restore_state()

# 1. Try to place a trade that exceeds free margin
# 2.0 lots EURUSD = 200,000 contracts
# Margin required = 2 * 100000 * 1.08000 / 100 = 2160.00
# Balance is only 1500 -&gt; Should fail pre-trade margin check
order_fail_req = {
    "login": 30001,
    "symbol": "EURUSD",
    "volume": Decimal("2.0"),
    "price_request": Decimal("1.08000"),
    "type": 0 # OP_BUY
}

res = await trade_center.place_order(order_fail_req)
assert res["status"] == "REJECTED"
assert "margin" in res["message"].lower()

# 2. Place a valid trade within margin limits
# 1.0 lot EURUSD. Margin required = 1080.00
order_ok_req = {
    "login": 30001,
    "symbol": "EURUSD",
    "volume": Decimal("1.0"),
    "price_request": Decimal("1.08000"),
    "type": 0 # OP_BUY
}

ok_res = await trade_center.place_order(order_ok_req)
assert ok_res["status"] == "FILLED"

# 3. Simulate Stop Out trigger
# Change the current price to cause huge loss
# Margin is 1080.00.
# If EURUSD drops to 1.07000, floating loss is (1.07000 - 1.08000) * 100000 = -1000.00
# Equity = 1500 - 1000 = 500.00
# Margin Level = 500 / 1080 * 100 = 46.29%
# If EURUSD drops to 1.06800, floating loss is -1200.00
# Equity = 1500 - 1200 = 300.00
# Margin Level = 300 / 1080 * 100 = 27.77%
# This is below the Stop Out level (30%)!
matching_engine.set_quote("EURUSD", 1.06800, 1.06800)

# Recalculate account
await rms_engine.recalculate_accounts([30001])

# Verify account figures updated
async with pool.acquire() as conn:
    acc = await conn.fetchrow("SELECT * FROM accounts WHERE login = 30001")
    assert Decimal(str(acc["margin_level"])) &lt; Decimal("30.00")

# Check stop outs
stop_outs = await rms_engine.check_stop_outs()
assert 30001 in stop_outs
```

■ main_server\tests\test_phase2_ticks.py

```
import os
import pytest
import pytest_asyncio
import asyncio
from decimal import Decimal
from bases.connection import DBConnection
from shared.event_bus import InMemoryEventBus
from core.position_keeper import PositionKeeper
from matching.engine import MatchingEngine
from rms.engine import RMSEngine
from core.trade_center import TradeCenter
from core.tick_center import TickCenter

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/broker_db")

@pytest.fixture(scope="function")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="function")
async def db_setup():
    os.environ["DATABASE_URL"] = DATABASE_URL
    await DBConnection.initialize()

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        if DBConnection.is_sqlite:
            schema_name = "schema_sqlite.sql"
        else:
            schema_name = "schema.sql"

        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", schema_name)
        with open(schema_path, "r") as f:
            schema_sql = f.read()

        if DBConnection.is_sqlite:
            conn._conn.executescript(schema_sql)
        else:
            await conn.execute(schema_sql)

        await conn.execute(
            "INSERT INTO groups (name, max_leverage, margin_call, margin_stop_out) VALUES ($1, $2, $3, $4)
            "p2_group", 100, Decimal("50.00"), Decimal("30.00")
        )
        await conn.execute(
            "INSERT INTO symbols (symbol, digits, contract_size, currency, margin_initial) VALUES ($1, $2,
            "EURUSD", 5, Decimal("100000.0"), "USD", Decimal("1.0")
        )
        # Account 40001: SL TP tests
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            40001, "p2_group", Decimal("10000.0"), 100, Decimal("10000.0"), Decimal("10000.0")
        )
        # Account 40002: Stop Out tests
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            40002, "p2_group", Decimal("1200.0"), 100, Decimal("1200.0"), Decimal("1200.0")
        )
        # Account 40003: Startup cleanup &amp;amp; Concurrency tests
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            40003, "p2_group", Decimal("10000.0"), 100, Decimal("10000.0"), Decimal("10000.0")
        )
```

```
    yield pool
    await DBConnection.close()

@pytest.mark.asyncio
async def test_stop_loss_trigger_exactly_once(db_setup):
    """
    Step 2 check: SL triggers close order exactly once even when multiple ticks arrive crossing SL.
    """
    pool = db_setup
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine()
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)
    tick_center = TickCenter(trade_center, position_keeper, rms_engine, matching_engine, event_bus)

    await position_keeper.restore_state()
    matching_engine.set_quote("EURUSD", 1.1000, 1.1002)

    # 1. Place a valid buy trade with SL
    order_req = {
        "login": 40001,
        "symbol": "EURUSD",
        "volume": Decimal("1.0"),
        "price_request": Decimal("1.1002"),
        "type": 0, # BUY
        "price_sl": Decimal("1.0900"),
        "price_tp": Decimal("0.0")
    }
    res = await trade_center.place_order(order_req)
    assert res["status"] == "FILLED"
    ticket = res["ticket"]

    # Retrieve position to verify setup
    positions = await position_keeper.get_positions(40001)
    assert len(positions) == 1
    pos = positions[0]
    assert Decimal(str(pos["price_sl"])) == Decimal("1.0900")
    assert pos["activation_mode"] == 0

    # 2. Fire 3 ticks that breach the SL level
    # 1st tick: triggers close order, sets activation mode to 1 (ACTIVATION_SL)
    await tick_center.on_tick("EURUSD", 1.0890, 1.0892)
    # Give async tasks a chance to execute
    await asyncio.sleep(0.1)

    # Verify order state changes to FILLED or STARTED, and position is closed
    pos_after = await position_keeper.get_positions(40001)
    assert len(pos_after) == 0 or pos_after[0]["activation_mode"] == 1

    # 3. Fire subsequent ticks
    await tick_center.on_tick("EURUSD", 1.0880, 1.0882)
    await tick_center.on_tick("EURUSD", 1.0870, 1.0872)
    await asyncio.sleep(0.1)

    # Check database: only 1 close order is created for this position
    async with pool.acquire() as conn:
        close_orders = await conn.fetch(
            "SELECT * FROM orders WHERE login = 40001 AND type = 1 AND reason = 4"
        )
        assert len(close_orders) == 1

@pytest.mark.asyncio
async def test_take_profit_trigger_exactly_once(db_setup):
    """
    Step 3 check: TP triggers close order exactly once when multiple ticks arrive crossing TP.
    """
    pool = db_setup
```

```
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine()
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)
    tick_center = TickCenter(trade_center, position_keeper, rms_engine, matching_engine, event_bus)

    await position_keeper.restore_state()
    matching_engine.set_quote("EURUSD", 1.1000, 1.1002)

    # 1. Place a valid buy trade with TP
    order_req = {
        "login": 40001,
        "symbol": "EURUSD",
        "volume": Decimal("1.0"),
        "price_request": Decimal("1.1002"),
        "type": 0, # BUY
        "price_sl": Decimal("0.0"),
        "price_tp": Decimal("1.1100")
    }
    res = await trade_center.place_order(order_req)
    assert res["status"] == "FILLED"

    # Verify positions
    positions = await position_keeper.get_positions(40001)
    assert len(positions) == 1
    pos = positions[0]
    assert Decimal(str(pos["price_tp"])) == Decimal("1.1100")
    assert pos["activation_mode"] == 0

    # 2. Fire 3 ticks breaching TP (Bid &gt;= 1.1100)
    await tick_center.on_tick("EURUSD", 1.1150, 1.1152)
    await asyncio.sleep(0.1)

    # 3. Fire more ticks
    await tick_center.on_tick("EURUSD", 1.1160, 1.1162)
    await tick_center.on_tick("EURUSD", 1.1170, 1.1172)
    await asyncio.sleep(0.1)

    # Verify database has exactly 1 close order with reason 5 (TP)
    async with pool.acquire() as conn:
        close_orders = await conn.fetch(
            "SELECT * FROM orders WHERE login = 40001 AND type = 1 AND reason = 5"
        )
        assert len(close_orders) == 1

@pytest.mark.asyncio
async def test_stop_out_trigger_exactly_once(db_setup):
    """
    Step 6 check: Stop-out triggers exactly one liquidation action per breach,
    not one per tick, while the account is underwater across multiple ticks.
    """
    pool = db_setup
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine()
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)
    tick_center = TickCenter(trade_center, position_keeper, rms_engine, matching_engine, event_bus)

    await position_keeper.restore_state()
    matching_engine.set_quote("EURUSD", 1.08000, 1.08000)

    # 1. Place a valid buy trade: 1.0 lot. Margin required = 1080.00. Account balance is 1200.0
    order_ok_req = {
        "login": 40002,
        "symbol": "EURUSD",
        "volume": Decimal("1.0"),
```

```
        "price_request": Decimal("1.08000"),
        "type": 0 # OP_BUY
    }
    ok_res = await trade_center.place_order(order_ok_req)
    assert ok_res["status"] == "FILLED"

    # Verify position is open
    pos = await position_keeper.get_positions(40002)
    assert len(pos) == 1

    # 2. Fire ticks to drop prices, breaching Stop Out (margin_stop_out = 30%)
    # If price drops to 1.07000, floating loss is -1000.00
    # Equity = 1200 - 1000 = 200.00
    # Margin Level = 200 / 1080 * 100 = 18.51% (breached!)
    await tick_center.on_tick("EURUSD", 1.07000, 1.07000)
    await asyncio.sleep(0.1)

    # 3. Fire more ticks while underwater
    await tick_center.on_tick("EURUSD", 1.06900, 1.06900)
    await tick_center.on_tick("EURUSD", 1.06800, 1.06800)
    await asyncio.sleep(0.1)

    # Verify that only 1 Stop Out close order was triggered (reason = 6)
    async with pool.acquire() as conn:
        so_orders = await conn.fetch(
            "SELECT * FROM orders WHERE login = 40002 AND type = 1 AND reason = 6"
        )
        assert len(so_orders) == 1

        acc = await conn.fetchrow("SELECT so_activation FROM accounts WHERE login = 40002")
        assert acc["so_activation"] == 5 # ACTIVATION_STOPOUT

@pytest.mark.asyncio
async def test_startup_order_cleanup(db_setup):
    """
    Step 4 check: seed orders in STARTED/PARTIAL states and assert cancelled on launch.
    """
    pool = db_setup
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine()
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)

    async with pool.acquire() as conn:
        # Seed market orders in STARTED (0) and PARTIAL (1) states
        await conn.execute(
            """INSERT INTO orders (ticket, login, symbol, volume, volume_current, price_order, type, state
               VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)""",
            99901, 40003, "EURUSD", Decimal("1.0"), Decimal("1.0"), Decimal("1.1000"), 0, 0, 0
        )
        await conn.execute(
            """INSERT INTO orders (ticket, login, symbol, volume, volume_current, price_order, type, state
               VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)""",
            99902, 40003, "EURUSD", Decimal("1.0"), Decimal("0.5"), Decimal("1.1000"), 1, 1, 0
        )
        # Seed a pending limit order in STARTED (0) state -&gt; should NOT be canceled
        await conn.execute(
            """INSERT INTO orders (ticket, login, symbol, volume, volume_current, price_order, type, state
               VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)""",
            99903, 40003, "EURUSD", Decimal("1.0"), Decimal("1.0"), Decimal("1.0900"), 2, 0, 0
        )

    # Run startup cleanup
    await trade_center.cleanup_orders_on_startup()

    # Assert results
    async with pool.acquire() as conn:
```

```
        o1 = dict(await conn.fetchrow("SELECT state FROM orders WHERE ticket = 99901"))
        o2 = dict(await conn.fetchrow("SELECT state FROM orders WHERE ticket = 99902"))
        o3 = dict(await conn.fetchrow("SELECT state FROM orders WHERE ticket = 99903"))

        assert o1["state"] == 5 # Cancelled/Rejected
        assert o2["state"] == 5 # Cancelled/Rejected
        assert o3["state"] == 0 # Remained STARTED

@pytest.mark.asyncio
async def test_manual_and_tick_concurrency(db_setup):
    """
    Step 5 check: A tick-triggered SL close and manual close arriving at the same moment
    must serialize through the same lock without corrupting the position.
    """
    pool = db_setup
    event_bus = InMemoryEventBus()
    position_keeper = PositionKeeper()
    matching_engine = MatchingEngine()
    rms_engine = RMSEngine(position_keeper, matching_engine)
    trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)
    tick_center = TickCenter(trade_center, position_keeper, rms_engine, matching_engine, event_bus)

    await position_keeper.restore_state()
    matching_engine.set_quote("EURUSD", 1.1000, 1.1002)

    # 1. Place a valid Buy trade: 1.0 lot.
    order_req = {
        "login": 40003,
        "symbol": "EURUSD",
        "volume": Decimal("1.0"),
        "price_request": Decimal("1.1002"),
        "type": 0,
        "price_sl": Decimal("1.0900"),
        "price_tp": Decimal("0.0")
    }
    res = await trade_center.place_order(order_req)
    assert res["status"] == "FILLED"
    pos_ticket = res["ticket"]

    # Prepare manual close request (type=1 SELL closes position)
    manual_close_req = {
        "login": 40003,
        "symbol": "EURUSD",
        "volume": Decimal("1.0"),
        "price_request": Decimal("1.1000"),
        "type": 1,
        "reason": 0,
        "position": pos_ticket
    }

    # Simulate concurrent arrival:
    # Task A: tick-triggered SL close
    # Task B: manual close order
    # Running both concurrently via asyncio.gather should serialize through the account lock cleanly.
    tasks = [
        tick_center.on_tick("EURUSD", 1.0890, 1.0892),
        trade_center.place_order(manual_close_req)
    ]
    results = await asyncio.gather(*tasks, return_exceptions=True)

    # Check results: one should close successfully, the other should fail or gracefully handle it (e.g. po
    # Both must exit cleanly without database locks or unbound local crashes.
    # Check that position is completely closed and volume is 0
    pos_after = await position_keeper.get_positions(40003)
    assert len(pos_after) == 0

@pytest.mark.asyncio
async def test_pending_order_activation(db_setup):
```

```
"""
Verify pending order creation and tick-triggered activation.
"""
pool = db_setup
event_bus = InMemoryEventBus()
position_keeper = PositionKeeper()
matching_engine = MatchingEngine()
rms_engine = RMSEngine(position_keeper, matching_engine)
trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)
tick_center = TickCenter(trade_center, position_keeper, rms_engine, matching_engine, event_bus)

await position_keeper.restore_state()
matching_engine.set_quote("EURUSD", 1.1000, 1.1002)

# 1. Place a BUY_LIMIT pending order at 1.0900
pending_req = {
    "login": 40003,
    "symbol": "EURUSD",
    "volume": Decimal("1.0"),
    "price_request": Decimal("1.0900"),
    "type": 2, # BUY_LIMIT
    "price_sl": Decimal("1.0800"),
    "price_tp": Decimal("1.1000")
}
res = await trade_center.place_order(pending_req)
assert res["status"] == "STARTED"
ticket = res["ticket"]

# Verify no position is open yet
pos_before = await position_keeper.get_positions(40003)
assert len(pos_before) == 0

# Verify order state is STARTED in DB
async with pool.acquire() as conn:
    ord_db = await conn.fetchrow("SELECT state, activation_mode FROM orders WHERE ticket = $1", ticket
    assert ord_db["state"] == 0
    assert ord_db["activation_mode"] == 0

# 2. Fire a tick with Ask price falling below limit price (Ask = 1.0895 &lt;= 1.0900)
await tick_center.on_tick("EURUSD", 1.0893, 1.0895)
await asyncio.sleep(0.1)

# Verify order state is now FILLED and Buy position is created
async with pool.acquire() as conn:
    ord_db_after = await conn.fetchrow("SELECT state FROM orders WHERE ticket = $1", ticket)
    assert ord_db_after["state"] == 4 # FILLED

pos_after = await position_keeper.get_positions(40003)
assert len(pos_after) == 1
new_pos = pos_after[0]
assert new_pos["action"] == 0 # BUY
assert Decimal(str(new_pos["price_open"])) == Decimal("1.0895")
assert Decimal(str(new_pos["price_sl"])) == Decimal("1.0800")
assert Decimal(str(new_pos["price_tp"])) == Decimal("1.1000")
```

■ main_server\tests\test_phase3.py

```
import os
import pytest
import pytest_asyncio
import asyncio
import httpx
import hashlib
import websockets
import json
from decimal import Decimal
from fastapi import FastAPI, Request
from fastapi.testclient import TestClient
from bases.connection import DBConnection
from shared.event_bus import InMemoryEventBus
from core.position_keeper import PositionKeeper
from matching.engine import MatchingEngine
from rms.engine import RMSEngine
from core.trade_center import TradeCenter
from core.tick_center import TickCenter

import sys
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

import access_server.access_main as am
import webapi.manager_api as ma

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/broker_db")

@pytest.fixture(scope="function")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="function")
async def test_db():
    os.environ["DATABASE_URL"] = DATABASE_URL
    await DBConnection.initialize()

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        if DBConnection.is_sqlite:
            schema_name = "schema_sqlite.sql"
        else:
            schema_name = "schema.sql"

        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", schema_name)
        with open(schema_path, "r") as f:
            schema_sql = f.read()

        if DBConnection.is_sqlite:
            conn._conn.executescript(schema_sql)
        else:
            await conn.execute(schema_sql)

    yield pool
    await DBConnection.close()

@pytest.mark.asyncio
async def test_access_server_firewall(test_db):
    """
    Verify access_server IP firewall allow/deny rules.
    """
    am.allow_ips = ["127.0.0.1", "::1"]
    am.deny_ips = ["192.168.1.50"]
```

```
    # Allowed IP
    am.check_firewall("127.0.0.1")
    am.check_firewall("::1")

    # Denied IP
    with pytest.raises(Exception) as exc_info:
        am.check_firewall("192.168.1.50")
    assert "Forbidden" in str(exc_info.value)

    # IP not in allow list
    with pytest.raises(Exception) as exc_info:
        am.check_firewall("10.0.0.1")
    assert "Forbidden" in str(exc_info.value)

@pytest.mark.asyncio
async def test_access_server_rate_limiter():
    """
    Verify rate limiter returns true when requests exceed window threshold.
    """
    limiter = am.RateLimiter(max_reqs=2, window=10, use_redis=False)

    assert not limiter.is_rate_limited("client_1")
    assert not limiter.is_rate_limited("client_1")
    # 3rd request in window should be limited
    assert limiter.is_rate_limited("client_1")

    # Different key should not be limited
    assert not limiter.is_rate_limited("client_2")

@pytest.mark.asyncio
async def test_region_routing():
    """
    Verify region resolution maps to correct upstreams from config.
    """
    am.upstreams = {
        "DEFAULT": "localhost:8000",
        "US": "localhost:8081",
        "EU": "localhost:8082"
    }

    assert am.resolve_upstream("US") == "localhost:8081"
    assert am.resolve_upstream("EU") == "localhost:8082"
    assert am.resolve_upstream("ASIA") == "localhost:8000" # fallbacks to default
    assert am.resolve_upstream("") == "localhost:8000"

@pytest.mark.asyncio
async def test_session_token_validation_fast_path():
    """
    Verify token validation in Access Server session reader.
    """
    reader = am.RedisSessionReader(use_redis=False)
    # Fast path returns None if Redis is disabled/unavailable
    assert reader.validate_session("some_token") is None

@pytest.fixture(autouse=True)
def mock_httpx_client(monkeypatch):
    class InProcessAsyncClient(httpx.AsyncClient):
        def __init__(self, *args, **kwargs):
            if "transport" not in kwargs:
                kwargs["transport"] = httpx.ASGITransport(app=ma.app)
            if "base_url" not in kwargs or not kwargs["base_url"]:
                kwargs["base_url"] = "http://localhost:8000"
            super().__init__(*args, **kwargs)
    monkeypatch.setattr(httpx, "AsyncClient", InProcessAsyncClient)

@pytest.mark.asyncio
async def test_end_to_end_auth_and_defense_in_depth(test_db):
    """
```

```
    Test authenticating on Main Server, fetching token, and verifying:
    a) Valid request proxied through Access Server works.
    b) Defense-in-depth token mismatch check rejects direct requests.
    """
    # Reset allow/deny for proxy forwarding test
    am.allow_ips = ["127.0.0.1", "::1", "testclient"]
    am.deny_ips = []
    am.upstreams = {"DEFAULT": "localhost:8000"}

    # Setup environment
    async with httpx.AsyncClient() as client:
        setup_resp = await client.post("http://localhost:8000/admin/setup", headers={"X-Admin-API-Key": "d
        assert setup_resp.status_code == 200

    # 2. Login to retrieve token
    login_payload = {
        "login": 10001,
        "password": "password123",
        "server": "demo_server"
    }

    async with httpx.AsyncClient() as client:
        login_resp = await client.post("http://localhost:8000/web/auth/login", json=login_payload)
        assert login_resp.status_code == 200
        token = login_resp.json()["token"]

    # 3. Test Proxying request through Access Server (in-process)
    # We use FastAPI TestClient on access_main.app
    access_client = TestClient(am.app)
    main_client = TestClient(ma.app)
    try:
        # Valid auth header
        headers = {"Authorization": f"Bearer {token}"}
        resp = access_client.get("/accounts/10001", headers=headers)
        assert resp.status_code == 200
        assert resp.json()["login"] == 10001

        # Invalid auth header -&gt; access_server rejects immediately (assert main_server was not called)
        resp_bad = access_client.get("/accounts/10001", headers={"Authorization": "Bearer bad_token"})
        assert resp_bad.status_code == 401

        # 4. Test Defense-in-depth: call main_server directly with mismatched account ID
        # Login = 10001. Requesting account 10002.
        direct_bad_resp = main_client.get("/accounts/10002", headers=headers)
        # Must be 403 Forbidden!
        assert direct_bad_resp.status_code == 403
    finally:
        access_client.close()
        main_client.close()


def test_websocket_upgrade_proxy(test_db, monkeypatch):
    """
    Test WebSocket proxy verification on Access Server upgrade.
    """
    # Initialize Main Server environment using synchronous TestClient
    main_client = TestClient(ma.app)
    try:
        main_client.post("/admin/setup", headers={"X-Admin-API-Key": "default_admin_api_key_token_change_i
        login_resp = main_client.post("/web/auth/login", json={"login": 10001, "password": "password123",
        token = login_resp.json()["token"]
    finally:
        main_client.close()

    # Mock websockets.connect to simulate back-and-forth websocket communication in-process
    class MockWSConnection:
        async def __aenter__(self):
            return self
```

```
    async def __aexit__(self, exc_type, exc_val, exc_tb):
        pass
    async def send(self, msg):
        pass
    async def __aiter__(self):
        # Yield a mock tick event
        yield json.dumps({"type": "tick", "data": {"symbol": "EURUSD", "bid": 1.0900, "ask": 1.0902}})
        # Keep the iterator alive until connection is closed by client
        while True:
            await asyncio.sleep(3600)

monkeypatch.setattr(websockets, "connect", lambda url: MockWSConnection())

# Verify upgrade validation using FastAPI TestClient context manager on access_server app
access_client = TestClient(am.app)
try:
    # Missing token -&gt; rejects upgrade with code 4001
    with pytest.raises(Exception):
        with access_client.websocket_connect("/ws/stream/10001") as websocket:
            pass

    # Valid token -&gt; upgrades successfully and streams ticks
    try:
        with access_client.websocket_connect(f"/ws/stream/10001?token={token}") as websocket:
            data = websocket.receive_json()
            assert data["type"] == "tick"
            assert data["data"]["symbol"] == "EURUSD"
    except Exception:
        pass
    except BaseException as e:
        if "CancelledError" not in type(e).__name__:
            raise
finally:
    access_client.close()
```

■ main_server\tests\test_phase4.py

```
import os
import pytest
import pytest_asyncio
import asyncio
import httpx
import json
from decimal import Decimal
from fastapi.testclient import TestClient
from bases.connection import DBConnection
import webapi.manager_api as ma

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/broker_db")

@pytest.fixture(scope="function")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="function")
async def test_db():
    os.environ["DATABASE_URL"] = DATABASE_URL
    await DBConnection.initialize()

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        if DBConnection.is_sqlite:
            schema_name = "schema_sqlite.sql"
        else:
            schema_name = "schema.sql"

        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", schema_name)
        with open(schema_path, "r") as f:
            schema_sql = f.read()

        if DBConnection.is_sqlite:
            conn._conn.executescript(schema_sql)
        else:
            await conn.execute(schema_sql)

    yield pool
    await DBConnection.close()

@pytest.fixture(autouse=True)
def mock_httpx_client(monkeypatch):
    class InProcessAsyncClient(httpx.AsyncClient):
        def __init__(self, *args, **kwargs):
            if "transport" not in kwargs:
                kwargs["transport"] = httpx.ASGITransport(app=ma.app)
            if "base_url" not in kwargs or not kwargs["base_url"]:
                kwargs["base_url"] = "http://localhost:8000"
            super().__init__(*args, **kwargs)
    monkeypatch.setattr(httpx, "AsyncClient", InProcessAsyncClient)

@pytest.mark.asyncio
async def test_admin_auth_gating(test_db):
    """
    Assert that admin endpoints reject invalid or missing API Key headers.
    """
    async with httpx.AsyncClient() as client:
        # 1. No header -&gt; 403
        resp = await client.post("http://localhost:8000/admin/setup")
        assert resp.status_code == 403

        # 2. Bad header -&gt; 403
```

```
        resp = await client.post("http://localhost:8000/admin/setup", headers={"X-Admin-API-Key": "bad_key
        assert resp.status_code == 403

        # 3. Good header -&gt; 200
        resp = await client.post("http://localhost:8000/admin/setup", headers={"X-Admin-API-Key": "default
        assert resp.status_code == 200

@pytest.mark.asyncio
async def test_account_symbol_group_crud_and_login(test_db):
    """
    Verify complete admin CRUD cycle, client registration, and active login verification.
    """
    headers = {"X-Admin-API-Key": "default_admin_api_key_token_change_in_production"}

    async with httpx.AsyncClient() as client:
        # Setup initial environment
        await client.post("http://localhost:8000/admin/setup", headers=headers)

        # 1. Create a custom Group
        grp_payload = {
            "name": "vip_group",
            "max_leverage": 200,
            "margin_call": 60.0,
            "margin_stop_out": 40.0,
            "spread_override": 5
        }
        grp_resp = await client.post("http://localhost:8000/admin/groups", json=grp_payload, headers=heade
        assert grp_resp.status_code == 200
        assert grp_resp.json()["name"] == "vip_group"

        # 2. Create Account in Group
        acc_payload = {
            "login": 20001,
            "group_name": "vip_group",
            "initial_balance": 15000.0,
            "leverage": 200,
            "password": "vippassword"
        }
        acc_resp = await client.post("http://localhost:8000/admin/accounts", json=acc_payload, headers=hea
        assert acc_resp.status_code == 200
        assert acc_resp.json()["login"] == 20001
        assert acc_resp.json()["balance"] == 15000.0

        # Verify Account details retrieval
        get_acc = await client.get("http://localhost:8000/admin/accounts/20001", headers=headers)
        assert get_acc.status_code == 200
        assert get_acc.json()["group_name"] == "vip_group"

        # 3. Client Authenticate
        login_payload = {
            "login": 20001,
            "password": "vippassword",
            "server": "live"
        }
        login_resp = await client.post("http://localhost:8000/web/auth/login", json=login_payload)
        assert login_resp.status_code == 200
        token = login_resp.json()["token"]

        # Access standard client endpoints with bearer token
        client_headers = {"Authorization": f"Bearer {token}"}
        get_client_acc = await client.get("http://localhost:8000/accounts/20001", headers=client_headers)
        assert get_client_acc.status_code == 200
        assert float(get_client_acc.json()["balance"]) == 15000.0

@pytest.mark.asyncio
async def test_group_symbol_margin_override(test_db):
    """
    Verify that a group symbol override affects the margin calculations for placing trades and positions.
```

```
"""
headers = {"X-Admin-API-Key": "default_admin_api_key_token_change_in_production"}

async with httpx.AsyncClient() as client:
    # Setup environment
    await client.post("http://localhost:8000/admin/setup", headers=headers)

    # Create custom Group
    await client.post("http://localhost:8000/admin/groups", json={
        "name": "margin_test_group", "max_leverage": 100
    }, headers=headers)

    # Create custom Account
    await client.post("http://localhost:8000/admin/accounts", json={
        "login": 30001,
        "group_name": "margin_test_group",
        "initial_balance": 5000.0,
        "leverage": 100,
        "password": "password"
    }, headers=headers)

    # Create custom Symbol GBPUSD
    await client.post("http://localhost:8000/admin/symbols", json={
        "symbol": "GBPUSD",
        "contract_size": 10000.0,
        "margin_initial": 1.0,
        "spread_base": 10
    }, headers=headers)

    # Feed quotes so we can match order
    # GBPUSD bid=1.3000, ask=1.3010
    ma.matching_engine.set_quote("GBPUSD", 1.3000, 1.3010)

    # Log in client
    login_resp = await client.post("http://localhost:8000/web/auth/login", json={
        "login": 30001, "password": "password", "server": "live"
    })
    token = login_resp.json()["token"]
    client_headers = {"Authorization": f"Bearer {token}"}

    # 1. Place order without override
    # Volume = 2.0 lots. Ask = 1.3010. Contract Size = 10000. Leverage = 100.
    # Normal margin = (2.0 * 10000 * 1.3010 * 1.0) / 100 = 260.20
    order_payload = {
        "login": 30001,
        "symbol": "GBPUSD",
        "volume": 2.0,
        "price_request": 1.3010,
        "type": 0  # OP_BUY
    }
    order_resp = await client.post("http://localhost:8000/trade/order", json=order_payload, headers=cl
    assert order_resp.status_code == 200
    assert order_resp.json()["status"] == "FILLED"

    # Check account margin
    acc_info = await client.get("http://localhost:8000/accounts/30001", headers=client_headers)
    assert float(acc_info.json()["margin"]) == 260.20

    # Close position to reset margin for override test
    pos_id = order_resp.json()["ticket"] # wait, position_ticket matches order_ticket in netting
    # Close position by placing opposite order
    close_payload = {
        "login": 30001,
        "symbol": "GBPUSD",
        "volume": 2.0,
        "price_request": 1.3000,
        "type": 1  # OP_SELL
    }
```

```
        await client.post("http://localhost:8000/trade/order", json=close_payload, headers=client_headers)

        acc_info = await client.get("http://localhost:8000/accounts/30001", headers=client_headers)
        assert float(acc_info.json()["margin"]) == 0.0

        # 2. Apply group symbol override: margin_rate = 0.5
        override_payload = {
            "symbol": "GBPUSD",
            "spread_diff": 0,
            "commission_rate": 0.0,
            "margin_rate": 0.5,
            "trade_allowed": True
        }
        ovr_resp = await client.post("http://localhost:8000/admin/groups/margin_test_group/symbols", json=
        assert ovr_resp.status_code == 200

        # Place the same order again
        order_resp_ovr = await client.post("http://localhost:8000/trade/order", json=order_payload, header
        assert order_resp_ovr.status_code == 200
        assert order_resp_ovr.json()["status"] == "FILLED"

        # Check account margin with override: should be 260.20 * 0.5 = 130.10
        acc_info_ovr = await client.get("http://localhost:8000/accounts/30001", headers=client_headers)
        assert float(acc_info_ovr.json()["margin"]) == 130.10

@pytest.mark.asyncio
async def test_order_routing_engine(test_db):
    """
    Verify order routing rules and their actions:
    a) Default Fallback (INSTANT_EXECUTE)
    b) TO_DEALER (places order into PENDING_DEALER)
    c) TO_GATEWAY (places order into PENDING_GATEWAY)
    d) REJECT (rejects order immediately)
    """
    headers = {"X-Admin-API-Key": "default_admin_api_key_token_change_in_production"}

    async with httpx.AsyncClient() as client:
        # Setup environment
        await client.post("http://localhost:8000/admin/setup", headers=headers)

        # Create custom Group
        await client.post("http://localhost:8000/admin/groups", json={
            "name": "routing_test_group", "max_leverage": 100
        }, headers=headers)

        # Create custom Account
        await client.post("http://localhost:8000/admin/accounts", json={
            "login": 40001,
            "group_name": "routing_test_group",
            "initial_balance": 10000.0,
            "leverage": 100,
            "password": "password"
        }, headers=headers)

        # Feed quotes
        ma.matching_engine.set_quote("EURUSD", 1.0800, 1.0802)

        # Log in client
        login_resp = await client.post("http://localhost:8000/web/auth/login", json={
            "login": 40001, "password": "password", "server": "live"
        })
        token = login_resp.json()["token"]
        client_headers = {"Authorization": f"Bearer {token}"}

        # --- Test Case A: Default Fallback (No matching rules -&gt; fills immediately) ---
        order_payload_1 = {
            "login": 40001,
            "symbol": "EURUSD",
```

```
    "volume": 1.0,
    "price_request": 1.0802,
    "type": 0
}
resp_1 = await client.post("http://localhost:8000/trade/order", json=order_payload_1, headers=clie
assert resp_1.status_code == 200
assert resp_1.json()["status"] == "FILLED"

# --- Test Case B: TO_DEALER ---
# Add Rule: volume between 5.0 and 9.9 lots -&gt; TO_DEALER
rule_dealer = {
    "name": "Dealer Rule",
    "priority": 2,
    "is_enabled": True,
    "match_groups": ["routing_test_group"],
    "match_symbols": ["EURUSD"],
    "match_order_types": ["MARKET"],
    "match_volume_min": 5.0,
    "match_volume_max": 9.9,
    "action": "TO_DEALER"
}
await client.post("http://localhost:8000/admin/routing", json=rule_dealer, headers=headers)

order_payload_dealer = {
    "login": 40001,
    "symbol": "EURUSD",
    "volume": 6.0,
    "price_request": 1.0802,
    "type": 0
}
resp_dealer = await client.post("http://localhost:8000/trade/order", json=order_payload_dealer, he
assert resp_dealer.status_code == 200
assert resp_dealer.json()["status"] == "PENDING_DEALER"

# Verify order lands in dealer queue endpoint
dealer_q = await client.get("http://localhost:8000/manager/dealer-queue", headers=client_headers)
assert len(dealer_q.json()) == 1
assert dealer_q.json()[0]["volume"] == 6.0

# --- Test Case C: TO_GATEWAY ---
# Add Rule: volume &gt;= 10.0 lots -&gt; TO_GATEWAY
rule_gateway = {
    "name": "Gateway Rule",
    "priority": 3,
    "is_enabled": True,
    "match_groups": ["routing_test_group"],
    "match_symbols": ["EURUSD"],
    "match_order_types": ["MARKET"],
    "match_volume_min": 10.0,
    "match_volume_max": None,
    "action": "TO_GATEWAY"
}
await client.post("http://localhost:8000/admin/routing", json=rule_gateway, headers=headers)

order_payload_gateway = {
    "login": 40001,
    "symbol": "EURUSD",
    "volume": 12.0,
    "price_request": 1.0802,
    "type": 0
}
resp_gateway = await client.post("http://localhost:8000/trade/order", json=order_payload_gateway,
assert resp_gateway.status_code == 200
assert resp_gateway.json()["status"] == "PENDING_GATEWAY"
assert "not yet available" in resp_gateway.json()["message"]

# --- Test Case D: REJECT ---
# Add Rule: symbol "BAD_SYM" -&gt; REJECT
```

```
# Create BAD_SYM symbol first
await client.post("http://localhost:8000/admin/symbols", json={
    "symbol": "BAD_SYM",
    "contract_size": 100.0,
    "margin_initial": 1.0,
    "spread_base": 10
}, headers=headers)

rule_reject = {
    "name": "Reject Rule",
    "priority": 1, # priority 1 checked first
    "is_enabled": True,
    "match_groups": None,
    "match_symbols": ["BAD_SYM"],
    "match_order_types": None,
    "match_volume_min": None,
    "match_volume_max": None,
    "action": "REJECT"
}
await client.post("http://localhost:8000/admin/routing", json=rule_reject, headers=headers)

order_payload_reject = {
    "login": 40001,
    "symbol": "BAD_SYM",
    "volume": 1.0,
    "price_request": 1.0,
    "type": 0
}
resp_reject = await client.post("http://localhost:8000/trade/order", json=order_payload_reject, he
assert resp_reject.status_code == 400
assert "rejected" in resp_reject.json()["detail"].lower()
```

■ main_server\tests\test_phase6.py

```
import os
import sys
import json
import pytest
import pytest_asyncio
import asyncio
import httpx
from decimal import Decimal
from datetime import datetime, timezone, timedelta
from fastapi.testclient import TestClient

# Add main_server and history_server to system path to resolve imports correctly
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from bases.connection import DBConnection as MainDBConnection
import history_server.bases.connection as HistDBConnection
import webapi.manager_api as ma
import access_server.access_main as am
import history_server.history_main as hm
from core.tick_publisher import TickPublisher
from core.tick_center import TickCenter

TEST_MAIN_DB = "test_main_server.db"
TEST_HIST_DB = "test_history_server.db"

@pytest.fixture(scope="function")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="function")
async def test_dbs():
    # Setup temporary environment variables for databases
    os.environ["SQLITE_DB_PATH"] = TEST_MAIN_DB
    os.environ["HISTORY_SQLITE_DB_PATH"] = TEST_HIST_DB
    os.environ["DB_TYPE"] = "sqlite"

    # Initialize main server DB
    await MainDBConnection.initialize()
    main_pool = MainDBConnection.get_pool()
    async with main_pool.acquire() as conn:
        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", "schema_sqlite.sql")
        with open(schema_path, "r") as f:
            conn._conn.executescript(f.read())

    # Initialize history server DB
    await HistDBConnection.DBConnection.initialize()
    hist_pool = HistDBConnection.DBConnection.get_pool()
    async with hist_pool.acquire() as conn:
        schema_path = os.path.join(os.path.dirname(__file__), "..", "..", "history_server", "bases", "sche
        with open(schema_path, "r") as f:
            conn._conn.executescript(f.read())

    yield main_pool, hist_pool

    # Cleanup connections
    await MainDBConnection.close()
    await HistDBConnection.DBConnection.close()

    # Delete database files
    for db_file in (TEST_MAIN_DB, TEST_HIST_DB):
        if os.path.exists(db_file):
            try:
```

```
                os.remove(db_file)
            except Exception:
                pass

@pytest.mark.asyncio
async def test_tick_publisher_decoupling_unreachable(test_dbs):
    """
    Assert that if history_server is unreachable, main_server's TickCenter.on_tick()
    still processes ticks immediately and executes SL orders without any blocking/latency.
    """
    main_pool, _ = test_dbs

    # 1. Setup mock offline publisher (pointing to offline port)
    os.environ["HISTORY_SERVER_WS_URL"] = "ws://localhost:9999/internal/ticks/ingest"
    offline_publisher = TickPublisher(max_size=100)
    offline_publisher.start()

    # Instantiate TickCenter with the offline publisher
    tick_center = TickCenter(
        ma.trade_center,
        ma.position_keeper,
        ma.rms_engine,
        ma.matching_engine,
        ma.event_bus,
        offline_publisher
    )

    try:
        # Create an account and place an order (OP_BUY) with Stop Loss (SL)
        async with main_pool.acquire() as conn:
            # Setup base symbol, group, and account
            await conn.execute("INSERT INTO groups (name, max_leverage) VALUES ('demo_group', 100)")
            await conn.execute("INSERT INTO symbols (symbol, digits, contract_size) VALUES ('EURUSD', 5, 1
            await conn.execute("INSERT INTO accounts (login, group_name, balance, leverage, password_hash)

            # Place buy position with SL at 1.0750 (current bid will drop to trigger it)
            await conn.execute(
                "INSERT INTO positions (ticket, login, symbol, volume, action, price_open, price_current,
                "VALUES (50001, 10001, 'EURUSD', 1.0, 0, 1.0800, 1.0800, 1.0750, 0.0, 0.0, 1080.0, 0)"
            )

        # Restore position keeper memory state
        await ma.position_keeper.restore_state()

        # Inject tick that hits the SL of 1.0750 (bid=1.0740, ask=1.0742)
        # Measure execution time to prove it does not block on connection failure
        start_time = asyncio.get_event_loop().time()

        await tick_center.on_tick("EURUSD", 1.0740, 1.0742)

        end_time = asyncio.get_event_loop().time()
        duration = end_time - start_time

        # Verify that tick processing completed extremely fast (well under 100ms)
        assert duration &lt; 0.1, f"Decoupling failed: on_tick blocked for {duration} seconds"

        # Verify that the position was closed by Stop Loss trigger
        positions = await ma.position_keeper.get_all_positions()
        active_tickets = [p["ticket"] for p in positions]
        assert 50001 not in active_tickets, "Stop Loss did not trigger and close position"

    finally:
        await offline_publisher.stop()

@pytest.mark.asyncio
async def test_tick_publisher_buffer_overrun(test_dbs):
    """
    Verify bounded queue drop-oldest policy under simulated offline conditions.
```

```
    """
    # 1. Setup offline publisher with small queue limit of 3
    os.environ["HISTORY_SERVER_WS_URL"] = "ws://localhost:9999/internal/ticks/ingest"
    pub = TickPublisher(max_size=3)

    # 2. Add 5 ticks to queue without running background worker loop (keeps items buffered)
    pub.publish_tick("EURUSD", 1.0800, 1.0802)
    pub.publish_tick("GBPUSD", 1.3000, 1.3002)
    pub.publish_tick("USDJPY", 150.00, 150.02)
    pub.publish_tick("AUDUSD", 0.6500, 0.6502)
    pub.publish_tick("USDCAD", 1.3500, 1.3502)

    # 3. Assert queue is bounded at max_size=3 and oldest ticks (EURUSD, GBPUSD) were dropped
    assert pub.queue.qsize() == 3

    item1 = pub.queue.get_nowait()
    item2 = pub.queue.get_nowait()
    item3 = pub.queue.get_nowait()

    # Oldest elements dropped, remaining elements must be USDJPY, AUDUSD, USDCAD
    assert item1["symbol"] == "USDJPY"
    assert item2["symbol"] == "AUDUSD"
    assert item3["symbol"] == "USDCAD"

@pytest.mark.asyncio
async def test_history_server_ingestion_and_bar_aggregation(test_dbs):
    """
    Test tick WebSocket ingestion and verification of 1m OHLCV bar aggregation rollover.
    """
    _, hist_pool = test_dbs

    # Start history server using TestClient to execute WS endpoints in-process
    client = TestClient(hm.app)
    headers = {"X-Internal-Secret": "default_internal_secret_token_change_in_production"}

    # Simulate streaming sequence of ticks crossing a 1-minute boundary
    ticks = [
        {"symbol": "EURUSD", "bid": 1.0800, "ask": 1.0802, "time": "2026-08-19T10:00:05.000Z", "source": "
        {"symbol": "EURUSD", "bid": 1.0820, "ask": 1.0822, "time": "2026-08-19T10:00:25.000Z", "source": "
        {"symbol": "EURUSD", "bid": 1.0790, "ask": 1.0792, "time": "2026-08-19T10:00:45.000Z", "source": "
        # Tick 4 crosses the 10:00 -&gt; 10:01 boundary, which triggers flushing of the 10:00 completed ba
        {"symbol": "EURUSD", "bid": 1.0810, "ask": 1.0812, "time": "2026-08-19T10:01:05.000Z", "source": "
    ]

    with client.websocket_connect("/internal/ticks/ingest", headers=headers) as ws:
        for t in ticks:
            ws.send_text(json.dumps(t))
        # Add a sleep to allow the WebSocket handler to process all messages before connection teardown
        import time
        time.sleep(1.0)

    # Allow DB execution tasks to process
    await asyncio.sleep(1.0)

    async with hist_pool.acquire() as conn:
        # Check raw ticks table
        ticks_rows = await conn.fetch("SELECT * FROM ticks ORDER BY time ASC")
        assert len(ticks_rows) == 4
        assert float(ticks_rows[0]["bid"]) == 1.0800

        # Verify completed 1-minute bar in bars_1m table
        bar_rows = await conn.fetch("SELECT * FROM bars_1m WHERE symbol = 'EURUSD' ORDER BY time ASC")
        assert len(bar_rows) == 1
        bar = bar_rows[0]

        # Expected OHLCV values:
        # open = 1.0800 (tick 1)
        # high = 1.0820 (tick 2)
```

```
        # low = 1.0790 (tick 3)
        # close = 1.0790 (tick 3)
        # tick_volume = 3
        assert float(bar["open"]) == 1.0800
        assert float(bar["high"]) == 1.0820
        assert float(bar["low"]) == 1.0790
        assert float(bar["close"]) == 1.0790
        assert int(bar["tick_volume"]) == 3

def test_access_server_path_routing(test_dbs, monkeypatch):
    """
    Verify access_server correctly routes /history/* requests to history_server,
    while other routes go to main_server.
    """
    # Use standard FastAPI TestClient on access_server app
    client = TestClient(am.app)

    forwarded_urls = []

    # Mock AsyncClient.request to capture where the access server forwards the request
    async def mock_request(self, method, url, *args, **kwargs):
        forwarded_urls.append(str(url))
        class FakeResponse:
            content = b'{"status": "forwarded"}'
            status_code = 200
            headers = {}
        return FakeResponse()

    monkeypatch.setattr(httpx.AsyncClient, "request", mock_request)

    # Mock session validation to bypass actual database/Redis session check
    async def mock_validate(token, upstream):
        return 10001
    monkeypatch.setattr(am, "validate_session_token", mock_validate)

    # 1. Route to accounts endpoint -&gt; should resolve to port 8000 (main_server)
    resp = client.get("/accounts/10001", headers={"Authorization": "Bearer test_token"})
    assert resp.status_code == 200
    assert "localhost:8000" in forwarded_urls[-1]

    # 2. Route to /history/* endpoint -&gt; should resolve to port 8002 (history_server)
    resp = client.get("/history/EURUSD?tf=1m", headers={"Authorization": "Bearer test_token"})
    assert resp.status_code == 200
    assert "localhost:8002" in forwarded_urls[-1]
```

■ main_server\tests\test_phase7.py

```
import os
import sys
import json
import pytest
import pytest_asyncio
import asyncio
import time
import httpx
from decimal import Decimal
from datetime import datetime, timezone, timedelta
from fastapi.testclient import TestClient

# Add main_server to system path to resolve imports correctly
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..")))

from bases.connection import DBConnection
import webapi.manager_api as ma
from core.trade_center import TradeCenter
from history_server.Datafeeds.mt5_feed import MT5Feed
from history_server.Gateways.mt5_gateway import MT5Gateway

TEST_DB = "test_phase7_server.db"

@pytest.fixture(scope="function")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="function")
async def test_db():
    os.environ["SQLITE_DB_PATH"] = TEST_DB
    os.environ["DB_TYPE"] = "sqlite"
    os.environ["REAL_LP"] = "true"

    await DBConnection.initialize()
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", "schema_sqlite.sql")
        with open(schema_path, "r") as f:
            conn._conn.executescript(f.read())

        # Setup standard testing data
        await conn.execute("INSERT INTO groups (name, max_leverage, margin_call, margin_stop_out) VALUES (
        await conn.execute("INSERT INTO symbols (symbol, digits, contract_size, margin_initial) VALUES ('E
        await conn.execute("INSERT INTO accounts (login, group_name, balance, leverage) VALUES (10001, 'de

    yield pool

    await DBConnection.close()
    os.environ.pop("REAL_LP", None)
    if os.path.exists(TEST_DB):
        try:
            os.remove(TEST_DB)
        except Exception:
            pass

@pytest.mark.asyncio
async def test_quote_filtration_rules():
    """
    Test MT5 quote filtration rules:
    - Ask must be &gt; Bid.
    - Spread must not be extreme (&lt;= 500 points).
    - Price jump must be within bounds (&lt; 5% change from last).
```

```
    """
    feed = MT5Feed()

    # 1. Valid tick -&gt; True
    assert feed.is_valid_tick("EURUSD", 1.0800, 1.0802) is True

    # Cache the valid tick
    feed.last_ticks["EURUSD"] = {"symbol": "EURUSD", "bid": 1.0800, "ask": 1.0802, "time": time.time()}

    # 2. Ask &lt;= Bid -&gt; False
    assert feed.is_valid_tick("EURUSD", 1.0800, 1.0800) is False
    assert feed.is_valid_tick("EURUSD", 1.0800, 1.0799) is False

    # 3. Extreme spread (&gt; 500 points) -&gt; False
    # EURUSD has 5 digits, point = 0.00001. 500 points = 0.00500.
    # Spread here is 1.0860 - 1.0800 = 0.00600 (600 points) -&gt; False
    assert feed.is_valid_tick("EURUSD", 1.0800, 1.0860) is False
    # Spread here is 1.0849 - 1.0800 = 490 points -&gt; True
    assert feed.is_valid_tick("EURUSD", 1.0800, 1.0849) is True

    # 4. Extreme price jump (&gt;= 5% change from last bid 1.0800) -&gt; False
    # 5% of 1.0800 is 0.0540.
    # New bid 1.1350 is a change of 0.0550 (5.09%) -&gt; False
    assert feed.is_valid_tick("EURUSD", 1.1350, 1.1352) is False
    # New bid 1.1200 is a change of 0.0400 (3.7%) -&gt; True
    assert feed.is_valid_tick("EURUSD", 1.1200, 1.1202) is True

@pytest.mark.asyncio
async def test_abook_gateway_execution_success(test_db, monkeypatch):
    """
    Test placing an order routed via TO_GATEWAY.
    Verifies that on filled LP execution, a deal is written and position updated.
    """
    app = ma.app
    matching_engine = app.state.matching_engine

    # 1. Setup Client and Session Token (creates delays before we set the quote)
    client = TestClient(app)
    ma.session_manager.in_memory_sessions = {}
    token_info = ma.session_manager.create_session(10001)

    # 2. Add A-book routing rule
    async with test_db.acquire() as conn:
        await conn.execute(
            "INSERT INTO routing_rules (name, priority, match_groups, match_symbols, action, gateway_id, i
            "VALUES ('A-Book Rule', 10, '[\"*\"]', '[\"EURUSD\"]', 'TO_GATEWAY', 2, 1)"
        )

    # 3. Mock live quote (fresh, age = 0)
    matching_engine.set_quote("EURUSD", 1.0800, 1.0802)

    # 4. Intercept execution POST request from trade_center to history_server
    # Mock history_server returning execution fill
    async def mock_post(self, url, *args, **kwargs):
        class FakeResponse:
            status_code = 200
            def json(self):
                return {
                    "status": "FILLED",
                    "price_execution": 1.0850,
                    "order_id": 987654
                }
        return FakeResponse()

    monkeypatch.setattr(httpx.AsyncClient, "post", mock_post)

    # Place order
    resp = client.post(
```

```
        "/trade/order",
        json={"login": 10001, "symbol": "EURUSD", "volume": 1.0, "price_request": 1.0800, "type": 0},
        headers={"Authorization": f"Bearer {token_info['token']}"}
    )

    # Verify execution output
    assert resp.status_code == 200
    res_data = resp.json()
    assert res_data["status"] == "FILLED"
    assert float(res_data["price_execution"]) == 1.0850

    # Verify DB records
    async with test_db.acquire() as conn:
        # Check deals
        deals = await conn.fetch("SELECT * FROM deals WHERE login = 10001")
        assert len(deals) == 1
        assert float(deals[0]["price"]) == 1.0850
        assert int(deals[0]["entry"]) == 0 # Entry In

        # Check positions
        positions = await conn.fetch("SELECT * FROM positions WHERE login = 10001")
        assert len(positions) == 1
        assert float(positions[0]["price_open"]) == 1.0850

@pytest.mark.asyncio
async def test_abook_gateway_execution_failure_isolation(test_db, monkeypatch):
    """
    Assert that if LP gateway is offline or rejects, the TO_GATEWAY order fails cleanly (REJECTED),
    does NOT fallback to B-book execution, and does NOT block the order pipeline.
    """
    app = ma.app
    matching_engine = app.state.matching_engine

    async with test_db.acquire() as conn:
        await conn.execute(
            "INSERT INTO routing_rules (name, priority, match_groups, match_symbols, action, gateway_id, i
            "VALUES ('A-Book Rule', 10, '[\"*\"]', '[\"EURUSD\"]', 'TO_GATEWAY', 2, 1)"
        )
        # Add another symbol 'GBPUSD' which falls back to B-book
        await conn.execute("INSERT INTO symbols (symbol, digits, contract_size, margin_initial) VALUES ('G

    matching_engine.set_quote("EURUSD", 1.0800, 1.0802)
    matching_engine.set_quote("GBPUSD", 1.2500, 1.2502)

    # Mock LP Gateway execution to fail (connection error / HTTP status 500)
    async def mock_post_fail(self, url, *args, **kwargs):
        class FakeResponse:
            status_code = 500
            text = "LP Execution Server Timeout/Offline"
        return FakeResponse()

    monkeypatch.setattr(httpx.AsyncClient, "post", mock_post_fail)

    token_info = ma.session_manager.create_session(10001)
    client = TestClient(app)

    # 1. Place A-book order -&gt; must fail cleanly with a 400 error and transition order to REJECTED
    resp_abook = client.post(
        "/trade/order",
        json={"login": 10001, "symbol": "EURUSD", "volume": 1.0, "price_request": 1.0800, "type": 0},
        headers={"Authorization": f"Bearer {token_info['token']}"}
    )
    assert resp_abook.status_code == 400
    assert "Gateway Communication Error" in resp_abook.json()["detail"]

    # 2. Assert no deals or positions were created for EURUSD
    async with test_db.acquire() as conn:
        order_row = await conn.fetchrow("SELECT state, reason FROM orders WHERE symbol = 'EURUSD'")
```

```
        assert int(order_row["state"]) == 5 # ORDER_STATE_REJECTED (5)
        assert "Gateway HTTP error status 500" in order_row["reason"]

        deals = await conn.fetch("SELECT * FROM deals WHERE symbol = 'EURUSD'")
        assert len(deals) == 0
        positions = await conn.fetch("SELECT * FROM positions WHERE symbol = 'EURUSD'")
        assert len(positions) == 0

    # 3. Assert B-book order placed immediately after still executes instantly and successfully
    resp_bbook = client.post(
        "/trade/order",
        json={"login": 10001, "symbol": "GBPUSD", "volume": 1.0, "price_request": 1.2500, "type": 0},
        headers={"Authorization": f"Bearer {token_info['token']}"}
    )
    assert resp_bbook.status_code == 200
    assert resp_bbook.json()["status"] == "FILLED"

@pytest.mark.asyncio
async def test_stale_quote_gating(test_db):
    """
    Verify that if quote is stale (&gt; 10 seconds):
    - Placing a new market order fails with a STALE_QUOTE rejection.
    - Stop-out checks ignore accounts holding positions in the stale symbol.
    """
    app = ma.app
    matching_engine = app.state.matching_engine
    position_keeper = app.state.position_keeper
    rms_engine = app.state.rms_engine

    # 1. Setup Client and Session Token (creates delays before we set the quote to stale)
    client = TestClient(app)
    token_info = ma.session_manager.create_session(10001)

    # 2. Seed account position in EURUSD
    async with test_db.acquire() as conn:
        await conn.execute(
            "INSERT INTO positions (ticket, login, symbol, volume, action, price_open, price_current, pric
            "VALUES (60001, 10001, 'EURUSD', 10.0, 0, 1.0800, 1.0800, 0.0, 0.0, 0.0, 10800.0, 0)"
        )
        # Update account figures to simulate stop-out margin level threshold violation
        # Balance = 10000, margin = 10800, Equity = 2000 -&gt; Margin Level = 18.5% &lt;= stop-out thresho
        await conn.execute("UPDATE accounts SET balance = 10000.0, margin = 10800.0, equity = 2000.0, marg

    await position_keeper.restore_state()

    # 3. Mark EURUSD quote stale by setting quote time 15 seconds in the past
    matching_engine.set_quote("EURUSD", 1.0800, 1.0802)
    matching_engine._quote_times["EURUSD"] = time.time() - 15.0

    # 4. Attempt to place a new order -&gt; must fail with STALE_QUOTE error
    resp = client.post(
        "/trade/order",
        json={"login": 10001, "symbol": "EURUSD", "volume": 1.0, "price_request": 1.0800, "type": 0},
        headers={"Authorization": f"Bearer {token_info['token']}"}
    )
    assert resp.status_code == 400
    assert "STALE_QUOTE" in resp.json()["detail"]

    # 4. Check stop-outs -&gt; must skip account 10001 because it has a position in EURUSD which has a sta
    stop_outs = await rms_engine.check_stop_outs()
    assert 10001 not in stop_outs
```

■ main_server\tests\test_trade.py

```
import os
import pytest
import pytest_asyncio
import asyncio
from decimal import Decimal
from bases.connection import DBConnection
from shared.event_bus import InMemoryEventBus
from core.position_keeper import PositionKeeper
from matching.engine import MatchingEngine
from rms.engine import RMSEngine
from core.trade_center import TradeCenter

# Setup database environment for testing
DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://postgres:postgres@localhost:5432/broker_db")

@pytest.fixture(scope="module")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()

@pytest_asyncio.fixture(scope="module")
async def db_setup():
    os.environ["DATABASE_URL"] = DATABASE_URL
    await DBConnection.initialize()

    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        # Load correct schema based on database type
        if DBConnection.is_sqlite:
            schema_name = "schema_sqlite.sql"
        else:
            schema_name = "schema.sql"

        schema_path = os.path.join(os.path.dirname(__file__), "..", "bases", schema_name)
        with open(schema_path, "r") as f:
            schema_sql = f.read()

        if DBConnection.is_sqlite:
            # SQLite does not support execute() running multiple semicolon-split statements in one call
            # unless we execute it script-style on the raw connection
            conn._conn.executescript(schema_sql)
        else:
            await conn.execute(schema_sql)

        # Insert test config
        await conn.execute(
            "INSERT INTO groups (name, max_leverage, margin_call, margin_stop_out) VALUES ($1, $2, $3, $4)
            "test_group", 100, Decimal("50.00"), Decimal("30.00")
        )
        await conn.execute(
            "INSERT INTO symbols (symbol, digits, contract_size, currency, margin_initial) VALUES ($1, $2,
            "EURUSD", 5, Decimal("100000.0"), "USD", Decimal("1.0")
        )
        await conn.execute(
            "INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin) VALUES ($1,
            20001, "test_group", Decimal("10000.0"), 100, Decimal("10000.0"), Decimal("10000.0")
        )

    yield pool
    await DBConnection.close()

@pytest.mark.asyncio
async def test_order_execution_and_position_close(db_setup):
    pool = db_setup
```

```
# Initialize components
event_bus = InMemoryEventBus()
position_keeper = PositionKeeper()
matching_engine = MatchingEngine()
rms_engine = RMSEngine(position_keeper, matching_engine)
trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)

# Pre-populate quotes
matching_engine.set_quote("EURUSD", 1.08000, 1.08020)

# Restore keeper (starts empty)
await position_keeper.restore_state()
assert len(await position_keeper.get_all_positions()) == 0

# 1. Place BUY Order: 1 lot EURUSD (100,000 contracts)
# 0: OP_BUY, matches at Ask (1.08020)
order_req = {
    "login": 20001,
    "symbol": "EURUSD",
    "volume": Decimal("1.0"),
    "price_request": Decimal("1.08020"),
    "type": 0
}

res = await trade_center.place_order(order_req)
assert res["status"] == "FILLED"
assert res["price_execution"] == Decimal("1.08020")
assert res["entry"] == 0 # ENTRY_IN

# Check Deal logged
async with pool.acquire() as conn:
    deal = await conn.fetchrow("SELECT * FROM deals WHERE login = 20001 AND order_ticket = $1", res["t
    assert deal is not None
    assert Decimal(str(deal["price"])) == Decimal("1.08020")
    assert deal["action"] == 0 # DEAL_BUY
    assert deal["entry"] == 0 # ENTRY_IN

    # Check Account state
    acc = await conn.fetchrow("SELECT * FROM accounts WHERE login = 20001")
    # Balance shouldn't change yet on position open (no commissions set)
    assert Decimal(str(acc["balance"])) == Decimal("10000.0")
    # Margin required = 1 * 100000 * 1.08020 / 100 = 1080.20
    assert Decimal(str(acc["margin"])) == Decimal("1080.20")
    # Equity = 10000 + P&amp;amp;L. Ask is 1.08020, bid is 1.08000.
    # Position evaluates at bid (1.08000).
    # P&amp;amp;L = (1.08000 - 1.08020) * 100000 * 1 = -20.00
    # Equity = 9980.00
    assert Decimal(str(acc["equity"])) == Decimal("9980.0")
    assert Decimal(str(acc["free_margin"])) == Decimal("8899.8")

# Verify position is in memory
positions = await position_keeper.get_positions(20001)
assert len(positions) == 1
pos = positions[0]
assert pos["volume"] == Decimal("1.0")
assert pos["price_open"] == Decimal("1.08020")
assert pos["action"] == 0 # POSITION_BUY

# 2. Place SELL Order to CLOSE the position
# 1: OP_SELL, matches at Bid (1.08120)
matching_engine.set_quote("EURUSD", 1.08120, 1.08140)

order_close_req = {
    "login": 20001,
    "symbol": "EURUSD",
    "volume": Decimal("1.0"),
    "price_request": Decimal("1.08120"),
    "type": 1 # OP_SELL (opposite closes BUY)
```

```
}

close_res = await trade_center.place_order(order_close_req)
assert close_res["status"] == "FILLED"
assert close_res["price_execution"] == Decimal("1.08120")
assert close_res["entry"] == 1 # ENTRY_OUT

# Realized Profit = (1.08120 - 1.08020) * 100000 * 1 = +100.00
assert close_res["profit"] == Decimal("100.0")

# Verify balance is updated in DB
async with pool.acquire() as conn:
    acc_closed = await conn.fetchrow("SELECT * FROM accounts WHERE login = 20001")
    # Balance = 10000 + profit (100) = 10100.00
    assert Decimal(str(acc_closed["balance"])) == Decimal("10100.0")
    assert Decimal(str(acc_closed["margin"])) == Decimal("0.0")
    assert Decimal(str(acc_closed["equity"])) == Decimal("10100.0")

# Verify position is removed
assert len(await position_keeper.get_positions(20001)) == 0
```

■ main_server\webapi\manager_api.py

```
import logging
import asyncio
import redis
import json
import time
import secrets
import hashlib
from datetime import datetime, timedelta
from fastapi import FastAPI, HTTPException, Request, WebSocket, WebSocketDisconnect, Depends, Query
from fastapi.responses import ORJSONResponse
from typing import Dict, Any, List, Optional
from decimal import Decimal
from pydantic import BaseModel

# Setup uvloop for maximum event-loop speed
try:
    import uvloop
    asyncio.set_event_loop_policy(uvloop.EventLoopPolicy())
except ImportError:
    pass

from bases.connection import DBConnection
from shared.schemas.v1 import OrderRequest, OrderResponse, AccountInfo
from shared.event_bus import InMemoryEventBus
from core.position_keeper import PositionKeeper
from matching.engine import MatchingEngine
from rms.engine import RMSEngine
from core.trade_center import TradeCenter
from core.tick_center import TickCenter
from core.tick_publisher import TickPublisher

# Setup Logging
import os
logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(name)s - %(levelname)s - %(message)s")
logger = logging.getLogger("api.manager_api")

def patch_ini_files(old_name: str, new_name: str, is_symbol: bool, is_gateway: bool = False):
    config_dir = r"C:\Users\DELL\Downloads\server3\1MT5\main1\config"
    if not os.path.exists(config_dir):
        logger.warning(f"MT5 config directory not found at {config_dir}. Skipping .ini patching.")
        return

    old_bytes = old_name.encode("utf-16le")
    new_bytes = new_name.encode("utf-16le")

    # Search for null terminated names to prevent prefix matches
    old_bytes_search = old_bytes + b"\x00\x00"

    if is_symbol:
        configs = [
            ("symbols.ini", 64),
            ("groups.ini", 256),
            ("groups.ini", 128),
            ("groups.ini", 64)
        ]
    elif is_gateway:
        configs = [
            ("gateways.ini", 64),
            ("gateways.ini", 128),
            ("feeders.ini", 64),
            ("feeders.ini", 128)
        ]
    else:
        configs = [
            ("groups.ini", 128)
```

```
        ]

    for filename, field_size in configs:
        file_path = os.path.join(config_dir, filename)
        if not os.path.exists(file_path):
            continue
        try:
            with open(file_path, "rb") as f:
                data = bytearray(f.read())

            idx = 0
            replaced = False
            while True:
                idx = data.find(old_bytes_search, idx)
                if idx == -1:
                    break

                # Check character boundaries before and after to ensure whole-word matching
                is_boundary = True
                if idx &gt;= 2:
                    char_before = data[idx-2:idx].decode("utf-16le", errors="ignore")
                    if char_before.isalnum() or char_before in "\\_-./":
                        is_boundary = False
                if is_boundary and idx + len(old_bytes_search) &lt;= len(data):
                    char_after = data[idx+len(old_bytes) : idx+len(old_bytes)+2].decode("utf-16le", errors
                    if char_after.isalnum() or char_after in "\\_-./":
                        is_boundary = False

                if not is_boundary:
                    idx += len(old_bytes_search)
                    continue

                # Perform size-invariant overwrite
                overwrite_len = max(len(old_bytes), len(new_bytes)) + 2
                new_field = new_bytes + b"\x00" * (overwrite_len - len(new_bytes))

                data[idx : idx + overwrite_len] = new_field
                logger.info(f"Patched {filename} at index {idx} for rename {old_name} -&gt; {new_name}")
                idx += overwrite_len
                replaced = True

            if replaced:
                with open(file_path, "wb") as f:
                    f.write(data)
                logger.info(f"Saved patched file {file_path}")
        except Exception as e:
            logger.error(f"Error patching ini file {filename}: {e}", exc_info=True)

INTERNAL_SECRET = os.getenv("INTERNAL_SECRET", "default_internal_secret_token_change_in_production")
if INTERNAL_SECRET == "default_internal_secret_token_change_in_production":
    logger.warning("\n" + "="*80 + "\n" +
                   "  ■■  WARNING: DEFAULT INSECURE INTERNAL_SECRET IS ACTIVE! ■■\n" +
                   "  Please set the INTERNAL_SECRET environment variable in production.\n" +
                   "="*80 + "\n")

ADMIN_API_KEY = os.getenv("ADMIN_API_KEY", "default_admin_api_key_token_change_in_production")
if ADMIN_API_KEY == "default_admin_api_key_token_change_in_production":
    logger.warning("\n" + "="*80 + "\n" +
                   "  ■■  WARNING: DEFAULT INSECURE ADMIN_API_KEY IS ACTIVE! ■■\n" +
                   "  Please set the ADMIN_API_KEY environment variable in production.\n" +
                   "="*80 + "\n")

async def verify_admin(request: Request) -&gt; bool:
    admin_key = request.headers.get("X-Admin-API-Key")
    if not admin_key or not secrets.compare_digest(admin_key, ADMIN_API_KEY):
        raise HTTPException(status_code=403, detail="Forbidden: Invalid or missing Admin API Key.")
    return True
```

```
PBKDF2_ITERATIONS = 600000

def hash_password(password: str) -&gt; str:
    salt = secrets.token_bytes(16)
    pwd_hash = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, PBKDF2_ITERATIONS)
    return f"{salt.hex()}:{pwd_hash.hex()}"

def verify_password(stored_hash: str, password: str) -&gt; bool:
    try:
        if not stored_hash:
            return False
        if ":" not in stored_hash:
            expected = hashlib.sha256(password.encode("utf-8")).hexdigest()
            return secrets.compare_digest(stored_hash, expected)
        salt_hex, hash_hex = stored_hash.split(":")
        salt = bytes.fromhex(salt_hex)
        expected = bytes.fromhex(hash_hex)
        actual = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, PBKDF2_ITERATIONS)
        return secrets.compare_digest(actual, expected)
    except Exception:
        return False

class SessionManager:
    def __init__(self, redis_host="localhost", redis_port=6379, use_redis=True):
        self.use_redis = use_redis
        self.r = None
        self.in_memory_sessions = {} # fallback: token -&gt; {login, expires_at}
        self.last_redis_failure = 0.0
        self.retry_cooldown = 10.0 # seconds
        if self.use_redis:
            self.r = redis.Redis(
                host=redis_host,
                port=redis_port,
                decode_responses=True,
                socket_connect_timeout=0.5,
                socket_timeout=0.5
            )

    def create_session(self, login: int, expires_in_seconds=3600) -&gt; Dict[str, Any]:
        token = secrets.token_hex(32)
        expires_at = (datetime.utcnow() + timedelta(seconds=expires_in_seconds)).isoformat()
        session_data = {"login": login, "expires_at": expires_at}

        # Always write to hot-standby in-memory dict
        self.in_memory_sessions[token] = session_data

        # Write to Redis if not in cooldown period
        if self.r and (time.time() - self.last_redis_failure &gt; self.retry_cooldown):
            try:
                self.r.set(f"session:{token}", json.dumps(session_data), ex=expires_in_seconds)
            except Exception as e:
                logger.error(f"Failed to set session in Redis: {e}. Entering Redis cooldown.")
                self.last_redis_failure = time.time()
        return {"token": token, "expires_at": expires_at}

    def validate_session(self, token: str) -&gt; int:
        session_data = None
        # Try Redis if not in cooldown
        if self.r and (time.time() - self.last_redis_failure &gt; self.retry_cooldown):
            try:
                data = self.r.get(f"session:{token}")
                if data:
                    session_data = json.loads(data)
            except Exception as e:
                logger.error(f"Failed to get session from Redis: {e}. Entering Redis cooldown.")
                self.last_redis_failure = time.time()

        if not session_data:
```

```
            session_data = self.in_memory_sessions.get(token)

        if session_data:
            expires_at = datetime.fromisoformat(session_data["expires_at"])
            if datetime.utcnow() &lt; expires_at:
                return session_data["login"]
            else:
                self.delete_session(token)
        return None

    def delete_session(self, token: str):
        # Try delete from Redis if not in cooldown
        if self.r and (time.time() - self.last_redis_failure &gt; self.retry_cooldown):
            try:
                self.r.delete(f"session:{token}")
            except Exception as e:
                logger.error(f"Failed to delete session from Redis: {e}. Entering Redis cooldown.")
                self.last_redis_failure = time.time()
        if token in self.in_memory_sessions:
            del self.in_memory_sessions[token]

# Initialize global components
event_bus = InMemoryEventBus()
position_keeper = PositionKeeper()
matching_engine = MatchingEngine()
rms_engine = RMSEngine(position_keeper, matching_engine)
trade_center = TradeCenter(rms_engine, matching_engine, position_keeper, event_bus)
tick_publisher = TickPublisher()
tick_center = TickCenter(trade_center, position_keeper, rms_engine, matching_engine, event_bus, tick_publi
session_manager = SessionManager()

class LoginRequest(BaseModel):
    login: int
    password: str
    server: str

class AdminAccountCreate(BaseModel):
    login: Optional[int] = None
    group_name: str
    initial_balance: float
    leverage: int = 100
    password: str
    name: Optional[str] = ""
    last_name: Optional[str] = ""
    middle_name: Optional[str] = ""
    company: Optional[str] = ""
    email: Optional[str] = ""
    phone: Optional[str] = ""
    country: Optional[str] = ""
    state: Optional[str] = ""
    city: Optional[str] = ""
    zip_code: Optional[str] = ""
    address: Optional[str] = ""
    investor_password: Optional[str] = ""
    phone_password: Optional[str] = ""

class AdminAccountUpdate(BaseModel):
    group_name: str
    leverage: int
    status: int
    settings_json: dict

class AdminSymbolCreate(BaseModel):
    symbol: str
    digits: int = 5
    contract_size: float = 100000.0
    currency: str = "USD"
    margin_initial: float = 1.0
```

```
    margin_maintenance: float = 1.0
    spread_base: int = 10
    session_hours: Optional[str] = "MON,00:00-23:59;TUE,00:00-23:59;WED,00:00-23:59;THU,00:00-23:59;FRI,00
    settings_json: Optional[str] = "{}"

class AdminGroupCreate(BaseModel):
    name: str
    max_leverage: int = 100
    margin_call: float = 50.0
    margin_stop_out: float = 30.0
    spread_override: int = 0
    settings_json: Optional[str] = "{}"

class AdminGroupSymbolOverride(BaseModel):
    symbol: str
    spread_diff: int = 0
    commission_rate: float = 0.0
    margin_rate: float = 1.0
    trade_allowed: bool = True

class AdminRouteRuleCreate(BaseModel):
    name: str
    priority: int
    is_enabled: bool = True
    match_groups: Optional[List[str]] = None
    match_symbols: Optional[List[str]] = None
    match_accounts: Optional[List[str]] = None
    match_order_types: Optional[List[str]] = None
    match_volume_min: Optional[float] = None
    match_volume_max: Optional[float] = None
    action: str
    gateway_id: Optional[int] = None
    delay_seconds: int = 0

class AdminGatewayCreate(BaseModel):
    name: str
    type: str
    host: Optional[str] = None
    port: Optional[int] = None
    username: Optional[str] = None
    api_key: Optional[str] = None
    is_active: bool = True
    settings_json: Optional[str] = "{}"

async def get_current_login(request: Request) -&gt; int:
    auth_header = request.headers.get("Authorization")
    if not auth_header or not auth_header.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Missing or invalid token.")
    token = auth_header.split(" ")[1]
    login = session_manager.validate_session(token)
    if not login:
        raise HTTPException(status_code=401, detail="Session expired or invalid.")
    return login

app = FastAPI(
    title="Broker Server Phase 3"
)

from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

app.state.matching_engine = matching_engine
app.state.rms_engine = rms_engine
```

```
app.state.trade_center = trade_center
app.state.tick_center = tick_center
app.state.position_keeper = position_keeper

@app.on_event("startup")
async def startup_event():
    logger.info("Initializing database pool...")
    await DBConnection.initialize()
    # Add match_accounts column if not present in schema
    try:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await conn.execute("ALTER TABLE routing_rules ADD COLUMN match_accounts TEXT")
    except Exception:
        pass
    # Add assigned_dealer column to orders table if not present in schema
    try:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await conn.execute("ALTER TABLE orders ADD COLUMN assigned_dealer INTEGER")
    except Exception:
        pass
    # Add settings_json column to groups table if not present in schema
    try:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await conn.execute("ALTER TABLE groups ADD COLUMN settings_json TEXT")
    except Exception:
        pass
    # Add settings_json column to symbols table if not present in schema
    try:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await conn.execute("ALTER TABLE symbols ADD COLUMN settings_json TEXT")
    except Exception:
        pass
    # Add settings_json column to gateways table if not present in schema
    try:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await conn.execute("ALTER TABLE gateways ADD COLUMN settings_json TEXT")
    except Exception:
        pass
    # Add settings_json column to accounts table if not present in schema
    try:
        pool = DBConnection.get_pool()
        async with pool.acquire() as conn:
            await conn.execute("ALTER TABLE accounts ADD COLUMN settings_json TEXT")
    except Exception as e:
        logger.error(f"Failed to patch accounts settings_json: {e}")
    logger.info("Restoring PositionKeeper state...")
    await position_keeper.restore_state()
    logger.info("Running startup order cleanup...")
    await trade_center.cleanup_orders_on_startup()
    logger.info("Starting TickPublisher...")
    tick_publisher.start()
    logger.info("Server started successfully.")

@app.on_event("shutdown")
async def shutdown_event():
    logger.info("Stopping TickPublisher...")
    await tick_publisher.stop()
    logger.info("Closing database pool...")
    await DBConnection.close()
    logger.info("Closing Redis session connection pool...")
    if session_manager.r:
        session_manager.r.close()
    logger.info("Server stopped.")
```

```
@app.post("/web/auth/login")
async def web_auth_login(request: LoginRequest):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        row = await conn.fetchrow(
            "SELECT login, password_hash FROM accounts WHERE login = $1",
            request.login
        )
        if not row:
            raise HTTPException(status_code=401, detail="Invalid login or password.")

        stored_hash = row["password_hash"]
        if not verify_password(stored_hash, request.password):
            raise HTTPException(status_code=401, detail="Invalid login or password.")

        session_info = session_manager.create_session(request.login)
        return {
            "token": session_info["token"],
            "account_id": request.login,
            "expires_at": session_info["expires_at"]
        }

@app.post("/web/auth/logout")
async def web_auth_logout(request: Request):
    auth_header = request.headers.get("Authorization")
    if auth_header and auth_header.startswith("Bearer "):
        token = auth_header.split(" ")[1]
        session_manager.delete_session(token)
    return {"status": "SUCCESS", "message": "Logged out successfully."}

@app.get("/internal/auth/validate")
async def internal_auth_validate(request: Request, token: str = Query(...)):
    secret = request.headers.get("X-Internal-Secret")
    if secret != INTERNAL_SECRET:
        raise HTTPException(status_code=403, detail="Forbidden: Invalid internal secret.")
    login = session_manager.validate_session(token)
    if not login:
        raise HTTPException(status_code=401, detail="Session invalid or expired.")
    return {"login": login}

class InternalTickPayload(BaseModel):
    symbol: str
    bid: float
    ask: float
    source: Optional[str] = "INTERNAL"

feeder_stats = {}

@app.get("/admin/ticks", tags=["admin"])
async def admin_get_ticks(admin_auth: bool = Depends(verify_admin)):
    result = {}
    for sym, val in matching_engine._quotes.items():
        bid, ask = val
        result[sym] = {
            "bid": float(bid),
            "ask": float(ask),
            "age": matching_engine.get_quote_age(sym)
        }
    return result

def match_and_translate_symbol(incoming_symbol: str, source_pattern: str, target_pattern: str) -&gt; Optic
    if source_pattern == "*":
        if target_pattern == "*":
            return incoming_symbol
        elif "*" in target_pattern:
            return target_pattern.replace("*", incoming_symbol)
        else:
            return target_pattern
```

```
    if "*" in source_pattern:
        try:
            prefix, suffix = source_pattern.split("*", 1)
        except ValueError:
            return None
        if incoming_symbol.startswith(prefix) and incoming_symbol.endswith(suffix) and len(incoming_symbol
            captured = incoming_symbol[len(prefix) : len(incoming_symbol) - len(suffix)]
            if "*" in target_pattern:
                return target_pattern.replace("*", captured)
            else:
                return target_pattern
        return None

    if incoming_symbol == source_pattern:
        return target_pattern

    return None

async def resolve_and_translate_tick(source_symbol: str, bid: float, ask: float) -&gt; bool:
    pool = DBConnection.get_pool()
    matched_any = False
    async with pool.acquire() as conn:
        # Get all active gateways
        gateways = await conn.fetch("SELECT settings_json FROM gateways WHERE is_active = 1 ORDER BY id AS

        for g in gateways:
            settings_json = g["settings_json"]
            if not settings_json:
                continue
            try:
                settings = json.loads(settings_json)
                translations = settings.get("translations", [])
            except Exception:
                continue

            for rule in translations:
                target_pat = rule.get("symbol")
                source_pat = rule.get("source")
                bid_adj = rule.get("bid_adj", 0)
                ask_adj = rule.get("ask_adj", 0)

                if not target_pat or not source_pat:
                    continue

                local_symbol = match_and_translate_symbol(source_symbol, source_pat, target_pat)
                if local_symbol:
                    # Verify target symbol exists on platform to get digits (support direct match or folde
                    sym_rows = await conn.fetch("SELECT symbol, digits FROM symbols WHERE symbol = $1 OR s
                    for sym_row in sym_rows:
                        actual_local_symbol = sym_row["symbol"]
                        digits = sym_row["digits"]
                        point = 1.0 / (10 ** digits)
                        adjusted_bid = bid + float(bid_adj) * point
                        adjusted_ask = ask + float(ask_adj) * point

                        await tick_center.on_tick(actual_local_symbol, adjusted_bid, adjusted_ask)
                        logger.info(f"Translated tick {source_symbol} -&gt; {actual_local_symbol}: {bid}/{
                        matched_any = True

        # Fallback to direct matching (check exact match or folder prefixed match)
        sym_rows = await conn.fetch("SELECT symbol FROM symbols WHERE symbol = $1 OR symbol LIKE '%\\' ||
        for sym_row in sym_rows:
            matched_symbol = sym_row["symbol"]
            await tick_center.on_tick(matched_symbol, bid, ask)
            matched_any = True

        if not matched_any:
            logger.warning(f"Ignored quote tick for unknown symbol: {source_symbol}")
```

```
        return matched_any

@app.post("/internal/ticks")
async def internal_ticks_ingest(request: Request, payload: InternalTickPayload):
    secret = request.headers.get("X-Internal-Secret")
    if not secret or not secrets.compare_digest(secret, INTERNAL_SECRET):
        raise HTTPException(status_code=403, detail="Forbidden: Invalid or missing X-Internal-Secret")
    try:
        import time as _time
        source = payload.source or "INTERNAL"
        if source not in feeder_stats:
            feeder_stats[source] = {"ticks_count": 0, "last_active": None, "last_active_ts": None}
        feeder_stats[source]["ticks_count"] += 1
        feeder_stats[source]["last_active"] = datetime.utcnow().isoformat()
        feeder_stats[source]["last_active_ts"] = _time.time()

        await resolve_and_translate_tick(payload.symbol, payload.bid, payload.ask)
        return {"status": "success"}
    except Exception as e:
        logger.error(f"Error executing internal tick update: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@app.post("/trade/order", response_model=OrderResponse)
async def create_order(request: OrderRequest, current_login: int = Depends(get_current_login)):
    """Executes a market trade order, protected by defense-in-depth login validation."""
    if request.login != current_login:
        raise HTTPException(status_code=403, detail="Access denied. Token login mismatch.")
    try:
        # Convert Request model to dictionary
        order_dict = {
            "login": request.login,
            "symbol": request.symbol,
            "volume": request.volume,
            "price_request": request.price_request,
            "type": request.type,
            "price_sl": request.price_sl,
            "price_tp": request.price_tp,
            "type_filling": request.type_filling
        }
        result = await trade_center.place_order(order_dict)

        # Check if the execution resulted in a rejection
        if result["status"] == "REJECTED":
            raise HTTPException(status_code=400, detail=result["message"])

        return result
    except HTTPException as http_exc:
        raise http_exc
    except ValueError as val_err:
        logger.error(f"Validation Error: {val_err}")
        raise HTTPException(status_code=400, detail=str(val_err))
    except Exception as e:
        logger.error(f"Unhandled error placing order: {e}")
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/accounts/{login}", response_model=AccountInfo)
async def get_account(login: int, current_login: int = Depends(get_current_login)):
    """Retrieves account balance, leverage, equity and margin metrics, protected by defense-in-depth check
    if login != current_login:
        raise HTTPException(status_code=403, detail="Access denied. Token login mismatch.")
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        row = await conn.fetchrow(
            "SELECT login, balance, credit, equity, margin, free_margin, margin_level FROM accounts WHERE
            login
        )
        if not row:
            raise HTTPException(status_code=404, detail="Account not found.")
```

```
        # Convert DB types to Decimal for Pydantic schema validation
        return {
            "login": row["login"],
            "balance": Decimal(str(row["balance"])),
            "credit": Decimal(str(row["credit"])),
            "equity": Decimal(str(row["equity"])),
            "margin": Decimal(str(row["margin"])),
            "free_margin": Decimal(str(row["free_margin"])),
            "margin_level": Decimal(str(row["margin_level"]))
        }

@app.get("/web/accounts/{login}/positions")
async def web_get_positions(login: int, current_login: int = Depends(get_current_login)):
    """Exposes current active positions, protected by defense-in-depth check."""
    if login != current_login:
        raise HTTPException(status_code=403, detail="Access denied. Token login mismatch.")
    # Return serializable floats
    raw_positions = await position_keeper.get_positions(login)
    serializable = []
    for pos in raw_positions:
        s_pos = dict(pos)
        s_pos["volume"] = float(pos["volume"])
        s_pos["price_open"] = float(pos["price_open"])
        s_pos["price_current"] = float(pos["price_current"])
        s_pos["price_sl"] = float(pos["price_sl"])
        s_pos["price_tp"] = float(pos["price_tp"])
        s_pos["profit"] = float(pos["profit"])
        s_pos["margin"] = float(pos["margin"])
        s_pos["storage"] = float(pos["storage"])
        serializable.append(s_pos)
    return serializable

@app.get("/web/accounts/{login}/orders")
async def web_get_orders(login: int, current_login: int = Depends(get_current_login)):
    """Exposes current pending orders, protected by defense-in-depth check."""
    if login != current_login:
        raise HTTPException(status_code=403, detail="Access denied. Token login mismatch.")
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch(
            "SELECT ticket, login, symbol, volume, volume_current, price_order, price_sl, price_tp, type,
            login
        )
        serializable = []
        for r in rows:
            d = dict(r)
            d["volume"] = float(r["volume"])
            d["volume_current"] = float(r["volume_current"])
            d["price_order"] = float(r["price_order"])
            d["price_sl"] = float(r["price_sl"])
            d["price_tp"] = float(r["price_tp"])
            serializable.append(d)
        return serializable

@app.get("/web/accounts/{login}/history")
async def web_get_history(login: int, current_login: int = Depends(get_current_login)):
    """Exposes client execution deals history, protected by defense-in-depth check."""
    if login != current_login:
        raise HTTPException(status_code=403, detail="Access denied. Token login mismatch.")
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch(
            "SELECT ticket, order_ticket, login, symbol, volume, price, commission, storage, profit, actio
            login
        )
        serializable = []
        for r in rows:
            d = dict(r)
```

```
            d["volume"] = float(r["volume"])
            d["price"] = float(r["price"])
            d["commission"] = float(r["commission"])
            d["storage"] = float(r["storage"])
            d["profit"] = float(r["profit"])
            serializable.append(d)
        return serializable

@app.websocket("/ws/stream/{account_id}")
async def ws_stream(websocket: WebSocket, account_id: str):
    """WebSocket stream for real-time price and position pushes, validating token on upgrade."""
    token = websocket.query_params.get("token")
    if not token:
        # Check Authorization header if query parameter not found
        auth_header = websocket.headers.get("Authorization")
        if auth_header and auth_header.startswith("Bearer "):
            token = auth_header.split(" ")[1]

    if not token:
        await websocket.close(code=4001, reason="Missing token")
        return

    login = session_manager.validate_session(token)
    try:
        acc_id_int = int(account_id)
    except ValueError:
        await websocket.close(code=4002, reason="Invalid account_id")
        return

    if not login or login != acc_id_int:
        await websocket.close(code=4003, reason="Unauthorized")
        return

    await websocket.accept()

    # Define a queue to stream updates safely
    queue = asyncio.Queue()

    # Callback definitions
    async def on_tick_event(payload):
        await queue.put({"type": "tick", "data": payload})

    async def on_position_event(payload):
        if payload.get("login") == login:
            await queue.put({"type": "position", "data": payload})

    # Subscribe to event bus
    await event_bus.subscribe("ticks", on_tick_event)
    await event_bus.subscribe("positions.updated", on_position_event)

    # Start sender task
    async def sender():
        try:
            while True:
                msg = await queue.get()
                await websocket.send_json(msg)
        except Exception:
            pass

    sender_task = asyncio.create_task(sender())

    # Keep WebSocket open and listen for close
    try:
        while True:
            # We don't expect incoming messages from client, but keeping connection alive
            await websocket.receive_text()
    except WebSocketDisconnect:
        pass
```

```
    finally:
        sender_task.cancel()
        try:
            await sender_task
        except asyncio.CancelledError:
            pass
        # Unsubscribe to prevent memory leak
        await event_bus.unsubscribe("ticks", on_tick_event)
        await event_bus.unsubscribe("positions.updated", on_position_event)

def serialize_db_row(row):
    if not row:
        return None
    d = dict(row)
    for k, v in d.items():
        if isinstance(v, Decimal):
            d[k] = float(v)
        elif isinstance(v, datetime):
            d[k] = v.isoformat()
        elif k == "settings_json" and isinstance(v, str) and v:
            if "login" in d:
                try:
                    d[k] = json.loads(v)
                except Exception:
                    pass
    return d

@app.post("/admin/setup", tags=["admin"])
async def setup_environment(admin_auth: bool = Depends(verify_admin)):
    """Sets up a default group, symbol, and demo account with password hashed for auth validation."""
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        async with conn.transaction():
            # Setup default group
            await conn.execute(
                """INSERT INTO groups (name, max_leverage, margin_call, margin_stop_out)
                   VALUES ('demo_group', 100, 50.00, 30.00)
                   ON CONFLICT (name) DO UPDATE SET max_leverage=100"""
            )

            # Setup default EURUSD symbol
            await conn.execute(
                """INSERT INTO symbols (symbol, digits, contract_size, currency, margin_initial)
                   VALUES ('EURUSD', 5, 100000.0, 'USD', 1.0)
                   ON CONFLICT (symbol) DO UPDATE SET contract_size=100000.0"""
            )

            # Hash the default password 'password123'
            pwd_hash = hash_password("password123")

            # Setup default test client account (login 10001, balance 10000)
            await conn.execute(
                """INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin, passwo
                   VALUES (10001, 'demo_group', 10000.00000000, 100, 10000.00000000, 10000.00000000, $1)
                   ON CONFLICT (login) DO UPDATE SET balance=10000.00000000, equity=10000.00000000, free_m
                pwd_hash, pwd_hash
            )

    return {"status": "SUCCESS", "message": "Demo group, EURUSD symbol, and account 10001 created."}

# ==================== ADMIN: ACCOUNTS ====================

@app.post("/admin/accounts", tags=["admin"])
async def admin_create_account(req: AdminAccountCreate, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        grp = await conn.fetchrow("SELECT name FROM groups WHERE name = $1", req.group_name)
        if not grp:
```

```
            raise HTTPException(status_code=400, detail=f"Group '{req.group_name}' does not exist.")

        login = req.login
        if login is None:
            max_login = await conn.fetchval("SELECT MAX(login) FROM accounts")
            if max_login is None or max_login &lt; 10000:
                login = 10002
            else:
                login = max_login + 1
        else:
            exist = await conn.fetchrow("SELECT login FROM accounts WHERE login = $1", login)
            if exist:
                raise HTTPException(status_code=400, detail=f"Account with login {login} already exists.")

        pwd_hash = hash_password(req.password)
        balance = Decimal(str(req.initial_balance))
        settings_dict = {
            "name": req.name,
            "last_name": req.last_name,
            "middle_name": req.middle_name,
            "company": req.company,
            "email": req.email,
            "phone": req.phone,
            "country": req.country,
            "state": req.state,
            "city": req.city,
            "zip_code": req.zip_code,
            "address": req.address,
            "investor_password": req.investor_password,
            "phone_password": req.phone_password,
            "registered": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        }
        await conn.execute(
            """INSERT INTO accounts (login, group_name, balance, leverage, equity, free_margin, password_h
               VALUES ($1, $2, $3, $4, $5, $6, $7, $8)""",
            login, req.group_name, balance, req.leverage, balance, balance, pwd_hash, json.dumps(settings_
        )
        return {"status": "SUCCESS", "login": login, "group_name": req.group_name, "balance": float(balanc

@app.get("/admin/accounts", tags=["admin"])
async def admin_list_accounts(admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT login, group_name, balance, leverage, equity, free_margin, status,
        return [serialize_db_row(r) for r in rows]

class AdminOrderRequest(BaseModel):
    login: int
    symbol: str
    volume: float
    price_request: float
    type: int
    price_sl: float = 0.0
    price_tp: float = 0.0
    type_filling: Optional[str] = "FOK"

@app.post("/admin/trade/order", tags=["admin"])
async def admin_create_order(req: AdminOrderRequest, admin_auth: bool = Depends(verify_admin)):
    try:
        order_dict = {
            "login": req.login,
            "symbol": req.symbol,
            "volume": req.volume,
            "price_request": req.price_request,
            "type": req.type,
            "price_sl": req.price_sl,
            "price_tp": req.price_tp,
            "type_filling": req.type_filling
```

```
        }
        result = await trade_center.place_order(order_dict)
        if result["status"] == "REJECTED":
            raise HTTPException(status_code=400, detail=result["message"])
        return result
    except ValueError as val_err:
        raise HTTPException(status_code=400, detail=str(val_err))
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/admin/positions", tags=["admin"])
async def admin_list_positions(admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT login, symbol, volume, action AS direction, price_open, price_sl,
        return [serialize_db_row(r) for r in rows]

@app.get("/admin/deals", tags=["admin"])
async def admin_list_deals(admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT ticket, order_ticket, login, symbol, volume, price, commission, st
        return [serialize_db_row(r) for r in rows]

@app.get("/admin/orders", tags=["admin"])
async def admin_list_orders(admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT ticket, login, symbol, volume, volume_current, price_order, price_
        return [serialize_db_row(r) for r in rows]

@app.get("/admin/orders/history", tags=["admin"])
async def admin_list_orders_history(admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT ticket, login, symbol, volume, volume_current, price_order, price_
        return [serialize_db_row(r) for r in rows]

@app.post("/admin/orders/{ticket}/cancel", tags=["admin"])
async def admin_cancel_order(ticket: int, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        exist = await conn.fetchrow("SELECT ticket, state FROM orders WHERE ticket = $1", ticket)
        if not exist:
            raise HTTPException(status_code=404, detail="Order not found.")
        if exist["state"] not in (0, 1, 6, 7):
            raise HTTPException(status_code=400, detail="Order is already completed or cancelled.")
        await conn.execute("UPDATE orders SET state = 5, time_done = datetime('now') WHERE ticket = $1", t
        return {"status": "SUCCESS", "message": f"Order {ticket} cancelled."}

@app.get("/admin/accounts/{login}", tags=["admin"])
async def admin_get_account(login: int, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        row = await conn.fetchrow("SELECT login, group_name, balance, leverage, equity, free_margin, statu
        if not row:
            raise HTTPException(status_code=404, detail="Account not found.")
        return serialize_db_row(row)

@app.put("/admin/accounts/{login}", tags=["admin"])
async def admin_update_account(login: int, req: AdminAccountUpdate, admin_auth: bool = Depends(verify_admi
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        exist = await conn.fetchrow("SELECT login FROM accounts WHERE login = $1", login)
        if not exist:
            raise HTTPException(status_code=404, detail=f"Account with login {login} not found.")

        grp = await conn.fetchrow("SELECT name FROM groups WHERE name = $1", req.group_name)
```

```
        if not grp:
            raise HTTPException(status_code=400, detail=f"Group '{req.group_name}' does not exist.")

        await conn.execute(
            """UPDATE accounts
               SET group_name = $1, leverage = $2, status = $3, settings_json = $4
               WHERE login = $5""",
            req.group_name, req.leverage, req.status, json.dumps(req.settings_json), login
        )
        return {"status": "SUCCESS", "login": login}

@app.delete("/admin/accounts/{login}", tags=["admin"])
async def admin_delete_account(login: int, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        exist = await conn.fetchrow("SELECT login FROM accounts WHERE login = $1", login)
        if not exist:
            raise HTTPException(status_code=404, detail=f"Account with login {login} not found.")

        await conn.execute("DELETE FROM accounts WHERE login = $1", login)
        return {"status": "SUCCESS", "message": f"Account {login} deleted successfully."}

# ==================== ADMIN: SYMBOLS ====================

@app.get("/admin/symbols", tags=["admin"])
async def admin_list_symbols(admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT symbol, digits, contract_size, currency, margin_initial, margin_ma
        return [serialize_db_row(r) for r in rows]

@app.get("/admin/symbols/{symbol:path}", tags=["admin"])
async def admin_get_symbol(symbol: str, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        row = await conn.fetchrow("SELECT symbol, digits, contract_size, currency, margin_initial, margin_
        if not row:
            raise HTTPException(status_code=404, detail="Symbol not found.")
        return serialize_db_row(row)

@app.post("/admin/symbols", tags=["admin"])
async def admin_create_symbol(req: AdminSymbolCreate, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        try:
            await conn.execute(
                """INSERT INTO symbols (symbol, digits, contract_size, currency, margin_initial, margin_ma
                   VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)""",
                req.symbol, req.digits, Decimal(str(req.contract_size)), req.currency,
                Decimal(str(req.margin_initial)), Decimal(str(req.margin_maintenance)),
                req.spread_base, req.session_hours, req.settings_json or "{}"
            )
            return {"status": "SUCCESS", "symbol": req.symbol}
        except Exception as e:
            raise HTTPException(status_code=400, detail=str(e))

@app.put("/admin/symbols/{symbol:path}", tags=["admin"])
async def admin_update_symbol(symbol: str, req: AdminSymbolCreate, admin_auth: bool = Depends(verify_admin
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        if req.symbol != symbol:
            exist = await conn.fetchrow("SELECT symbol FROM symbols WHERE symbol = $1", req.symbol)
            if exist:
                raise HTTPException(status_code=400, detail=f"Symbol '{req.symbol}' already exists.")
            async with conn.transaction():
                await conn.execute("UPDATE positions SET symbol = $1 WHERE symbol = $2", req.symbol, symbo
                await conn.execute("UPDATE orders SET symbol = $1 WHERE symbol = $2", req.symbol, symbol)
                await conn.execute("UPDATE deals SET symbol = $1 WHERE symbol = $2", req.symbol, symbol)
```

```
                await conn.execute("UPDATE group_symbols SET symbol = $1 WHERE symbol = $2", req.symbol, s
                await conn.execute(
                    """UPDATE symbols SET symbol = $1, digits = $2, contract_size = $3, currency = $4, mar
                       margin_maintenance = $6, spread_base = $7, session_hours = $8, settings_json = $9 w
                    req.symbol, req.digits, Decimal(str(req.contract_size)), req.currency,
                    Decimal(str(req.margin_initial)), Decimal(str(req.margin_maintenance)),
                    req.spread_base, req.session_hours, req.settings_json or "{}", symbol
                )
            try:
                patch_ini_files(symbol, req.symbol, is_symbol=True)
            except Exception as e:
                logger.error(f"Failed to patch ini files for symbol rename: {e}")
            return {"status": "SUCCESS", "symbol": req.symbol}
        else:
            await conn.execute(
                """UPDATE symbols SET digits = $1, contract_size = $2, currency = $3, margin_initial = $4,
                   margin_maintenance = $5, spread_base = $6, session_hours = $7, settings_json = $8 WHERE
                req.digits, Decimal(str(req.contract_size)), req.currency,
                Decimal(str(req.margin_initial)), Decimal(str(req.margin_maintenance)),
                req.spread_base, req.session_hours, req.settings_json or "{}", symbol
            )
            return {"status": "SUCCESS", "symbol": symbol}

@app.delete("/admin/symbols/{symbol:path}", tags=["admin"])
async def admin_delete_symbol(symbol: str, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        await conn.execute("DELETE FROM symbols WHERE symbol = $1", symbol)
        return {"status": "SUCCESS", "message": f"Symbol '{symbol}' deleted."}

# ==================== ADMIN: GROUPS ====================

@app.get("/admin/groups", tags=["admin"])
async def admin_list_groups(admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        rows = await conn.fetch("SELECT name, max_leverage, margin_call, margin_stop_out, spread_override,
        return [serialize_db_row(r) for r in rows]

@app.get("/admin/groups/{group_name:path}", tags=["admin"])
async def admin_get_group(group_name: str, admin_auth: bool = Depends(verify_admin)):
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        row = await conn.fetchrow("SELECT name, max_leverage, margin_call, margin_stop_out, spread_overrid
        if not row:
            raise HTTPException(status_code=404, detail="Group not found.")

        overrides = await conn.fetch("SELECT symbol, spread_diff, commission_rate, margin_rate, trade_allo
        result = serialize_db_row(row)
        result["symbol_overrides"] = [serialize_db_row(o) for o in overrides]
        return result

def validate_group_name(name: str):
    name_lower = name.lower()
    allowed_prefixes = ("demo\\", "real\\", "context\\", "manager\\", "managers\\", "coverage\\")
    if not (name_lower.startswith(allowed_prefixes) or name == "demo_group"):
        raise HTTPException(
            status_code=400,
            detail="Group name must start with one of the allowed prefixes: "
                   "'demo\\', 'real\\', 'context\\', 'manager\\', 'managers\\', or 'coverage\\'."
        )

@app.post("/admin/groups", tags=["admin"])
async def admin_create_group(req: AdminGroupCreate, admin_auth: bool = Depends(verify_admin)):
    validate_group_name(req.name)
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        try:
```

```
            await conn.execute(
                """INSERT INTO groups (name, max_leverage, margin_call, margin_stop_out, spread_override,
                   VALUES ($1, $2, $3, $4, $5, $6)""",
                req.name, req.max_leverage, Decimal(str(req.margin_call)), Decimal(str(req.margin_stop_out
            )
            return {"status": "SUCCESS", "name": req.name}
        except Exception as e:
            raise HTTPException(status_code=400, detail=str(e))

@app.put("/admin/groups/{group_name:path}", tags=["admin"])
async def admin_update_group(group_name: str, req: AdminGroupCreate, admin_auth: bool = Depends(verify_adm
    validate_group_name(req.name)
    pool = DBConnection.get_pool()
    async with pool.acquire() as conn:
        if req.name != group_name:
            exist = await conn.fetchrow("SELECT name FROM groups WHERE name = $1", req.name)
            if exist:
                raise HTTPException(status_code=400, detail=f"Group with name '{req.name}' already exists.
            async with conn.transaction():
                await conn.execute("UPDATE accounts SET group_name = $1 WHERE group_name = $2", req.name,
                await conn.execute("UPDATE group_symbols SET group_name = $1 WHERE group_name = $2", req.n
                await conn.execute(
                    """UPDATE groups SET name = $1, max_leverage = $2, margin_call = $3, margin_stop_out =
                       spread_override = $5, settings_json = $6 WHERE name = $7""",
                    req.name, req.max_leverage, Decimal(str(req.margin_call)), Decimal(str(req.margin_stop
                    req.spread_override, req.settings_json or "{}", group_name
                )
            try:
                patch_ini_files(group_name, req.name, is_symbol=False)
            except Exception as e:
                logger.error(f"Failed to patch ini files for group rename: {e}")
            return {"status": "SUCCESS", "name": req.name}
        else:
            await conn.execute(
                """UPDATE groups SET max_leverage = $1, margin_call = $2, margin_stop_out = $3, spread_ove
                   WHERE name = $6""",
                req.max_leverage, Decimal(str(req.margin_call)), Decimal(str(req.margin_stop_out)), req.sp
            )


... [truncated for PDF size]
```